

深度学习（Deep Learning）Transformer

为什么我还是无法理解transformer？

网上几乎所有资料介绍transformer时都只是解释了Q K V的公式，然后基本就结束了。我看到现在还是不理解。神经网络很好理解，本质上是通过反向传... 显示全部

关注者

3,708

被浏览

1,491,222

关注问题

写回答

邀请回答

好问题

130

10 条评论

分享

...

[查看全部 317 个回答](#)



[IT 招财胖熊猫](#)  
信息技术行业 产品规划专家
+ 关注

 收录于 · [硬件工程师的AI之路](#)

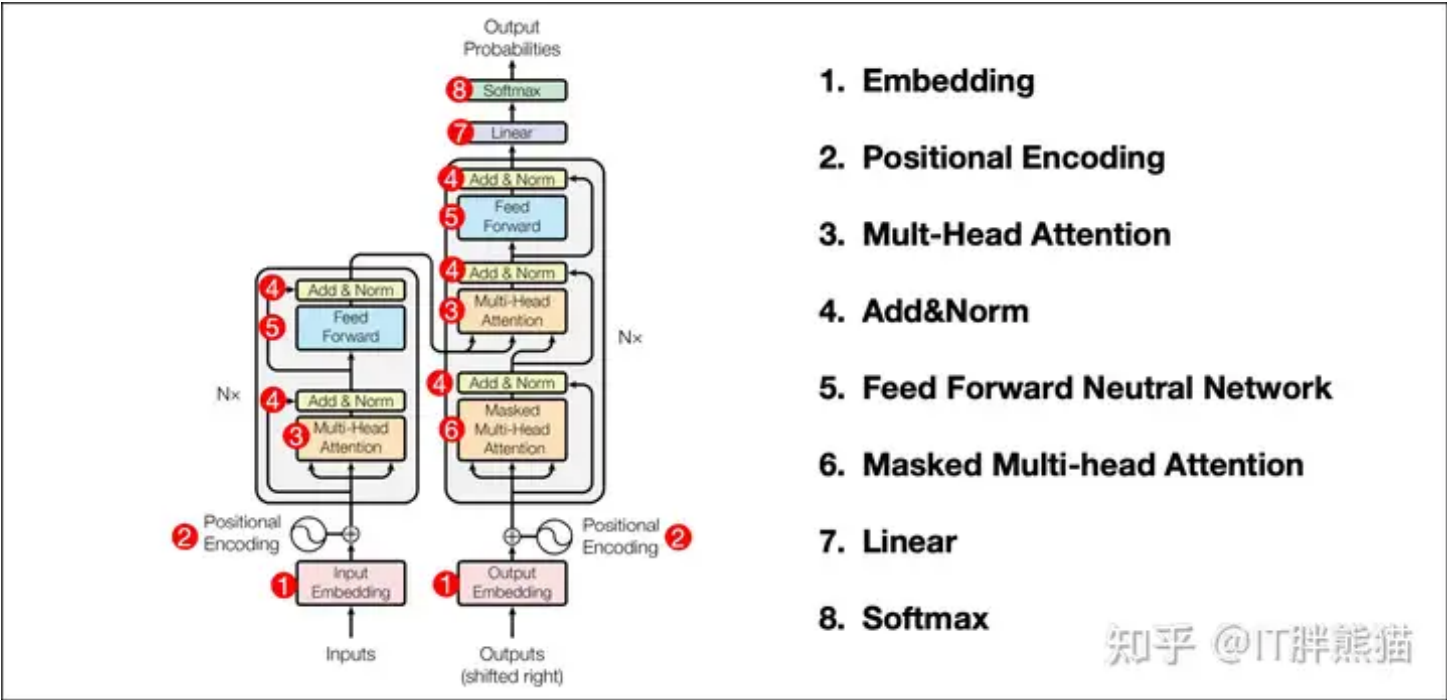
2942 人赞同了该回答
>
≡
目录

想彻底理解Transformer，我不得不提到用Excel手搓Transformer的[Yeh大神](#)，这个材料应该是他教学使用的，只是非常遗憾没有看到任何注释，我仔细阅读过，发现它对我理解Transformer非常有帮助，我把自己的注释贴在下面。



致敬Prof.Tom Yeh

我把自己的笔记贴在下面，如果有兴趣的观众也可以自行验算，相信对于理解Transformer非常有帮助，那我们开始~



Transformer架构图

参考：IT胖熊猫：从零基础到理解Transformer原理

1 背景信息

1.1 输入输出

输入:	小	狗	和	小	猫	玩
输出:	the	dog	and	cat	play	

背景: 输入输出的案例

1.2 Vocabulary字典

字典本质就是映射，即用一个数字向量来代表每一个输入的字词或者符号。这里涉及到Tokenize和Embedding的故事，这个后面会详细讲，这里简化了，只是意思一下，只有5维，例如：“狗”：[3,0,0,0,1,3]，实务中向量维度都非常大（可能是768甚至是1024）。

字典1

狗	猫	牛	大	中	小	吃	玩	跑	和	你	我	他	的	好	在
↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
3	0	2	1	2	3	2	2	3	0	1	0	0	0	0	1
0	3	0	3	0	2	2	1	1	3	0	1	1	0	1	0
0	1	1	2	3	1	1	3	0	1	1	0	1	1	0	1
1	1	3	3	3	3	2	1	3	2	1	1	0	1	1	0
3	1	0	3	0	2	3	2	2	2	0	0	0	1	0	0

字典2

[S]	a	and	apples	brings	by	cat	day	dog	eat	fill	friend	happy	house	laughter	light	music	night	play	quiet	smile	the	to	water
↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
3	0	2	1	2	3	2	2	3	0	2	2	0	2	2	2	0	0	0	2	2	0	3	3
0	3	0	3	0	2	2	1	1	3	3	1	0	1	0	3	2	0	1	1	1	3	3	0
0	1	1	2	3	1	1	3	0	1	0	0	3	1	3	2	1	3	1	0	3	2	3	0
1	1	3	3	3	3	2	1	3	2	0	2	2	3	2	3	1	2	1	2	1	0	1	2
3	1	0	3	0	2	3	2	2	2	3	2	1	3	2	3	2	3	1	2	1	0	1	2

字典

关于字典：Tokenization和Embedding两个阶段可以选择不同的字典，实际上很多大模型在用的是同一个字典，我们这里也一样。

1.3 流程说明

Transformer架构图是分左右两列的，左边的叫Encoder，右边的叫Decoder，数据流分别都是从下到上，每个Encoder/Decoder Block都是可以多层堆叠的，甚至可以选择同时实现两者或其中之一。

Encoder部分的输入是一次性输入的，即模型瞬间就看到了所有的输入，并同时并行计算完成了。随后，Encoder最后的输出也会分别进入Decoder每一层（这个会在Decoder中详解）。

Decoder部分和Encoder类似，但它是自回归模型，它的输入就是它是输出，你可能好奇，那第一个字符是啥？——当然是空的了，是个空的Token [S]（输出的来源自然来自Encoder部分的输入，需要仔

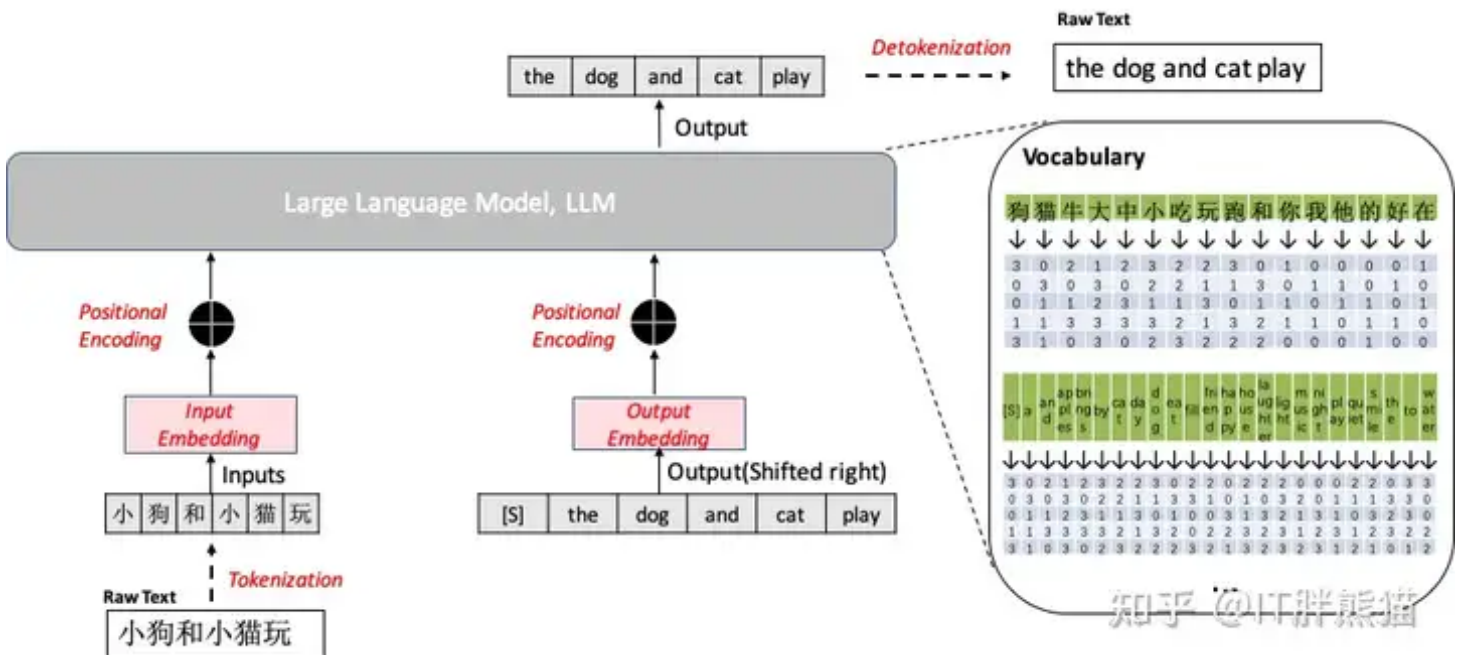
细理解)，你得等Decoder的第一个字符输出了，才能灌回Decoder的输入，可以看到，Transformer框架图里的Decoder输入注释成（shifted right），非常形象。

Decoder的输出其实就是基于Encoder的输入，和它每一个token的输出共同决定的，也就是要一个token一个token的输出。

2 编码

2.1 整体说明

这部分讲的是讲输出输出数据的预处理，如下图。



Tokenization, Embedding and Positional Encoding

输入其实有两部分，一部分是标准的输入，另一部分也是它本身的输出。Transformer的decoder部分是自回归，会将输出逐个的作为decoder的输入返回给Transformer，它是不需要考虑Tokenization的（严谨吧？）。

在正式进入大模型输入前，还要做三件事：

1. **Tokenize化**：将输入的raw text标准化。
2. **Embedding**：将自然语言向量化，可以得到一个矩阵，将输入的token用另一个维度来表示
3. **Positional Encoding**：导入位置信息。

2.2 tokenization

请参考我的回答：[NLP领域中的token和tokenization到底指的是什么？](#)

这部分比较简单，将输入分解为字典里的token，这部分就是按照字典展开，取決模型自己的设计，没有固定模式。

2.3 Embedding

请参考我的回答：[怎么形象理解embedding这个概念？](#)

请观察下面字典的例子：

例如“狗”可以用一个**Embedding向量**[3,0,0,0,1,3]来表示（当然实务中可能是原论文的 512 维甚至更高）

Vocabulary size

狗	猫	牛	大	中	小	吃	玩	跑	和	你	我	他	的	好	在
↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
3	0	2	1	2	3	2	2	3	0	1	0	0	0	0	1
0	3	0	3	0	2	2	1	1	3	0	1	1	0	1	0
0	1	1	2	3	1	1	3	0	1	1	0	1	1	0	1
1	1	3	3	3	3	2	1	3	2	1	1	0	1	1	0
3	1	0	3	0	2	3	2	2	2	0	0	0	1	0	0

Embedding维度

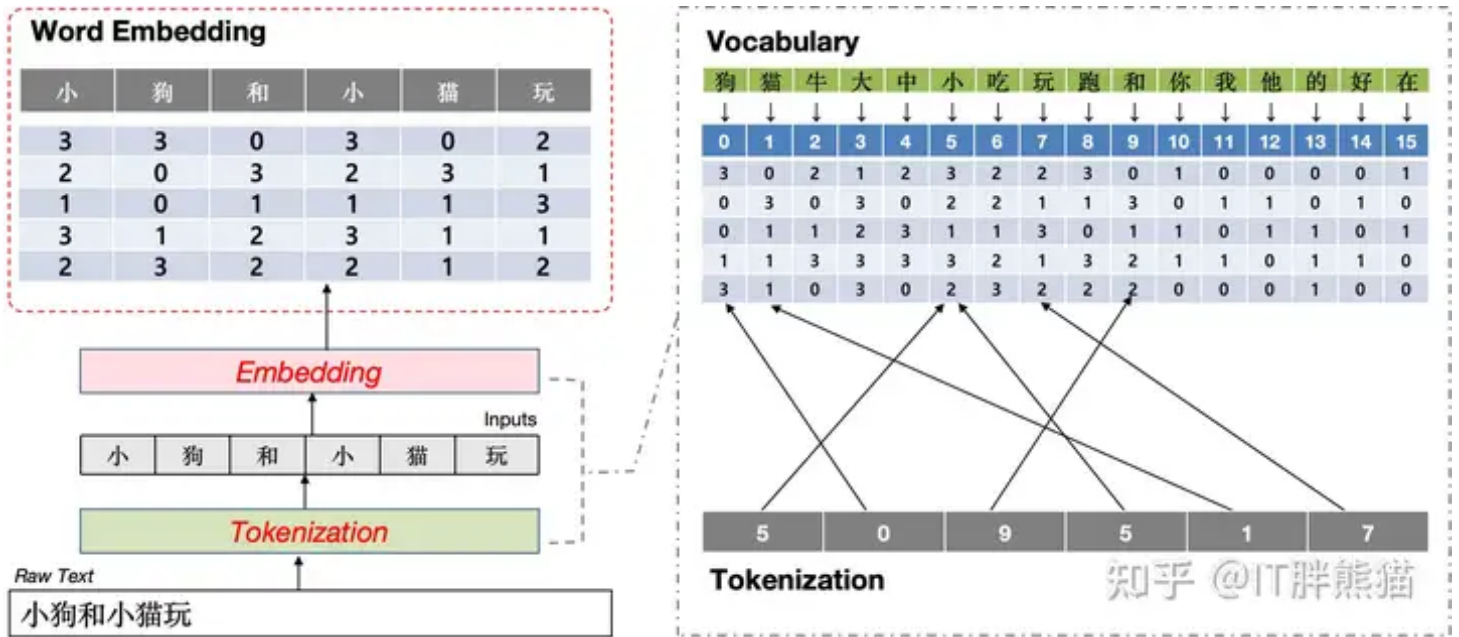
Vocabulary的例子

字典的词数叫**Vocabulary size**，其中单语言模型 30k-50k，多语言模型 100k-250k。

有必要多说一下这个维度：

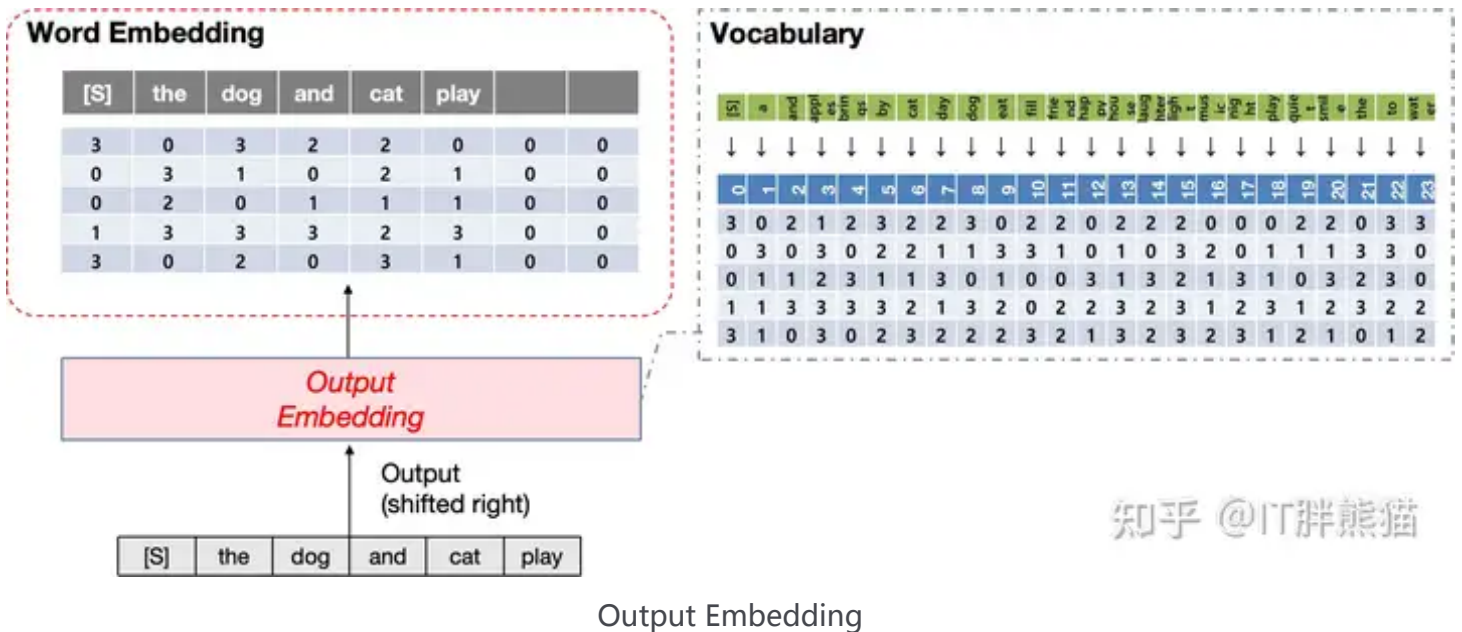
1. **embedding的维度**：表示用多少维的向量表示一个token。
2. **模型内部的标准维度d_model**：通常和embedding一致。
3. **如果不一致也没关系**，可以通过矩阵把它映射成同一维度（如本文就是这样的情况）

Encoder的embedding结果是一个矩阵，见于红色框线内。



Encoder部分的Tokenization & Embedding

Decoder的自回归输入的embedding结果也是一个矩阵，见于红色框线内。



Output Embedding

2.4 Positional Encoding

请参考我的回答：[Position Encoding 是怎么回事？](#)

注意力机制是不考虑Token位置的，这个就很可怕了，主语宾语都不分的话，那肯定无法正确理解意思的，每个词的位置信息也是很重要的，但是需要保证每个位置的PE编码不一样就行。

PE 的值可以直接按照位置顺序给到，当然也可以使用某种公式计算。Transformer 中采用了后者，即便是用公式，位置向量也有多种表示方式，其中一种常见的是正弦 - 余弦位置编码（Sinusoidal Position Encoding）。

公式如下（这个公式看着挺好的，比起单纯的数值，都不需要考虑位数的问题，容量无限扩展）：

$$PE_{pos,2i} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{pos,2i+1} = \cos(pos/10000^{2i/d_{model}})$$

我这里的 $d_{model} = 6$ ，一共 6 个 tokens：

6 tokens						Embedding 维度: 5
0.0	0.8415	0.9093	0.1411	-0.7568	-0.9589	
1.0	0.5403	-0.4161	-0.9899	-0.6536	0.2837	
0.0	0.0471	0.0379	0.0286	0.0191	0.0095	
1.0	0.9989	0.9993	0.9996	0.9998	0.9999	
0.0	0.0022	0.0043	0.0065	0.0086	0.0107	

正弦 - 余弦位置编码案例

Yeh老师给的数据显然没有考虑正弦-余弦公式（我也没猜出来是什么逻辑得到的，但其实不影响结果）。

Word Embedding:

3	3	0	3	0	2
2	0	3	2	3	1
1	0	1	1	1	3
3	1	2	3	1	1
2	3	2	2	1	2

+

=

3.922	3.987	0.998	4	1	3
2.715	0.312	3.126	2.05	3.02	1.008
1.368	0.889	1.982	1.997	2	4
4	1.592	2.25	3.1	1.04	1.016
1.592	3.702	2.951	2.992	1.999	3

Positional Encoding:

t	0	1	2	3	4	5
d	0.9217964	0.987	0.998	1	1	1
1	0.7147135	0.312	0.126	0.05	0.02	0.008
2	0.3676445	0.889	0.982	0.997	1	1
3	0.999766	0.592	0.25	0.1	0.04	0.016
4	-0.407523	0.702	0.951	0.992	0.999	1

知乎 @IT胖熊猫

Encoder部分Positional Encoding

Word Embedding:

3	0	3	2	2	0	0	0
0	3	1	0	2	1	0	0
0	2	0	1	1	1	0	0
1	3	3	3	2	3	0	0
3	0	2	0	3	1	0	0
1	1	1	2	1	2	0	0

+

=

4	0.894	4	2.977	2.989	0.995	0.998	0.999
1	3.801	2	0.418	2.289	1.199	0.136	0.093
#	2.177	1	1.798	1.905	1.955	0.979	0.99
1	3.959	4	3.759	2.554	3.389	0.269	0.185
2	-0.68	2	0.474	3.743	1.878	0.943	0.973
0	1.349	2	2.962	1.771	2.565	0.397	0.275

Positional Encoding:

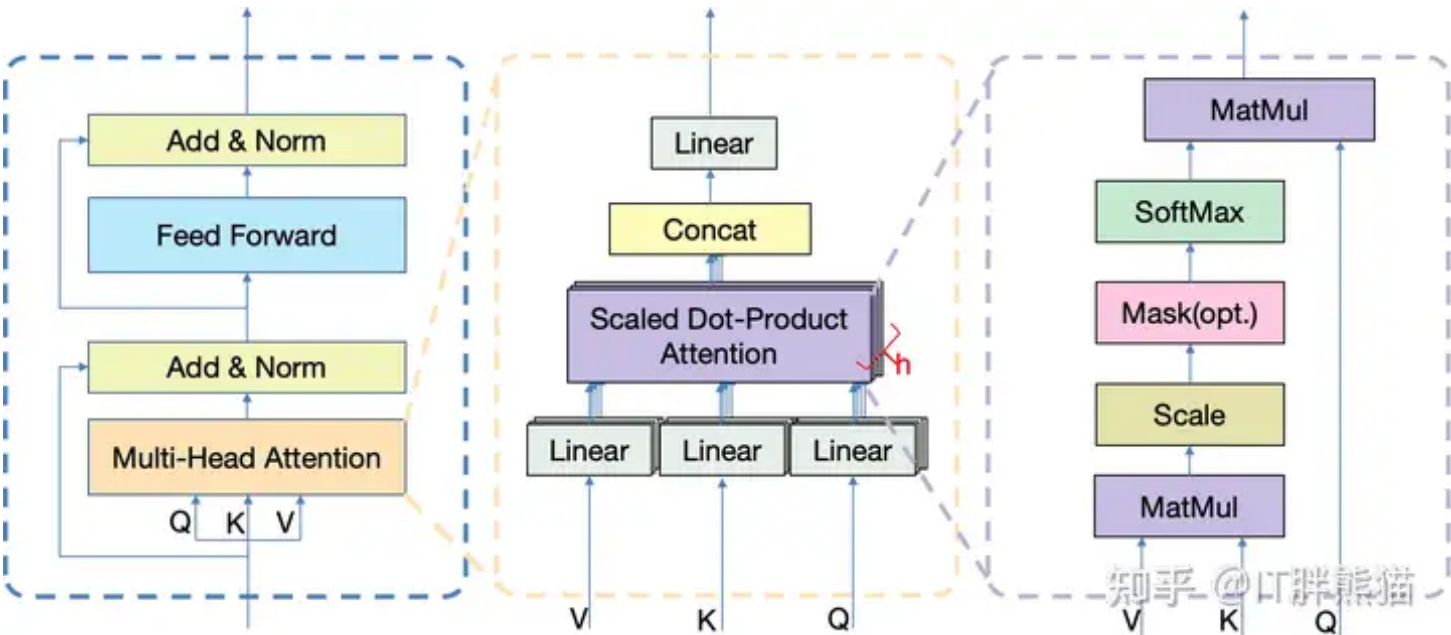
t	0	1	2	3	4	5	6	7
d								
0	0.777	0.894	0.95	0.977	0.989	0.995	1	0.999
1	0.978	0.801	0.591	0.418	0.289	0.199	0.1	0.093
2	-0.46	0.177	0.583	0.798	0.905	0.955	1	0.99
3	0.404	0.959	0.954	0.759	0.554	0.389	0.3	0.185
4	-0.97	-0.68	-0.01	0.474	0.743	0.878	0.9	0.973
5	-0.81	0.349	0.947	0.962	0.771	0.565	0.4	0.275

知乎 @IT胖熊猫

Decoder部分Positional Encoding

3 Encoder

3.1 Encoder整体说明



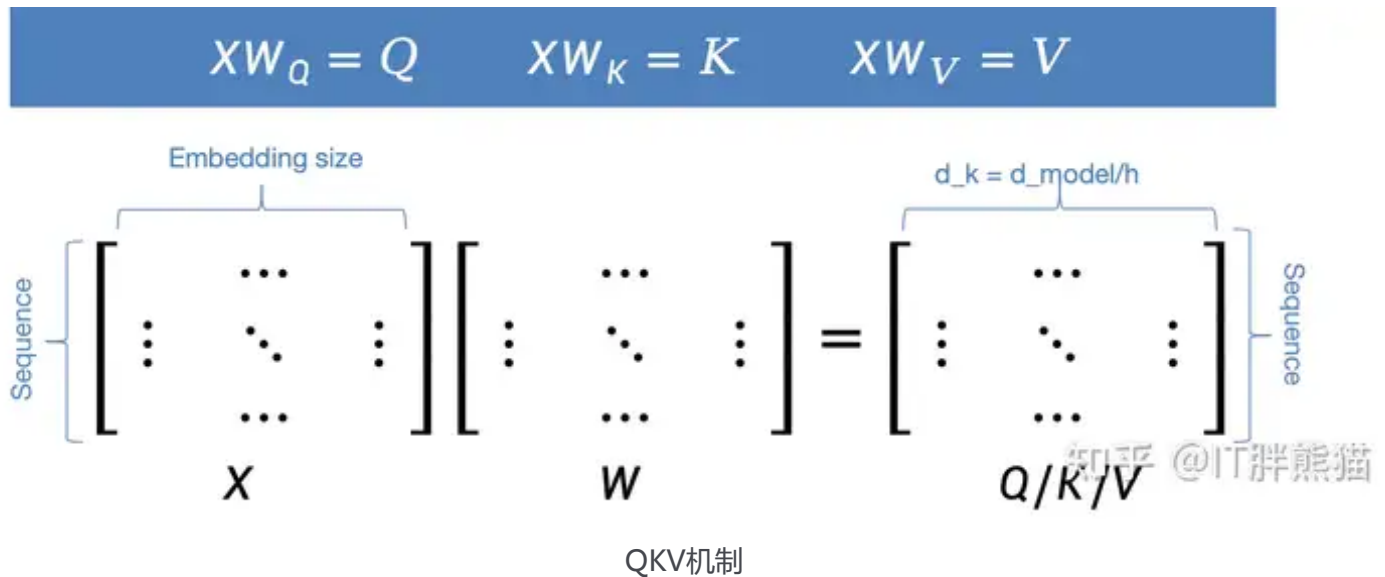
Transformer Encoder部分说明

Transformer的Encoder部分总共分两个阶段，先做多头注意力，再做FFN，这两步结束后都要做Add & Norm，主要的功能就是做归一化和标准化，提升模型的稳定性和训练速度。

需要专注的是多头注意力机制（[Multi-Head Attention](#)，MHA），它的结果其实是多个注意力头最后拼接（concat）而成的，对于每个注意力头的机制都有详细的过程。

Encoder部分如果理解清楚了，Decoder就非常容易了。

3.2 Mult-HeadAttention



这里可以理解为将每个输入向量（维度为embedding）映射到三个键向量（维度为 $d_{\text{model}}/\text{head}$ ）。

- **Query（查询）**：代表当前关注的位置（如当前处理的单词）。
- **Key（键）**：代表序列中所有位置的标识，用于与 Query 匹配相似性。
- **Value（值）**：存储实际的信息，通过注意力权重加权后生成最终输出。

$XW = Q \quad (XW)^T = W^T X^T = Q^T$

Q的计算

Yeh老师的材料里都是做了转置的，我已经用公式做了说明，后面每个计算都是类似的方法，就没有一一赘述了（以防我有的地方忘了解释）。

后面就是 d_{model} 了，这里是 $d_{model} = 6$ ，你可能不信，那就看下下面的 W_q 矩阵结构就清楚了，这里有 2 个注意力头，每个注意力头是 3 维，等于将embedding的维度用 $2 \times 3 = 6$ 维来代替了（这里如果你如果没有完全理解，后面走完全流程你就能完全理解了）。

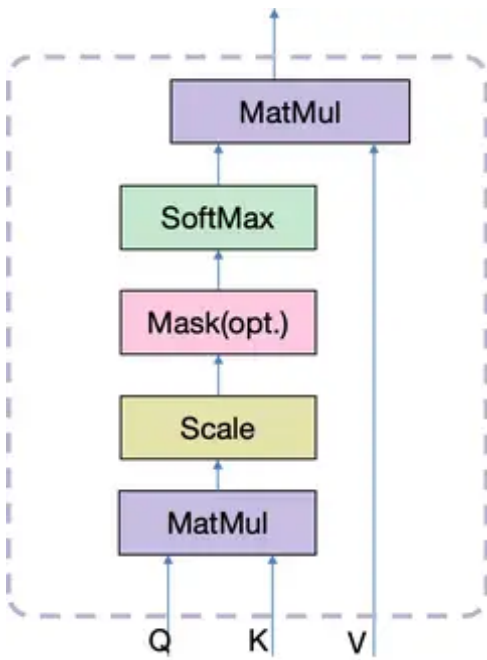
Head		Wq					
0		-1	-1	1	1	1	→
		1	-1	-1	0	-1	→
		-1	-1	0	0	1	→
1		0	1	1	1	1	→
		0	1	-1	1	1	→
		-1	1	0	-1	0	→

注意力头的结构

同理得到K、V。

K	K^T					V	V^T				
0.323	1.884	3.059	2.04	1.018	4.008	6.861	7.112	5.092	7.045	4.019	3.008
-1.75	-0.92	-7.06	-3.04	-6.02	-5.01	6.714	1.904	5.376	5.151	4.06	2.024
-5.04	-0.6	-1.17	-3.06	-2.02	-1.01	-1.68	-3.31	-5.81	-3.94	-5.98	-6.99
-1.06	1.532	1.845	-0.06	1.979	-1.01	1.753	0.915	7.061	3.04	6.018	5.008
1.347	-0.58	1.144	0.053	1.02	-2.99	-1.51	-6.1	-1.7	-3.89	-1.96	-4.98
-5.21	-5.27	-0.12	-5.05	0.98	-3.01	-5.37	-2.48	-4.23	-5.1	-3.04	-3.02

K、V结果



$$Attention(Q, K, V) = \text{Softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

对当前 Query 和所有的 Key 计算相似度，将这个相似度值通过 Softmax 进行得到一组权重，根据这组权重与对应 Value 乘积求和得到 Attention 下 Value 值。

Scaled Dot-Product Attention 的实际意义在于它提供了一种机制，使模型能够在处理输入数据时动态地分配注意力，捕获上下文信息，提高计算效率，并提供一定程度的可解释性。

scaled Dot-Product Attention 说明

Q、K、V 的计算过程，材料中也是做了转置的，参见下面。

$$Attention(Q, K, V) = \text{Softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

$$Attention(Q, K, V)^T = \left(\text{Softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \right)^T = V^T \text{Softmax} \left(\frac{KQ^T}{\sqrt{d_k}} \right)$$

Scaled Dot-Production Attention 公式

从 Head 0 开始，看看怎么计算 Scaled Dot-Production Attention。

Head 0为例:

			Q						
			0.32	1.88	3.059	2.04	1.018	4.01	
			-1.8	-0.9	-7.06	-3	-6.02	-5	
			-5	-0.6	-1.17	-3.1	-2.02	-1	
K^T			K^T * Q / sqrt(dk)						
0.32	-1.8	-5	→	16.5	3.02	11.13	12.4	12.17	8.75
1.88	-0.9	-0.6	→	3.02	2.74	7.464	4.88	4.985	7.35
3.06	-7.1	-1.2	→	11.1	7.46	34.98	18.1	27.7	28.2
2.04	-3	-3.1	→	12.4	4.88	18.06	13.1	15.33	15.3
1.02	-6	-2	→	12.2	4.99	27.7	15.3	23.87	20.9
4.01	-5	-1	→	8.75	7.35	28.17	15.3	20.93	24.3

按列做Softmax, 相当于对每个token去看

t	0	1	2	3	4	5
0	0.968	0.01	4E-11	0.003	2E-07	4E-09
1	1E-06	0	1E-12	2E-06	1E-10	9E-10
2	0.004	0.48	0.998	0.879	0.978	0.978
3	0.015	0.04	5E-08	0.006	4E-06	2E-06
4	0.012	0.04	0.001	0.057	0.021	0.001
5	4E-04	0.43	0.001	0.055	0.001	0.021

Softmax

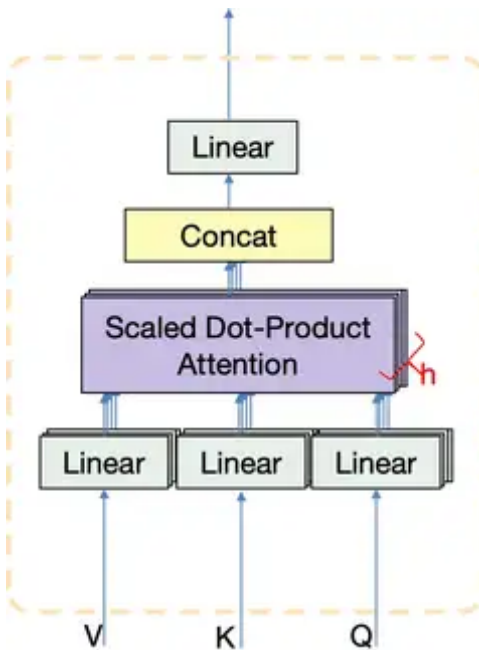
×

6.819	4.24	5.089	4.935	5.067	5.048
6.65	3.86	5.371	5.12	5.344	5.304
-1.78	-6.2	-5.81	-5.86	-5.81	-5.83

V					
6.86	7.11	5.092	7.04	4.019	3.01
6.71	1.9	5.376	5.15	4.06	2.02
-1.7	-3.3	-5.81	-3.9	-5.98	-7

Scaled Dot-Production Attention公式

同理可以计算 Head 1, 得到结果拼接后做多头注意力。



Multi-head Attention其实就是多个Self-Attention结构的结合, 每个head学习到在不同表示空间中的特征。

$$MultiHead(Q, K, V) = Concat(H_1, H_2, \dots, H_h) W^0$$

$$H_i = Attention(QW_i^Q, KW_i^K, VW_i^V), i = 1, \dots, h$$

知乎 @IT胖熊猫

多头注意力

也需要转置。

$$\text{MultiHead}(Q, K, V) = \text{Concat}(H_1, H_2, \dots, H_h) W^0$$

$$\text{MultiHead}(Q, K, V)^T = W^{0T} \text{Concat}(H_1, H_2, \dots, H_h)^T$$

知乎 @IT胖熊猫

多头注意力公式

接着做线性变换，将拼接的矩阵维度，从 $h \times d_v = 2 \times 3 = 6$ ，重新投影到 5 维矩阵，这样的结果就和输入很像了，每列代表一个token的注意力结果。

你可能有这个问题：为啥要做维度变换呢？（这里我只解释一次，后面会有很多次相同的操作。）

答案：因为它需要做add，即把最初的输入加进去，这样下一层就相当于得到了本层的结果和最初的输入信息，免得丢失信息，既然要做加法，自然结果要变成和输入一致。

Linear					
0	0	1	1	0	-1
1	1	1	1	1	-1
-1	-1	-1	-1	-1	1
-1	0	-1	1	0	1
-1	-1	-1	-1	0	1

×

6.819	4.24	5.089	4.935	5.067	5.048
6.65	3.86	5.371	5.12	5.344	5.304
-1.78	-6.2	-5.81	-5.86	-5.81	-5.83
0.925	0.94	6.468	0.922	6.306	0.93
-6.09	-6	-1.85	-6.09	-1.89	-6.09
-2.49	-2.5	-3.55	-2.48	-3.37	-2.49

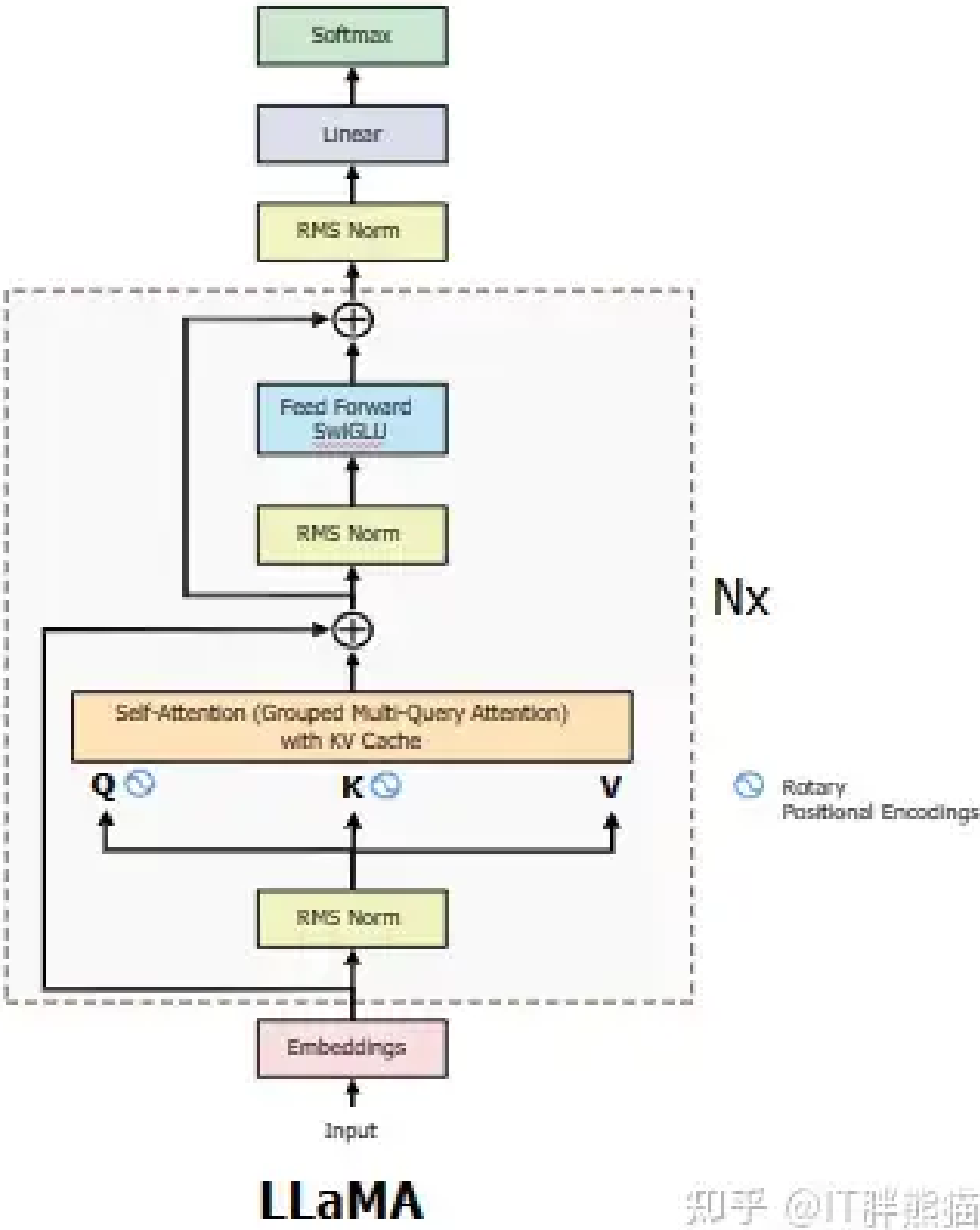
=

1.63	-2.8	4.213	-2.45	3.862	-2.42
9.014	-0.7	12.83	1.51	12.39	1.847
-9.01	0.7	-12.8	-1.51	-12.4	-1.85
-6.6	0.4	3.634	-0.64	3.684	-0.77
-15.1	-5.3	-14.7	-7.6	-14.3	-7.93

线性变换

3.3 Add&Norm

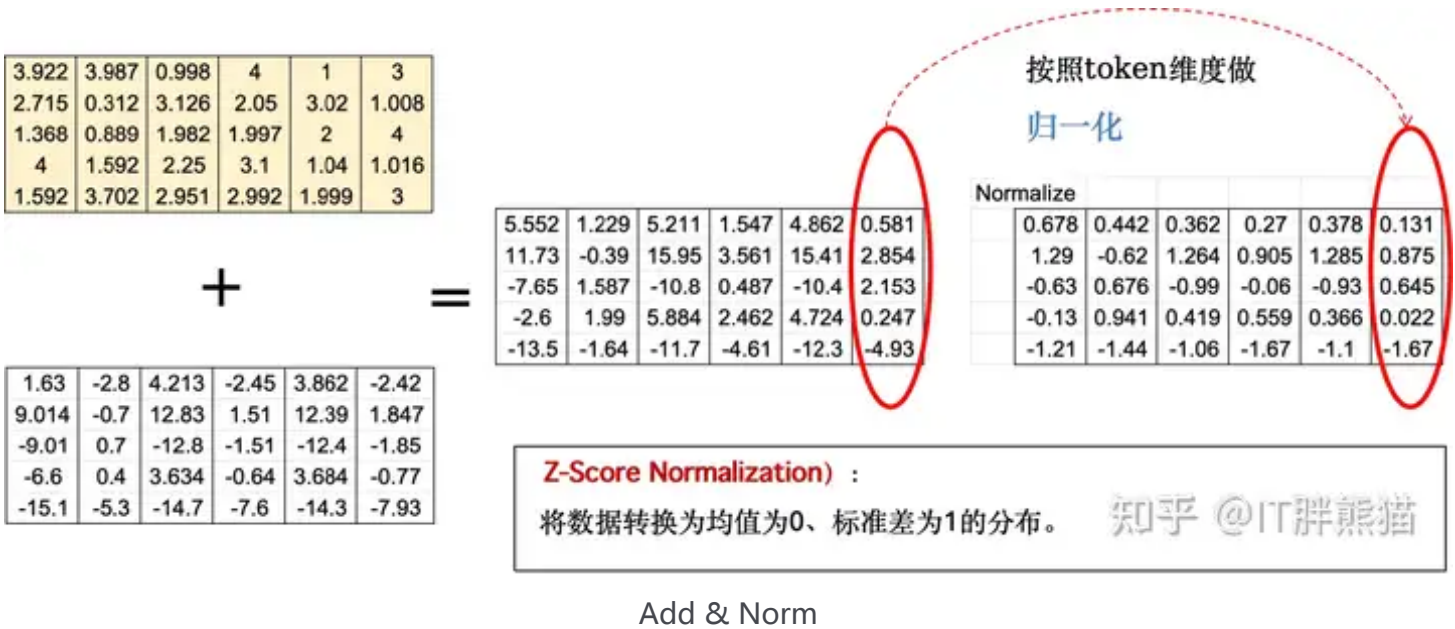
这个Add & Norm层其实是神经网络网络中的一个基本操作，目的只是为了训练的稳定来做的，在实务中，很多大型神经网络的Norm是在Attention之前做的，例如LLaMa，它是先做Norm再做Add的。



LLaMa架构 | 来源：网络

再说**Add**，它被叫做**残差连接**，是一种在深度神经网络中非常常用的技术，将输入直接传递到输出，与经过网络层变换后的结果相加，目的就是让多层的神经网络也能感受到最初的输入。

残差连接 ([Residual Connection](#)) 最早由何恺明等人在2015年提出的ResNet (残差网络) 中引入。它的核心思想是通过引入“跳跃连接” (Skip Connection) , 从而缓解深度网络中的梯度消失或爆炸问题。

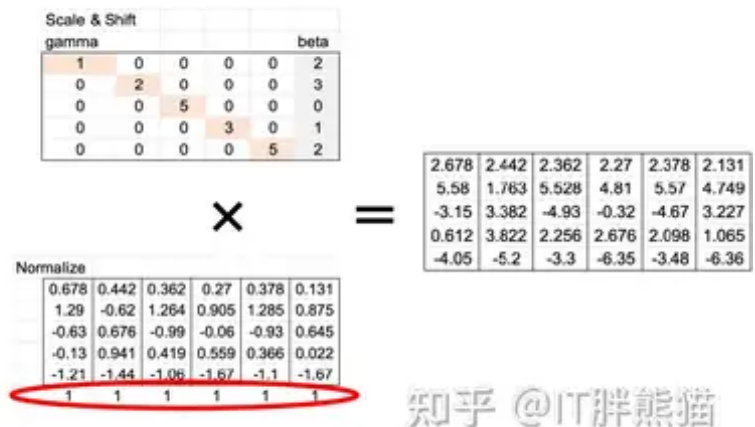


另外在Add & Norm层中还有个操作, 叫做Scaled & shift (缩放和偏移), 其核心作用是对输入数据进行标准化处理, 以提高模型的稳定性和训练效率。

就是缩放+偏移值, 效果就是提升模型的稳定度, 这个是深度学习的技巧了, 可以忽略。

- γ (Scale) : 缩放参数, 维度为 (d_model,), 初始化为全1。
- β (Shift) : 偏移参数, 维度为 (d_model,), 初始化为全0。

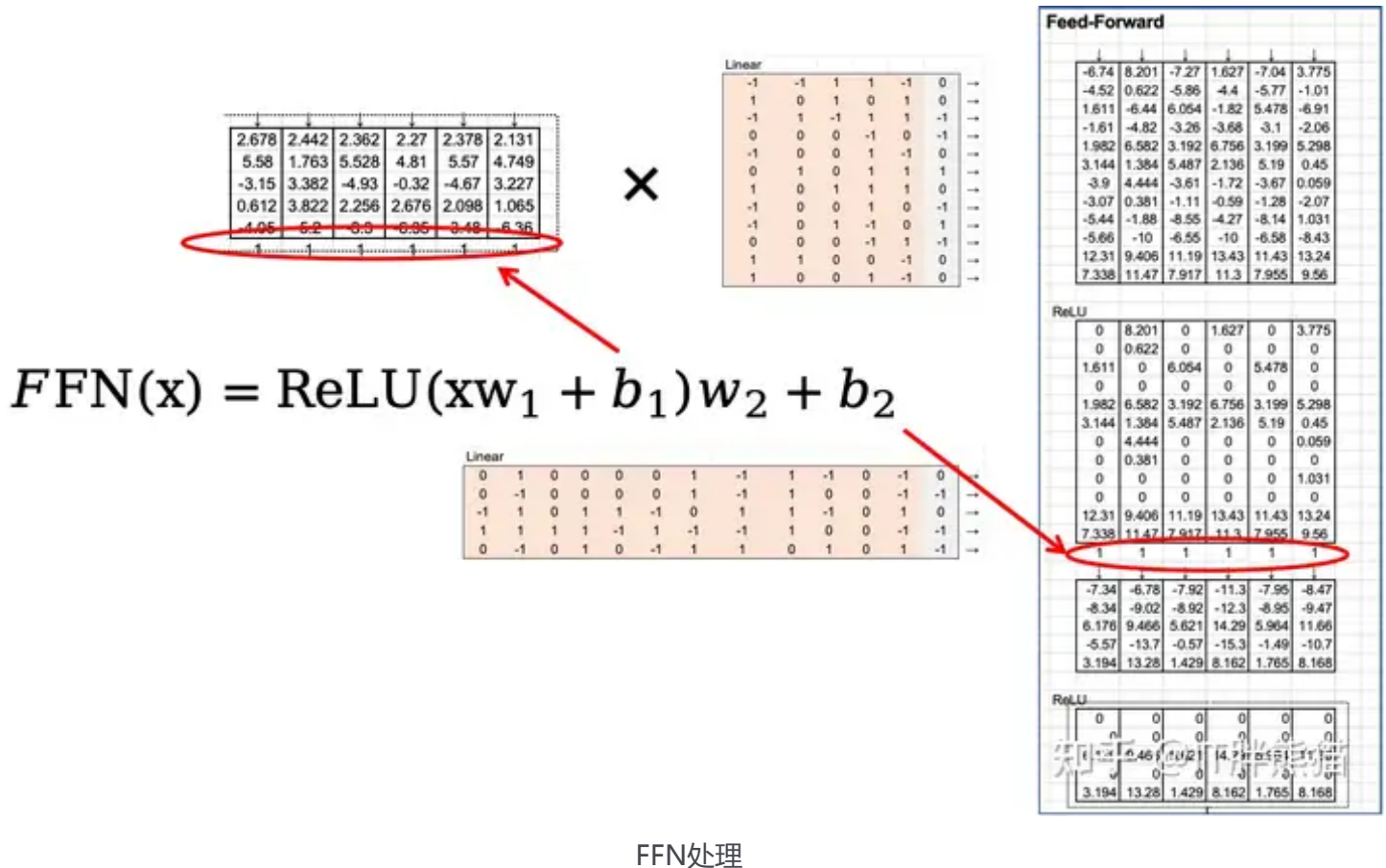
$$Output = \gamma \odot \frac{x - \mu}{\sigma} + \beta$$



3.4 [Feed Forward Neutral Network](#)+

以上注意力就完成了，我们走到下面的FFN（涉及到深度学习的部分我们先不展开，只要知道它是全互联就好了。）

1. **第一层全连接**：将输入升维到 $4 \times d_{model}$ ，这里放大 2 倍。
2. **ReLU**：引入非线性，过滤负值。
3. **第二层全连接**：降维回 d_{model} 。



全连接要做的事就是不要丢掉信息，并提升表达能力，所以这里要放大维度，然后在整理会标准的维度处理，实际是个数学问题，我简单给点背景信息，不用纠结，知道大概要怎么算就好，变换里带了两个偏移量，都是为了提升模型稳定度的。

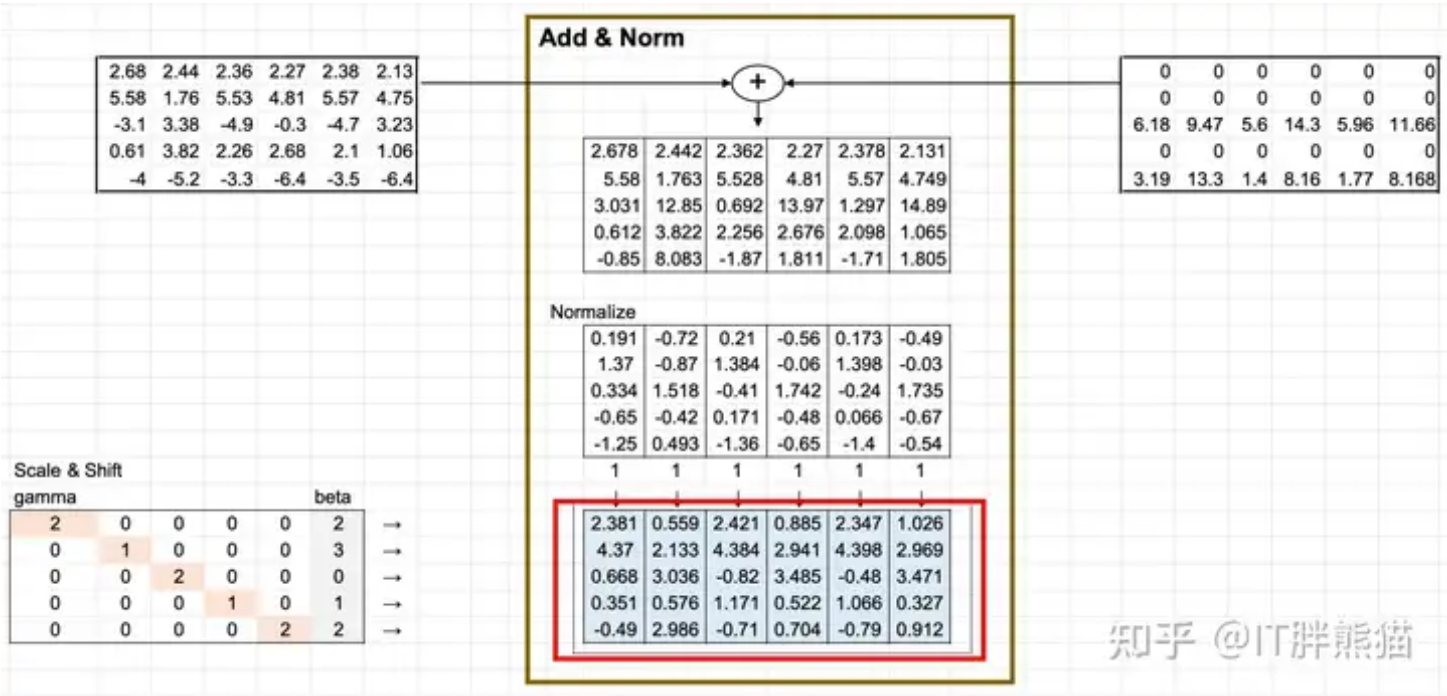
b_1 提供了额外的自由度，允许模型对输入进行平移修正，增强对数据分布的适应性，虽然理论上可以赋0，但在实践中必须保留它。偏置项是神经网络中实现灵活特征变换的核心组件，移除它会损害模型的表达能力。这种设计是深度学习模型在大量实验验证后的最佳实践。

b_2 的作用虽不如 b_1 关键，但它提供了输出层的平移自由度，帮助模型适配目标分布偏移。实践中应保留除非在极端资源受限的场景中且实验证明移除后性能损失可接受。

3.5 Add&Norm

还是Add & Norm，再来一次Scaled & Shift。

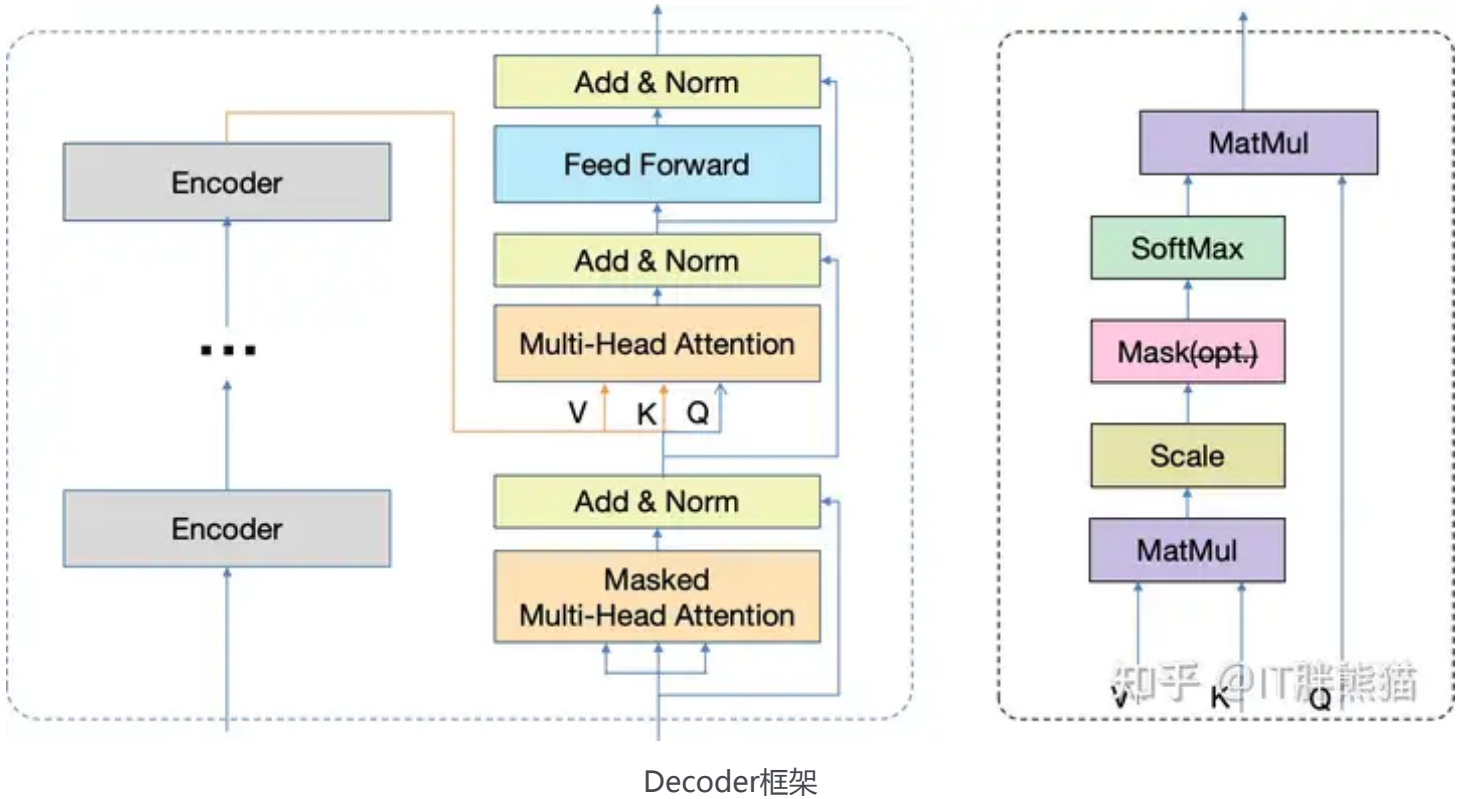
得到的结果见红框，它是给Decoder作为输出的，它和输入的维度是一致的，就像刚刚embedding完。



Encoder最后输出

4 Decoder

4.1 Decoder 框架



Decoder部分和Encoder非常相似，可以看到除了前面的Masked MHA以外，和Encoder一模一样的，所以 **Decoder = Masked MHA + Encoder**。

Masked MHA输出只负责下一层“Encoder”的Q，其中的K、V值来自Encoder的最后输出，所以对于我们讲解过的Encoder部分一带而过。

4.2 Masked Multi-Head Attention

其实Masked MHA的核心还是scaled dot-production attention，还是从制备Q、K和V开始。

$$XW = Q \quad (XW)^T = W^T X^T = Q^T$$

Head		W _Q																		
0		<table><tr><td>-1</td><td>-1</td><td>1</td><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>-1</td><td>-1</td><td>0</td><td>0</td><td>-1</td></tr><tr><td>-1</td><td>-1</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table>	-1	-1	1	1	0	1	1	-1	-1	0	0	-1	-1	-1	0	0	0	1
	-1	-1	1	1	0	1														
	1	-1	-1	0	0	-1														
-1	-1	0	0	0	1															
1		<table><tr><td>0</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td><td>1</td></tr><tr><td>-1</td><td>1</td><td>0</td><td>-1</td><td>1</td><td>0</td></tr></table>	0	1	1	1	1	1	0	1	0	1	0	1	-1	1	0	-1	1	0
	0	1	1	1	1	1														
	0	1	0	1	0	1														
-1	1	0	-1	1	0															
2		<table><tr><td>1</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td></tr><tr><td>-1</td><td>1</td><td>-1</td><td>1</td><td>0</td><td>1</td></tr><tr><td>0</td><td>0</td><td>-1</td><td>-1</td><td>0</td><td>1</td></tr></table>	1	0	0	0	1	1	-1	1	-1	1	0	1	0	0	-1	-1	0	1
	1	0	0	0	1	1														
	-1	1	-1	1	0	1														
0	0	-1	-1	0	1															
		W _K																		
0		<table><tr><td>-1</td><td>-1</td><td>1</td><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>-1</td><td>-1</td><td>0</td><td>1</td><td>-1</td></tr><tr><td>-1</td><td>-1</td><td>0</td><td>1</td><td>0</td><td>1</td></tr></table>	-1	-1	1	1	0	1	1	-1	-1	0	1	-1	-1	-1	0	1	0	1
	-1	-1	1	1	0	1														
	1	-1	-1	0	1	-1														
-1	-1	0	1	0	1															
1		<table><tr><td>0</td><td>1</td><td>-1</td><td>-1</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td><td>-1</td><td>0</td></tr></table>	0	1	-1	-1	0	1	0	1	0	1	0	0	0	1	1	0	-1	0
	0	1	-1	-1	0	1														
	0	1	0	1	0	0														
0	1	1	0	-1	0															
2		<table><tr><td>1</td><td>0</td><td>1</td><td>0</td><td>0</td><td>-1</td></tr><tr><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td><td>1</td></tr><tr><td>0</td><td>0</td><td>1</td><td>-1</td><td>0</td><td>1</td></tr></table>	1	0	1	0	0	-1	0	1	0	1	0	1	0	0	1	-1	0	1
	1	0	1	0	0	-1														
	0	1	0	1	0	1														
0	0	1	-1	0	1															
		W _V																		
0		<table><tr><td>1</td><td>1</td><td>-1</td><td>0</td><td>1</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>-1</td><td>0</td><td>1</td><td>1</td><td>0</td></tr></table>	1	1	-1	0	1	1	0	1	0	1	0	0	0	-1	0	1	1	0
	1	1	-1	0	1	1														
	0	1	0	1	0	0														
0	-1	0	1	1	0															
1		<table><tr><td>-1</td><td>1</td><td>1</td><td>0</td><td>0</td><td>1</td></tr><tr><td>-1</td><td>0</td><td>0</td><td>1</td><td>0</td><td>-1</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td><td>0</td><td>0</td></tr></table>	-1	1	1	0	0	1	-1	0	0	1	0	-1	0	1	1	0	0	0
	-1	1	1	0	0	1														
	-1	0	0	1	0	-1														
0	1	1	0	0	0															
2		<table><tr><td>1</td><td>1</td><td>0</td><td>0</td><td>1</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td><td>1</td><td>1</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td><td>-1</td><td>1</td><td>2</td></tr></table>	1	1	0	0	1	1	0	1	0	1	1	0	1	1	0	-1	1	2
	1	1	0	0	1	1														
	0	1	0	1	1	0														
1	1	0	-1	1	2															

×

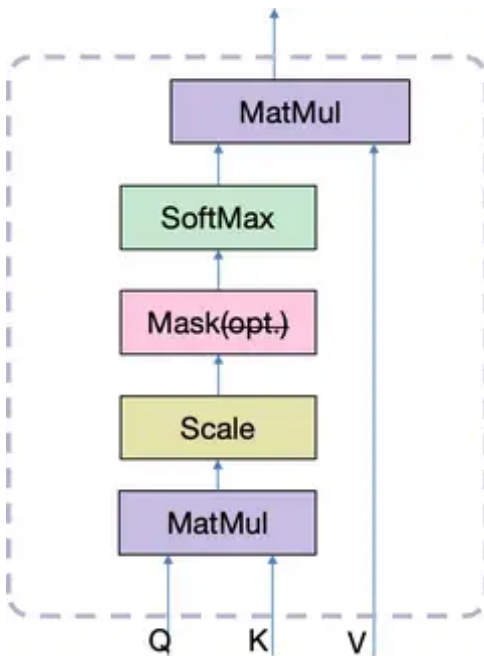
4	0.894	4	2.977	2.989	0.995	0.998	0.999
1	3.801	2	0.418	2.289	1.199	0.136	0.093
#	2.177	1	1.798	1.905	1.955	0.979	0.99
1	3.959	4	3.759	2.554	3.389	0.269	0.185
2	-0.68	2	0.474	3.743	1.878	0.943	0.973
0	1.349	2	2.962	1.771	2.565	0.397	0.275

=

Q	4	2.791	0.9	5.124	0.951	5.716	0.512	0.358
3.1	-6.43	-0	-2.2	-2.98	-4.72	-0.51	-0.36	
-5	-3.35	-4	-0.43	-3.51	0.371	-0.74	-0.82	
4.2	10.6	10	9.41	12.26	10.99	2.724	2.516	
2.6	9.109	7.5	7.138	6.614	7.153	0.802	0.552	
-2	-1.73	-4	-5.84	0.489	-1.31	-0.19	-0.12	
6	1.562	7.9	6.412	8.503	5.437	2.338	2.247	
-1	6.038	3	2.363	1.721	4.202	-1.17	-1.44	
-1	-4.79	-3	-2.6	-2.69	-2.78	-0.85	-0.9	
K	4	2.791	0.9	5.124	0.951	5.716	0.512	0.358
5.1	-7.11	1.8	-1.73	0.767	-2.85	0.428	0.614	
-3	0.614	0.4	3.326	-0.95	3.76	-0.47	-0.63	
0.2	-0.99	-1	-2.18	-0.4	-1.58	-0.72	-0.81	
2.4	7.76	5.5	4.177	4.843	4.588	0.405	0.277	
-2	6.66	0.2	1.742	0.451	1.276	0.172	0.11	
3.1	1.722	2.6	1.814	3.123	0.386	1.570	1.714	
2.6	9.109	7.5	7.138	6.614	7.153	0.802	0.552	
-2	-0.43	-1	1.001	1.122	1.131	1.108	1.081	
V	4	3.185	8.9	5.032	8.888	4.681	1.494	1.349
2.4	7.76	5.5	4.177	4.843	4.588	0.405	0.277	
2.5	-0.52	4.4	3.815	4.007	4.068	1.076	1.065	
-3	6.433	0.2	2.201	2.976	4.724	0.515	0.359	
-3	1.716	-2	-2.18	-2.21	-0.17	-1.13	-1.09	
0.5	5.97	2.2	2.218	4.195	4.195	1.111	1.111	
7	5.367	1.5	6.18	10.79	6.436	2.771	3.24	
4.4	7.078	7.5	4.65	8.586	6.466	1.348	1.25	
2.6	10.25	8	10.64	6.983	7.401	1.965	1.659	

计算Q、K、V

得到了Q、K、V之后，进行Masked MHA计算，数学上只要做个矩阵加法，即增加一个极大负值（这里需要注意转置的问题）。



$$Attention(Q, K, V) = \text{Softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V + \text{Casual Mask}$$

和Encoder的Scaled Dot-Production Attention相似，只不过在softmax前，将无效位置（如未来词或填充位置）设为极大负值，防止模型“作弊”（如解码时看到未来信息），或处理变长序列（如填充部分不参与计算）

知乎 @IT胖熊猫

Scaled dot-product attention打开Mask

看下实际的计算，依然是以Head 0 为例，注意Casual Mask也做了转置。需要注意的是这里使用了 3 个注意力头，后面还会切换成 2 个注意力头，非常灵活。

Casual Mask	t	0	1	2	3	4	5	6	7
	0	0	0	0	0	0	0	0	0
	1	-99	0	0	0	0	0	0	0
	2	-99	-99	0	0	0	0	0	0
	3	-99	-99	-99	0	0	0	0	0
	4	-99	-99	-99	-99	0	0	0	0
	5	-99	-99	-99	-99	-99	0	0	0
	6	-99	-99	-99	-99	-99	-99	0	0
	7	-99	-99	-99	-99	-99	-99	-99	0

Head	0
Q	
	-3.617 2.7911 0.9419 5.1243 0.9511 5.7157 0.5123 0.36
	3.0652 -6.433 -0.171 -2.201 -2.976 -4.724 -0.515 -0.4
	-4.566 -3.346 -3.594 -0.433 -3.507 0.371 -0.736 -0.8

K^T	
	-3.6 5.1 -3.2
	2.79 -7.1 0.61
	0.94 1.82 0.36
	5.12 -1.7 3.33
	0.95 0.77 -1
	5.72 -2.8 3.76
	0.51 0.43 -0.5
	0.36 0.61 -0.6

K^T * Q / sqrt(dk) + casual mask	
	24.916 -18.66 4.0919 -16.39 -4.346 -26.52 -1.242 -0.3
	-119 29.736 0.9451 17.144 12.514 28.745 2.6795 1.76
	-98.7 -104.9 -0.412 0.3858 -3.335 -1.776 -0.415 -0.4
	-121.5 -90.75 -102.9 16.523 -0.954 22.332 0.6156 -0.2
	-97.12 -98.47 -96.58 -96.92 1.1359 0.8436 0.4586 0.49
	-125.9 -86.48 -103.4 -79.41 -98.59 27.429 0.9386 -0
	-98.08 -98.86 -97.79 -97.91 -98.51 -98.58 0.2228 0.24
	-96.99 -99.48 -97.55 -98.56 -98.58 -99.63 -98.81 0.24

V	
	7.4345 3.1849 8.8956 5.0316 8.8877 4.6806 1.4944 1.35
	2.3829 7.76 5.5447 4.1767 4.8433 4.5881 0.405 0.28
	2.461 -0.523 4.3521 3.8151 4.0072 4.0683 1.0759 1.07

t	0	1	2	3	4	5	6	7
	1	1E-21	0.949	2E-15	5E-08	8E-25	0.013	0.055
	3E-63	1	0.041	0.65	1	0.787	0.64	0.438
	2E-54	3E-59	0.01	3E-08	1E-07	4E-14	0.029	0.053
	3E-64	5E-53	3E-47	0.35	1E-06	0.001	0.081	0.065
	1E-53	2E-56	2E-44	2E-50	1E-05	6E-13	0.069	0.122
	3E-66	3E-51	2E-47	8E-43	6E-49	0.211	0.112	0.075
	4E-54	1E-56	5E-45	7E-51	6E-49	4E-56	0.055	0.095
	1E-53	8E-57	7E-45	4E-51	6E-49	1E-56	5E-45	0.096

	7.434 3.185 7.277 3.831 3.185 3.503 4.026 4.313
	2.383 7.76 2.635 6.507 7.76 7.085 6.374 5.099
	2.461 -0.52 2.359 0.994 -0.52 0.453 0.927 1.385

计算Masked

同理得到 Head 1、Head 2 的结果，通过线性变换搬回到系统维度 d_model。

Linear								
	0	0	1	-1	0	-1	1	0
	1	1	1	0	-1	-1	1	1
	-1	-1	-1	0	-1	1	-1	-1
	-1	0	-1	-1	0	0	1	0
	-1	-1	-1	0	1	1	-1	0
	1	1	1	0	0	0	1	-1

×

=

9.34	-17.8	1.46	1.268	-11	-12.8	0.214	0.81
23.07	4.917	20.76	20.31	11.9	9.847	17.73	16.5
-17.9	-8.35	-16.8	-16	-15.3	-13	-15.1	-13.6
2.791	6.519	7.815	13.35	7.425	6.526	2.888	1.491
-18.7	2.162	-13.3	-12.8	-4.67	-2.49	-13.5	-12.8
17.48	18.96	22.13	21.14	19.71	20.43	16.47	15.48

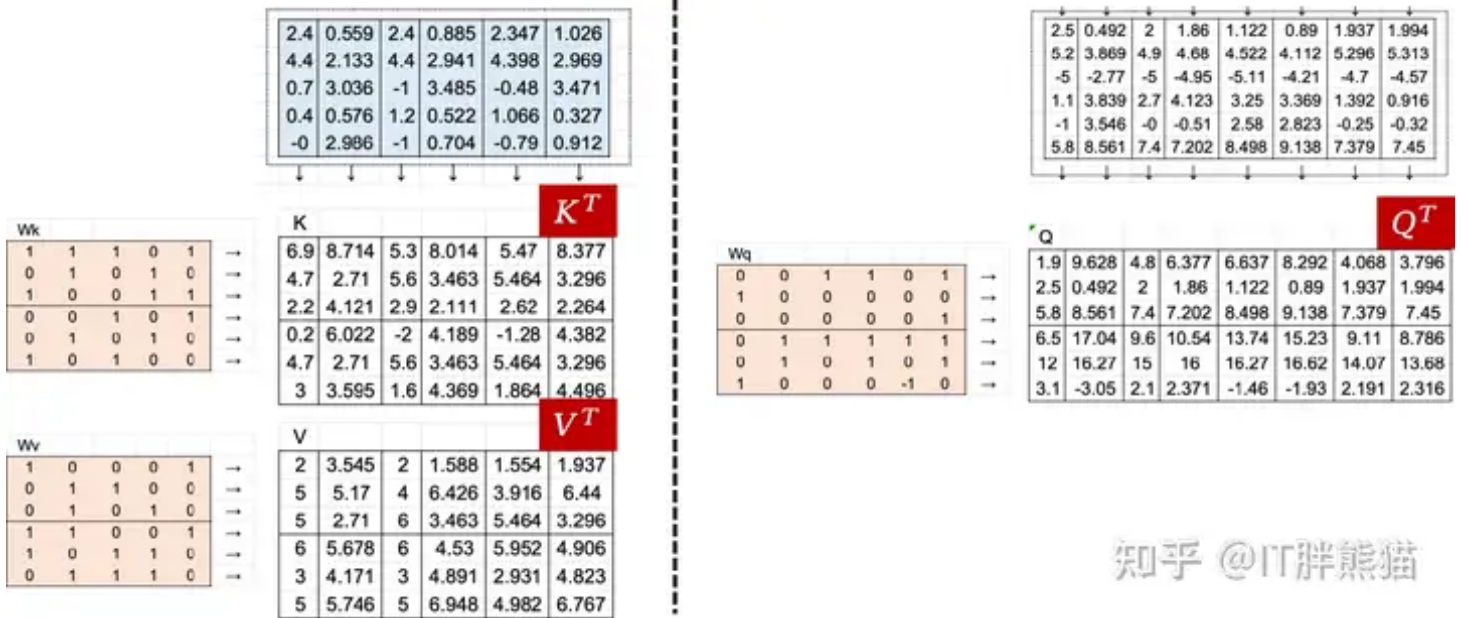
7.434	3.185	7.277	3.831	3.185	3.503	4.026	4.313
2.383	7.76	2.635	6.507	7.76	7.085	6.374	5.099
2.461	-0.52	2.359	0.994	-0.52	0.453	0.927	1.385
-3.07	6.433	-0.1	-0.9	6.432	6.262	1.546	1.167
-2.56	1.716	-1.99	-2.15	1.715	1.568	-1.33	-1.44
0.523	5.978	2.039	1.625	5.978	5.883	3.084	2.783
6.979	5.362	9.199	9.131	9.229	7.822	6.652	5.866
4.418	7.078	7.493	7.459	7.229	7.355	4.24	3.677
2.642	10.25	8.155	8.136	7.29	8.921	2.735	2.49

继续搬回系统维度，d_model = 6

知乎 @IT胖熊猫

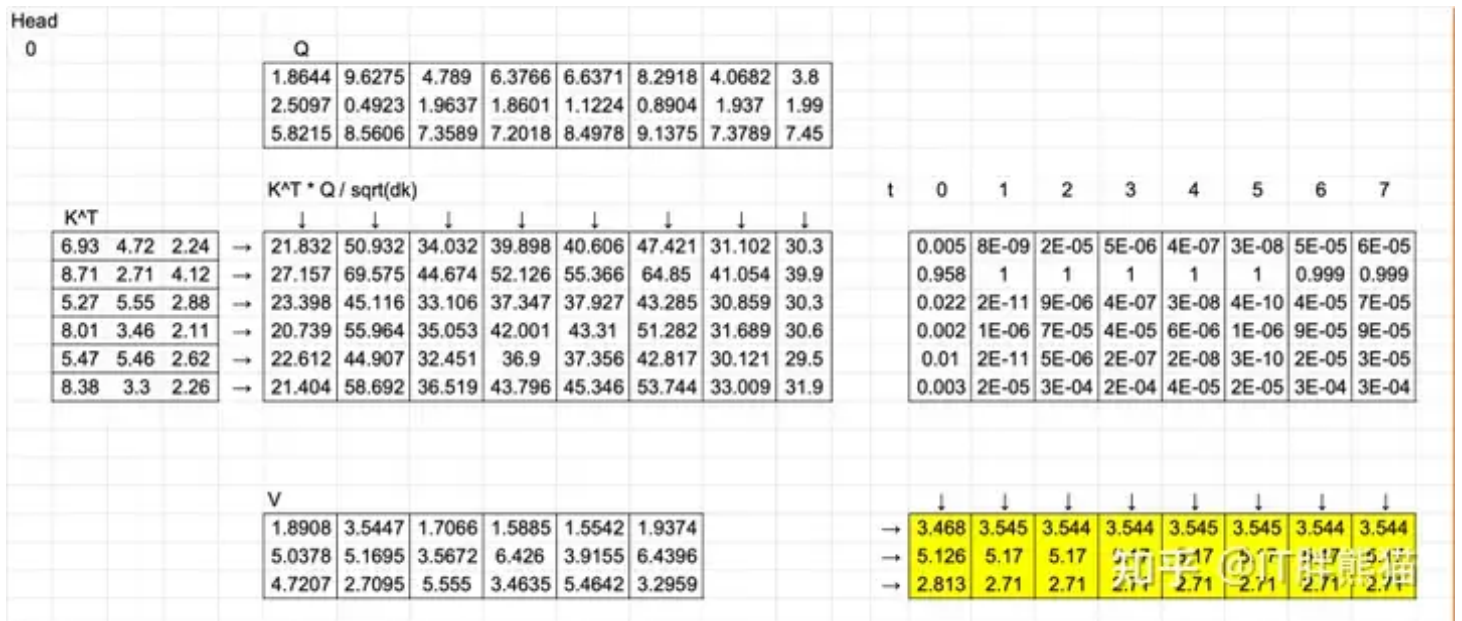
从3个注意力头切换到系统维度 d_model = 6

Add



Multi-Head Cross-Attention

计算标准的MHA，以 head 0 为例。



以Head 0为例计算MHA

同理计算 head 1，并做线性变换到系统维度。

3.468	3.545	3.544	3.544	3.545	3.545	3.544	3.544
5.126	5.17	5.17	5.17	5.17	5.17	5.17	5.17
2.813	2.71	2.71	2.71	2.71	2.71	2.71	2.71
5.095	5.678	5.575	5.606	5.677	5.678	5.555	5.527
4.581	4.171	4.244	4.222	4.172	4.171	4.258	4.278
6.413	5.746	5.864	5.828	5.746	5.746	5.887	5.919

Linear							
0	0	1	1	0	-1		→
0	1	1	0	1	0		→
0	1	0	0	1	1		→
-1	0	0	1	0	1		→
1	0	0	-1	0	1		→
1	0	-1	0	1	0		→

1.495	2.642	2.421	2.488	2.64	2.641	2.378	2.319
12.52	12.05	12.12	12.1	12.05	12.05	12.14	12.16
16.12	15.09	15.28	15.22	15.09	15.09	15.32	15.37
8.04	7.879	7.895	7.89	7.879	7.879	7.898	7.902
4.786	3.613	3.833	3.766	3.614	3.613	3.876	3.935
5.236	5.006	5.078	5.056	5.007	5.006	5.092	5.111

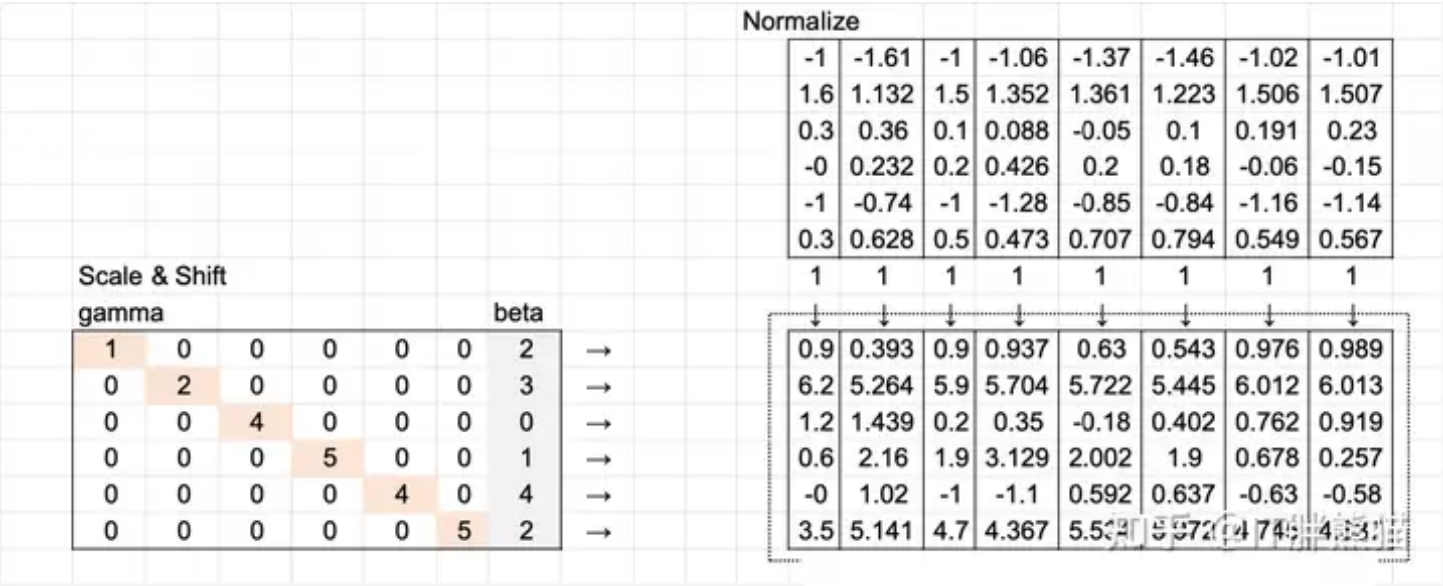
线性变换

Add

2.5	0.492	2	1.86	1.122	0.89	1.937	1.994		1.495	2.642	2.421	2.488	2.64	2.641	2.378	2.319
5.2	3.869	4.9	4.68	4.522	4.112	5.296	5.313		12.52	12.05	12.12	12.1	12.05	12.05	12.14	12.16
-5	-2.77	-5	-4.95	-5.11	-4.21	-4.7	-4.57		16.12	15.09	15.28	15.22	15.09	15.09	15.32	15.37
1.1	3.839	2.7	4.123	3.25	3.369	1.392	0.916		8.04	7.879	7.895	7.89	7.879	7.879	7.898	7.902
-1	3.546	-0	-0.51	2.58	2.823	-0.25	-0.32		4.786	3.613	3.833	3.766	3.614	3.613	3.876	3.935
5.8	8.561	7.4	7.202	8.498	9.138	7.379	7.45		5.236	5.006	5.078	5.056	5.007	5.006	5.092	5.111
								+								
								=								
4	3.134	4.4	4.348	3.763	3.532	4.315	4.312									
18	15.92	17	16.78	16.57	16.16	17.43	17.47									
11	12.31	10	10.27	9.976	10.87	10.61	10.8									
9.1	11.72	11	12.01	11.13	11.25	9.29	8.818									
4.2	7.158	3.7	3.255	6.193	6.436	3.622	3.612									
11	13.57	12	12.26	13.5	14.14	12.47	12.56									

add

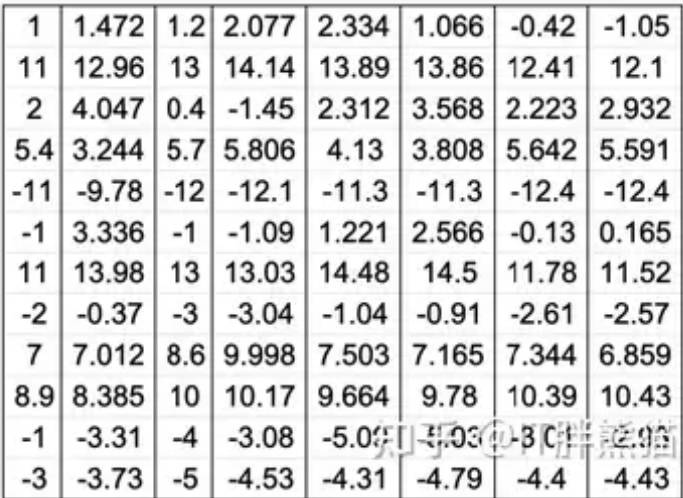
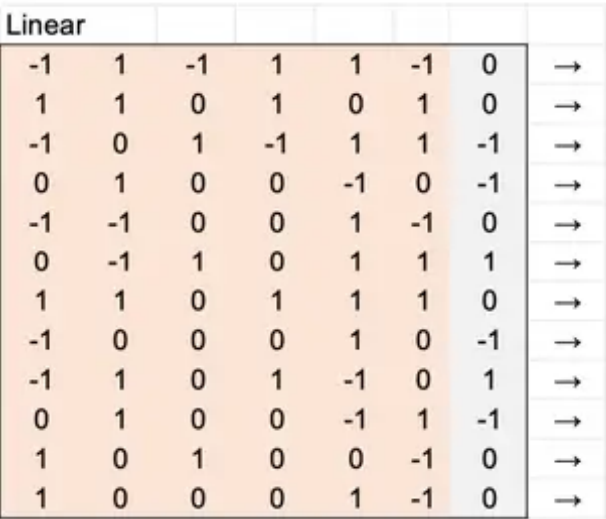
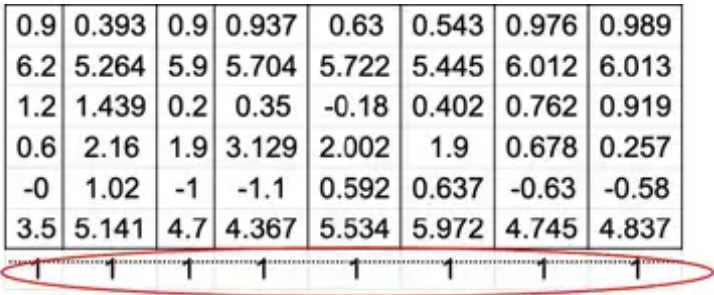
Normal , scale & shift



Normal , scale & shift

4.4 Feed Forward Neutral Network

依然是扩维到 $2 \times d_{model} = 12$, 维度不够，拿 1 来凑。



扩维

再变换回去

ReLU							
1	1.472	1.2	2.077	2.334	1.066	0	0
11	12.96	13	14.14	13.89	13.86	12.41	12.1
2	4.047	0.4	0	2.312	3.568	2.223	2.932
5.4	3.244	5.7	5.806	4.13	3.808	5.642	5.591
0	0	0	0	0	0	0	0
0	3.336	0	0	1.221	2.566	0	0.165
11	13.98	13	13.03	14.48	14.5	11.78	11.52
0	0	0	0	0	0	0	0
7	7.012	8.6	9.998	7.503	7.165	7.344	6.859
8.9	8.385	10	10.17	9.664	9.78	10.39	10.43
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1

Linear											
0	1	0	0	0	0	1	-1	1	-1	0	-1
0	-1	0	0	0	0	1	-1	1	0	0	-1
-1	1	0	1	1	-1	0	1	1	-1	0	1
1	1	1	1	-1	1	-1	-1	1	0	0	-1
1	-1	0	0	-1	0	-1	-1	-1	0	-1	1
0	-1	0	1	0	-1	1	1	0	1	0	-1

20	25.56	24	27	26.2	25.74	21.15	20.05
5.9	7.032	6.9	7.896	7.094	6.802	5.714	5.281
14	10.02	16	17.69	12.3	11.42	15.01	13.95
15	17.09	16	17.98	15.91	16.54	14.84	15.12
-28	-32.5	-34	-35.1	-33.5	-34.5	-31.5	-30.5
13	8.314	14	13.88	12.16	10.66	14.4	14.28

调整到系统维度

Add

0.9	0.393	0.9	0.937	0.63	0.543	0.976	0.989
6.2	5.264	5.9	5.704	5.722	5.445	6.012	6.013
1.2	1.439	0.2	0.35	-0.18	0.402	0.762	0.919
0.6	2.16	1.9	3.129	2.002	1.9	0.678	0.257
-0	1.02	-1	-1.1	0.592	0.637	-0.63	-0.58
3.5	5.141	4.7	4.367	5.534	5.972	4.745	4.837

+

ReLU							
20	25.56	24	27	26.2	25.74	21.15	20.05
5.9	7.032	6.9	7.896	7.094	6.802	5.714	5.281
14	10.02	16	17.69	12.3	11.42	15.01	13.95
15	17.09	16	17.98	15.91	16.54	14.84	15.12
0	0	0	0	0	0	0	0
13	8.314	14	13.88	12.16	10.66	14.4	14.28

=

21	25.96	25	27.93	26.83	26.29	22.13	21.04
12	12.3	13	13.6	12.82	12.25	11.73	11.29
15	11.46	16	18.04	12.12	11.82	15.77	14.87
15	19.25	17	21.11	17.91	18.44	15.52	15.38
-0	1.02	-1	-1.1	0.592	0.637	-0.63	-0.58
17	13.45	19	18.24	17.7	16.63	19.14	19.11

add

Normal , scale & shift

							Normalize							
							1.1	1.443	1.2	1.192	1.404	1.401	1.027	0.975
							-0	-0.19	-0	-0.28	-0.21	-0.25	-0.28	-0.29
							0.2	-0.29	0.2	0.178	-0.29	-0.3	0.23	0.176
							0.3	0.64	0.3	0.493	0.375	0.48	0.198	0.242
							-2	-1.54	-2	-1.78	-1.62	-1.61	-1.83	-1.83
							0.4	-0.05	0.5	0.199	0.35	0.268	0.653	0.726
							1	1	1	1	1	1	1	1
							↓	↓	↓	↓	↓	↓	↓	↓
							3.1	3.443	3.2	3.192	3.404	3.401	3.027	2.975
							2.7	2.614	2.5	2.445	2.574	2.508	2.443	2.422
							0.9	-1.17	0.6	0.712	-1.17	-1.18	0.919	0.702
							2.3	4.2	2.4	3.465	2.874	3.4	1.988	2.208
							-3	-2.17	-3	-3.14	-2.41	-2.43	-3.32	-3.32
							4.2	1.729	4.3	2.994	3.751	3.342	5.265	5.63

Scale & Shift								
gamma							beta	
1	0	0	0	0	0	2	→	
0	2	0	0	0	0	3	→	
0	0	4	0	0	0	0	→	
0	0	0	5	0	0	1	→	
0	0	0	0	4	0	4	→	
0	0	0	0	0	5	2	→	

Normal , scale & shift

5 Output处理

5.1 Linear

这里的线性变换是为了凑成字典要求的维度，现在要输出的只是一个token。

3.1	3.443	3.2	3.192	3.404	3.401	3.027	2.975
2.7	2.614	2.5	2.445	2.574	2.508	2.443	2.422
0.9	-1.17	0.6	0.712	-1.17	-1.18	0.919	0.702
2.3	4.2	2.4	3.465	2.874	3.4	1.988	2.208
-3	-2.17	-3	-3.14	-2.49	-2.43	-3.32	-3.32
4.2	1.729	4.3	2.994	3.751	3.342	5.265	5.63
1	1	1	1	1	1	1	1
↓	↓	↓	↓	↓	↓	↓	↓

0	0	2	0	0	1	1	0	→	6.1	4.786	6	4.745	6.409	5.927	6.834	7.156
1	1	0	-1	-2	0	1	-3	→	-1	-5.06	-1	-4.45	-0.42	-1.87	0.397	0.488
2	-1	2	-1	-2	-2	0	0	→	3.6	-1.1	2.8	0.336	2.153	0.862	3.601	3.388
3	1	0	0	1	0	1	-4	→	5.6	5.372	5.8	5.651	6.029	6.143	6.28	6.814
4	0	1	-2	0	0	0	0	→	0.8	4.957	1.2	1.021	4.921	4.873	0.605	1.018
5	-2	-1	0	1	-2	1	2	→	6.5	2.774	6.2	5.908	4.225	4.296	7.391	8.103
6	0	2	1	-2	0	0	0	→	1.7	-4.34	0.9	-1.33	-1.77	-2.97	1.83	1.131
7	-2	0	1	0	0	1	-1	→	-2	-7.33	-2	-3.68	-5.23	-5.64	-0.87	-0.62
8	1	-1	2	-1	0	0	0	→	0	-5.71	-0	-1.29	-4.39	-4.87	0.434	-0.25
9	1	-1	-1	0	-1	0	-1	→	1.9	3.173	2.3	2.175	3.494	3.507	1.983	2.17
10	1	0	2	1	-2	0	0	→	14	9.645	13	14.36	8.914	9.3	13.49	13.22
11	0	0	1	-2	1	1	2	→	-1	-8.01	-1	-4.36	-3.66	-5.07	0.89	0.598
12	2	0	2	1	2	0	0	→	3.5	4.398	3.6	4.995	2.353	2.973	3.244	2.926
13	0	0	2	-2	1	2	0	→	2.3	-9.46	1.8	-2.66	-3.08	-4.91	5.074	4.931
14	2	1	-2	-2	0	-2	1	→	-5	0.985	-5	-4.51	-0.52	-0.81	-6.85	-7.71
15	2	-2	2	0	-2	1	0	→	14	5.388	13	12.19	8.046	7.627	14.91	14.78
16	0	1	-1	0	2	-1	0	→	-9	-2.29	-9	-7.54	-4.99	-4.52	-10.4	-10.5
17	1	0	0	-2	0	0	0	→	-1	-4.96	-2	-3.74	-2.34	-3.4	-0.95	-1.44
18	1	1	1	2	2	0	0	→	4.4	8.941	4.6	7	5.57	6.663	3.729	3.878
19	-2	0	1	-2	-2	2	1	→	6.5	-7.65	5.5	0.666	-0.24	-2.24	9.055	9.234
20	1	2	0	-1	-1	0	0	→	9.6	6.644	9	7.758	8.17	7.449	9.245	8.931
21	0	1	0	1	0	0	1	→	5.9	7.814	5.9	6.91	6.448	6.908	5.432	5.63
22	0	-1	0	-1	0	-2	0	→	-13	-10.3	-13	-11.9	-11.5	-12.6	-11.5	-11.9
23	-2	2	0	1	0	0	-1	→	0.4	1.543	-0	0.971	0.214	0.614	-0.18	0.102

线性变化

5.2 Softmax

请参考我的回答：[关于Softmax的说明](#)

Prediction (Y) (Output Probabilities)									
	t	0	1	2	3	4	5	6	7
[S]	0	0	0.004	0	6E-05	0.037	0.021	2E-04	4E-04
a	1	###	2E-07	###	6E-09	4E-05	9E-06	4E-07	5E-07
and	2	###	1E-05	###	7E-07	0.001	1E-04	1E-05	9E-06
apples	3	0	0.008	0	1E-04	0.026	0.026	1E-04	3E-04
brings	4	###	0.005	###	1E-06	0.008	0.007	5E-07	9E-07
by	5	0	0.001	0	2E-04	0.004	0.004	4E-04	0.001
cat	6	###	5E-07	###	1E-07	1E-05	3E-06	2E-06	1E-06
day	7	###	2E-08	###	1E-08	3E-07	2E-07	1E-07	2E-07
dog	8	###	1E-07	###	1E-07	8E-07	4E-07	4E-07	2E-07
eat	9	###	0.001	###	5E-06	0.002	0.002	2E-06	3E-06
fill	10	0.6	0.569	0.5	0.895	0.457	0.622	0.194	0.173
friend	11	###	1E-08	###	7E-09	2E-06	4E-07	7E-07	6E-07
happy	12	###	0.003	###	8E-05	0.001	0.001	7E-06	6E-06
house	13	###	3E-09	###	4E-08	3E-06	4E-07	4E-05	4E-05
laughter	14	###	1E-04	###	6E-09	4E-05	3E-05	3E-10	1E-10
light	15	0.4	0.008	0.5	0.102	0.192	0.117	0.8	0.819
music	16	###	4E-06	###	3E-10	4E-07	6E-07	8E-12	8E-12
night	17	###	3E-07	###	1E-08	6E-06	2E-06	1E-07	7E-08
play	18	###	0.281	###	0.001	0.016	0.044	1E-05	2E-05
quiet	19	0	2E-08	0	1E-06	5E-05	6E-06	0.002	0.003
smile	20	0	0.028	0	0.001	0.217	0.098	0.003	0.002
the	21	0	0.091	0	0.001	0.039	0.057	6E-05	9E-05
to	22	###	1E-09	###	4E-12	1E-10	2E-10	9E-14	4E-14
water	23	###	2E-04	###	1E-06	8E-05	1E-04	2E-07	3E-07

Last Token Position: 5

	t	0	1	2	3	4	5	6	7
[S]	0						0.021		
a	1						9E-06		
and	2						1E-04		
apples	3						0.026		
brings	4						0.007		
by	5						0.004		
cat	6						3E-06		
day	7						2E-07		
dog	8						4E-07		
eat	9						0.002		
fill	10						0.622		
friend	11						4E-07		
happy	12						0.001		
house	13						4E-07		
laughter	14						3E-05		
light	15						0.117		
music	16						6E-07		
night	17						2E-06		
play	18						0.044		
quiet	19						6E-06		
smile	20						0.098		
the	21						0.039		
to	22						2E-10		
water	23						1E-04		

得到结果

这个结果其实有问题的，因为是输出是一个token一个token进去的，Decoder部分算上PE，总共成了7个token，其实应该是5个，不过问题不大，并不影响理解。

实务中就是最后一列可能性最大的token就是输出的token，依次类推直到输出终止符号结束推理。

~结束~

6 和算力基础设施的联系

这个话题都很复杂，篇幅有限，可以参考我提供的其他回答，这里只是简单结合实务介绍。

6.1 模型到底多大

模型参数指的就是那些 W 矩阵，这里都是转置形式体现的，这些 W 矩阵的每一个值就是一个parameter，加在一起就是模型参数量。

从整体上看，系统维度是 d_model ，但是中间做了太多次变换了，感兴趣的各位可以自己计算。

实务中可能每一层并不一样，比如DeepSeek有61层的Decoder，并没有Encoder，但是这61层前3层和后58层又不一样，非常复杂。

6.2 模型的权重是怎么更新的

这个就要涉及到模型训练了，有三个问题：

- **前向传播 (Forward Propagation) :**

输入到输出各层计算（如卷积、池化层等），产生输出并完成损失函数计算。

- **反向传播 (Back Propagation) :**

由输出到输入反向完成整个模型中各层的权重和输出对损失函数的梯度求解。

- **梯度更新 (Weight Update) :**

根据指定的学习率，对模型权重通过梯度下降算法完成权重的更新。

以上的都是有框架比如PyTorch自动给你算出来，刚开始每个权重矩阵都是随机值，后面每运行一次，都可以改变一次矩阵值，该怎么改就要看你的梯度了，按道理来讲要让梯度变成 0，但是变得太快也不行，会走得过头了，怎么改就要看你的优化器算法了，本质是数学问题，作为硬件工程师，我其实并不关心，我关注的是它对于我们硬件需求有什么变化。

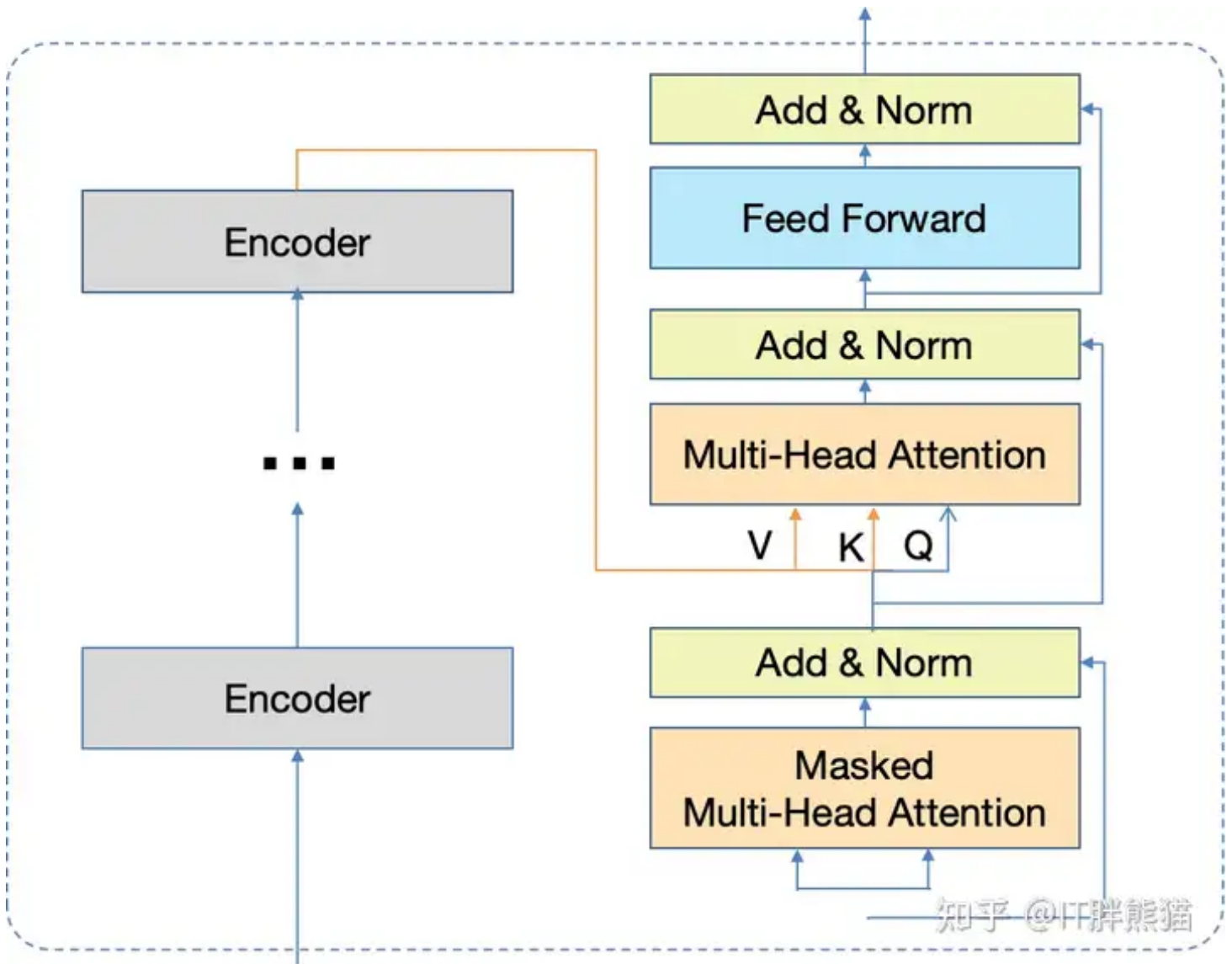
请参考下面：

IT胖熊猫：想学AI，从深度学习到大模型，看这一篇就足够了

6.3 KV cache到底是什么

请参考计算KV显存需求：[什么配置能本地部署并运行满血671B的DeepSeek-R1？](#)

清楚了整个推理过程之后，就知道KV cache是什么了。



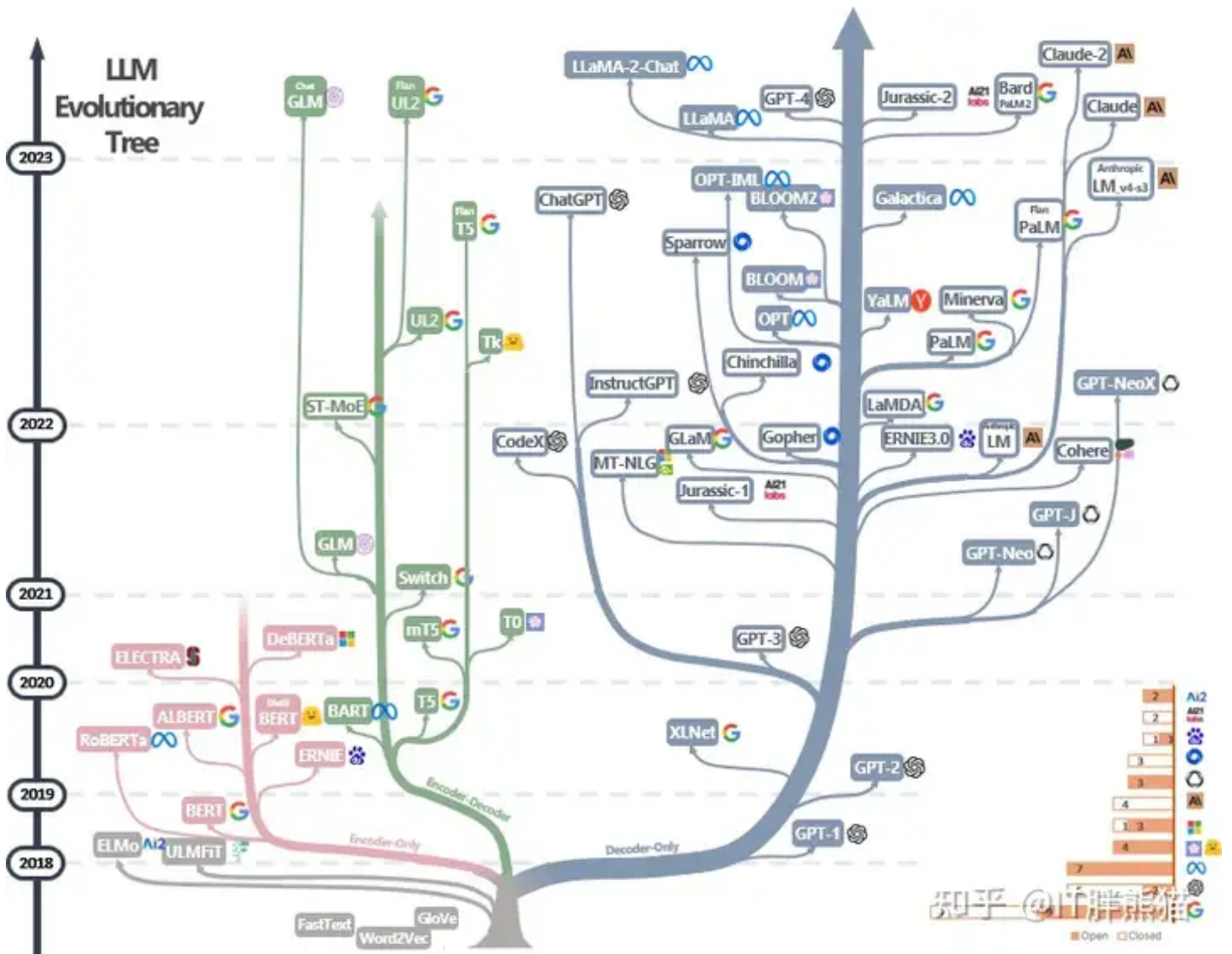
从Transformer架构看KV Cache

对于一次推理过程，Decoder每输出一个token就要做一次运算，你会发现这个K和V都来自Encoder部分，其实就是输入序列算出来的，每一层的Decoder都有一份，那就没有必要每次都重算一遍了，直接把这批KV存起来就可以了，那每一层有多少个kv cache呢？

其实就是 d_{model} 个，也就是所谓的hidden_size，到这里理解KV cache的显存计算就再也没有问题了。

7 Transformer 的发展

Transformer架构自从发明之后，沿着几条道路快速发展，它模型分Encoder和Decoder两个部分，怎么做成实际的模型是个排列组合问题：只选择Encoder，只选择Decoder，或者两者都选取，后面的问题就是多少层叠加的问题了。



Transformer演化路径 | 来源：A Survey on ChatGPT and Beyond

2023年这个图传的很广，是8个华人画的，清晰梳理了2018到2023**五年间**所有的大语言模型“代表作”，并将这些模型架构分成三大类，其中同一分支上的模型关系更近。

基于transformer的模型显示为非灰色颜色：仅**解码器模型（Decoder-only）**显示为蓝色分支,仅**编码器模型（Encoder-only）**显示为粉红色分支,而**编码器-解码器（Encoder-Decoder）**模型显示为绿色分支。时间轴上模型的垂直位置代表其发布日期。开源模型由实心方块表示,而闭源模型由空心方块表示。右下角的堆叠条形图显示来自各公司和机构的模型数量。

基本结论是：

在语言模型发展的早期阶段，**仅解码器模型不如仅编码器模型和编码器-解码器模型流行**。然而，在2021年之后，随着改变游戏规则的语言模型**GPT-3+**的推出，仅解码器模型经历了显著的繁荣。与此同时，在BERT带来的初始爆炸性增长之后，仅编码器模型逐渐开始消失，**仅解码器模型（Decoder-**

Only) 在语言模型的发展中逐渐占主导地位。谷歌、Meta在仅解码器、仅编码器和编解码器三个方向都有布局，但是效果惨淡，时至今日，只搞仅解码器 (Decoder-Only) 的OpenAI占据上风。

7.1 仅编码器模型 (Encoder-Only) —— Bert考古

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Jacob Devlin Ming-Wei Chang Kenton Lee Kristina Toutanova
Google AI Language
{jacobdevlin, mingweichang, kentonl, kristout}@google.com

Abstract

We introduce a new language representation model called **BERT**, which stands for **Bidirectional Encoder Representations from**

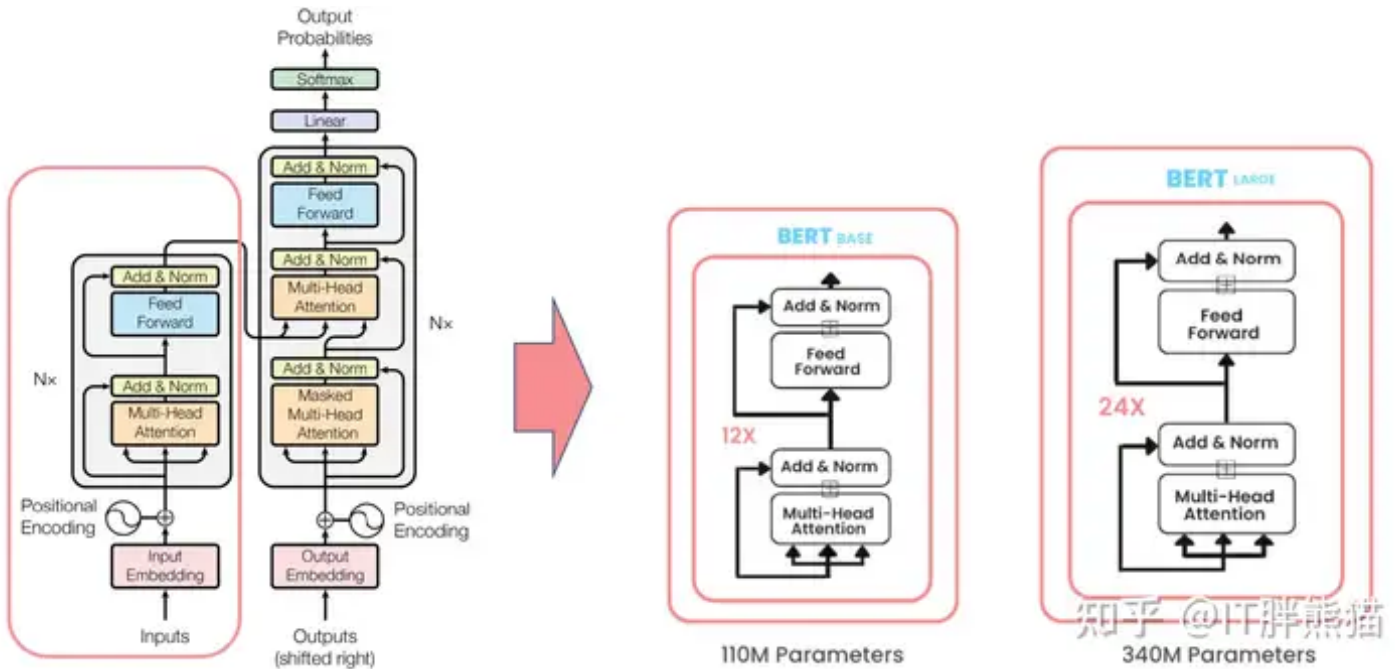
There are two existing strategies for applying pre-trained language representations to downstream tasks: *feature-based* and *token-classification*. The *feature-based* approach, such as ELMo (Peters

Bert模型论文

BERT，全称为 **Bidirectional Encoder Representations from Transformers**，是 Google 在 2018 年引入的一种开创性模型。如果仔细看过前面的内容，就应该能明白，如果只选用了Encoder，因为没有掩码，输出是同时生成的，并不会一个一个的生成，因为有位置编码的关系，还是能够决定文字的实际顺序，但是输出其实是同时的。

BERT 同时针对两个目标进行训练：

1. 从一系列单词中预测缺失的单词
2. 在一系列句子之后预测新的句子



Bert模型架构 Base & Large

代码实例

```
import torch
```

```
# 1.加载文本并进行Tokenization
```

```
tokenizer = torch.hub.load('huggingface/pytorch-transformers', 'tokenizer', 'bert-base-cas
```

```
text_1 = "I understand equations, both the simple and quadratical."
```

```
text_2 = "What kind of equations do I understand?"
```

```
indexed_tokens = tokenizer.encode(text_1, text_2, add_special_tokens=True)
```

```
# 2.文本分段，每看到一个[SEP] 标记时就增加 segment_id
```

```
cls_token = 101
```

```
sep_token = 102
```

```
def get_segment_ids(indexed_tokens):
```

```
    segment_ids = []
```

```
    segment_id = 0
```

```
    for token in indexed_tokens:
```

```
        if token == sep_token:
```

```
            segment_id += 1
```

```
            segment_ids.append(segment_id)
```

```
    segment_ids[-1] -= 1 # Last [SEP] is ignored
```

```
    return torch.tensor([segment_ids]), torch.tensor([indexed_tokens])
```

```

segments_tensors, tokens_tensor = get_segment_ids(indexed_tokens)

# 3. 文本掩码 (Text Masking)
masked_index = 5
indexed_tokens[masked_index] = tokenizer.mask_token_id
tokens_tensor = torch.tensor([indexed_tokens])
masked_lm_model = torch.hub.load('huggingface/pytorch-transformers', 'modelForMaskedLM', '

# 4. 完成Embedding
embedding_table = next(masked_lm_model.bert.embeddings.word_embeddings.parameters)

# 5. Predict the start and end positions logits
with torch.no_grad():
    out = question_answering_model(tokens_tensor, token_type_ids=segments_tensors)

# 6. Q&A and Detokenization
question_answering_model = torch.hub.load('huggingface/pytorch-transformers', 'modelForQue
answer_sequence = indexed_tokens[torch.argmax(out.start_logits):torch.argmax(out.end_logit
question_answering_tokenizer.convert_ids_to_tokens(answer_sequence)
['quad', '##ratic', '##al']
question_answering_tokenizer.decode(answer_sequence)
'quadratical'

```

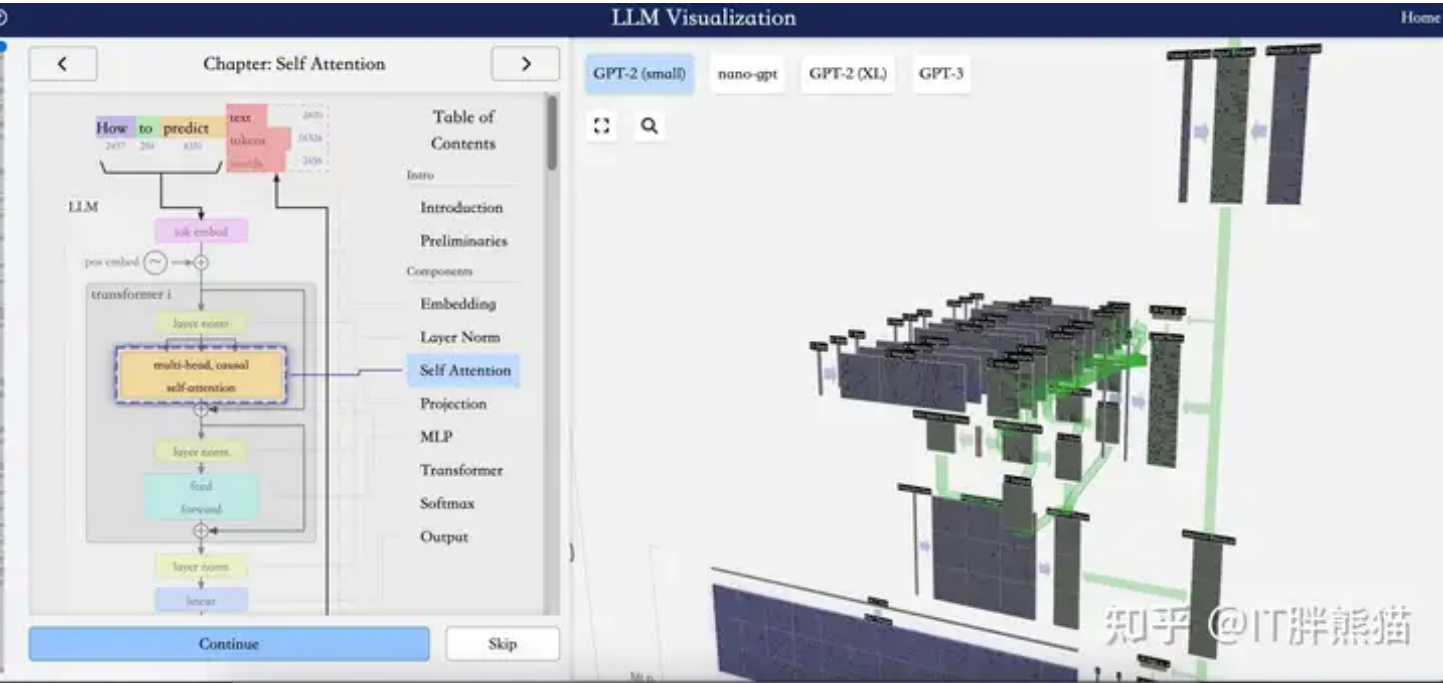
7.2 仅解码器模型 (Decoder-Only) —— GPT模型

Decoder-only的带头大哥——OpenAI，连续发布了4.5个版本，GPT-4 及之后版本，OpenAI 采取了更为保守和保密的做法，并未像之前一样公开模型的参数量、层数、隐藏层维度等详细信息。唯一能确认的是，GPT-4 提供了更长的上下文窗口（8K/32K tokens），并且在理解、生成文本以及多模态任务上有了明显改进。

GPT Model	GPT-1	GPT-2	GPT-3	GPT-4	GPT-4.5
Parameters	117 Million	1.5 Billion	175 Billion		-
Decoder Layers	12	48	96		-
Context Token Size	512	1024	2048	8192/32768	-
Hidden Layer	768	1600	12288		-
Batch Size	64	512	3.2M		

GPT参数规模

推荐个网址，LLM Visualization，可以看GPT的数据流向，下面是我做的一个截图。



LLM Visualization

7.3 编码器-解码器 (Encoder-Decoder)

1. T5

T5 模型通过将所有 NLU 和 NLG 任务转换为文本到文本任务来统一它们。所有任务都被构建为序列到序列的任务，所以采用 encoder-decoder 架构是很自然的。例如，对于文本分类问题，这意味着文本用作 encoder 输入，并且 decoder 必须将标签生成为普通文本而不是类别。T5 架构采用原有的 Transformer 架构。使用大型爬网 C4 数据集，通过将所有这些任务转换为文本到文本任务，使用 MLM 以及 SuperGLUE 任务对模型进行预训练。11B 模型在多个基准测试中产生了领先的结果。

2. [BART+](#)

BART 在 encoder-decoder 架构中结合了 BERT 和 GPT 的预训练过程。输入序列会经历几种可能的转换之一，从简单的 mask token 到句子排列、删除 token 和文档旋转。这些修改后的输入通过 encoder 传递，decoder 必须重建原始文本。这使得模型更加灵活，因为可以将其用于 NLU 和 NLG 任务，并且在这两个任务上都实现了领先的性能。

3. [M2M-100+](#)

通常，翻译模型是针对一种语言对和翻译方向构建的。当然，这并不能扩展到许多语言，此外，语言对之间可能存在共享知识，可用于稀有语言之间的翻译。M2M-100 是第一个可以在 100 种语言之间进行翻译的翻译模型。这可以在稀有语言和代表性不足的语言之间进行高质量翻译。该模型使用前缀 token（类似于特殊的 [CLS] token）来指示源语言和目标语言。

4. [BigBird+](#)

由于注意力机制的二次内存要求，Transformer 模型的一个主要限制是最大上下文大小。BigBird 通过使用线性扩展的稀疏注意力形式来解决这个问题。这允许上下文从大多数 BERT 模型中的 512 个 token 大幅扩展至 BigBird 中的 4096 个。

8 参考

1. [ImagineAILab/ai-by-hand-excel: 用 Excel 手搓各种 AI 算法和模型](#)
2. [Attention is All your Need](#)
3. [Recurrent Continuous Translation Models](#)
4. [Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation](#)
5. [Sequence to Sequence Learning with Neural Networks](#)
6. [NEURAL MACHINE TRANSLATION BY JOINTLY LEARNING TO ALIGN AND TRANSLATE](#)
7. [AI System](#)

8. Harnessing the Power of LLMs in Practice: A Survey on ChatGPT and Beyond
9. Visualizing and Explaining Transformer Models From the Ground Up - Deepgram Blog ↗ | Deepgram
10. Understanding Attention In Transformers Models
11. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

更新记录：

2025/03/8：第一次更新

2025/03/11: 补充残差连接内容

2025/03/17: 补充第七节，Transformer的演进方向

2025/05/01: 应网友吐槽，修改错别字(1)

2025/05/06: 应网友吐槽，修改错别字(2)

如果还有疑问请留言指出，谢谢。

▲赞同 2942



●81 条评论

🔗分享

★收藏

♥喜欢

...

收起 ▾

更多回答



初识CV

西安电子科技大学 电子科学与技术硕士

+ 关注

1329 人赞同了该回答

›

建议大家看一下李宏毅老师讲解的Transformer，非常简单易懂（个人觉得史上最强transformer讲解）：youtube.com/watch?...

前言

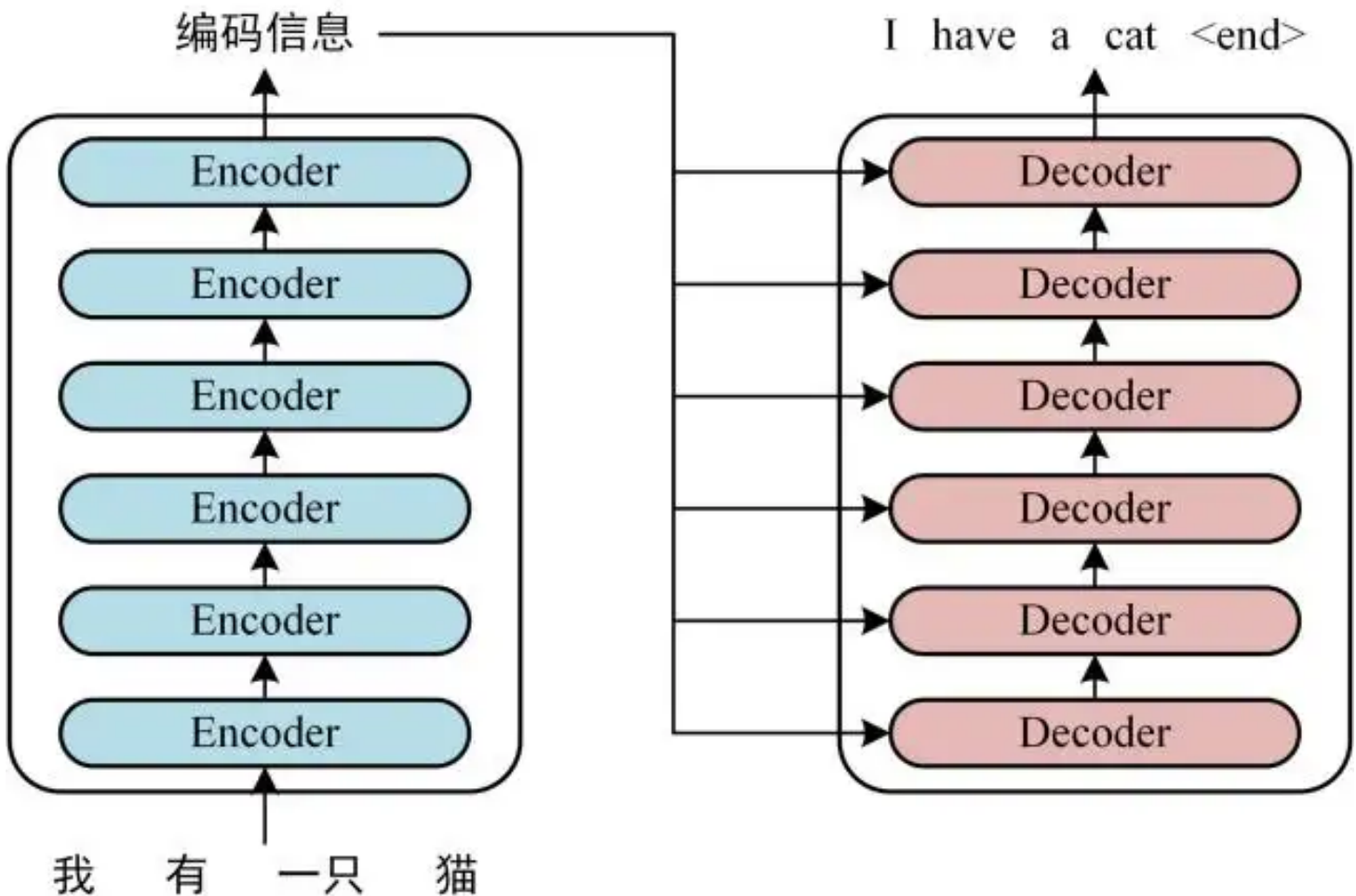
Transformer由论文[《Attention is All You Need》](#)提出，现在是谷歌云TPU推荐的参考模型。论文相关的[Tensorflow](#)的代码可以从GitHub获取，其作为Tensor2Tensor包的一部分。哈佛的NLP团队也实现了一个基于[PyTorch](#)的版本，并注释该论文。

在本文中，我们将试图把模型简化一点，并逐一介绍里面的核心概念，希望让普通读者也能轻易理解。

Attention is All You Need : [Attention Is All You Need](#)

1.Transformer 整体结构

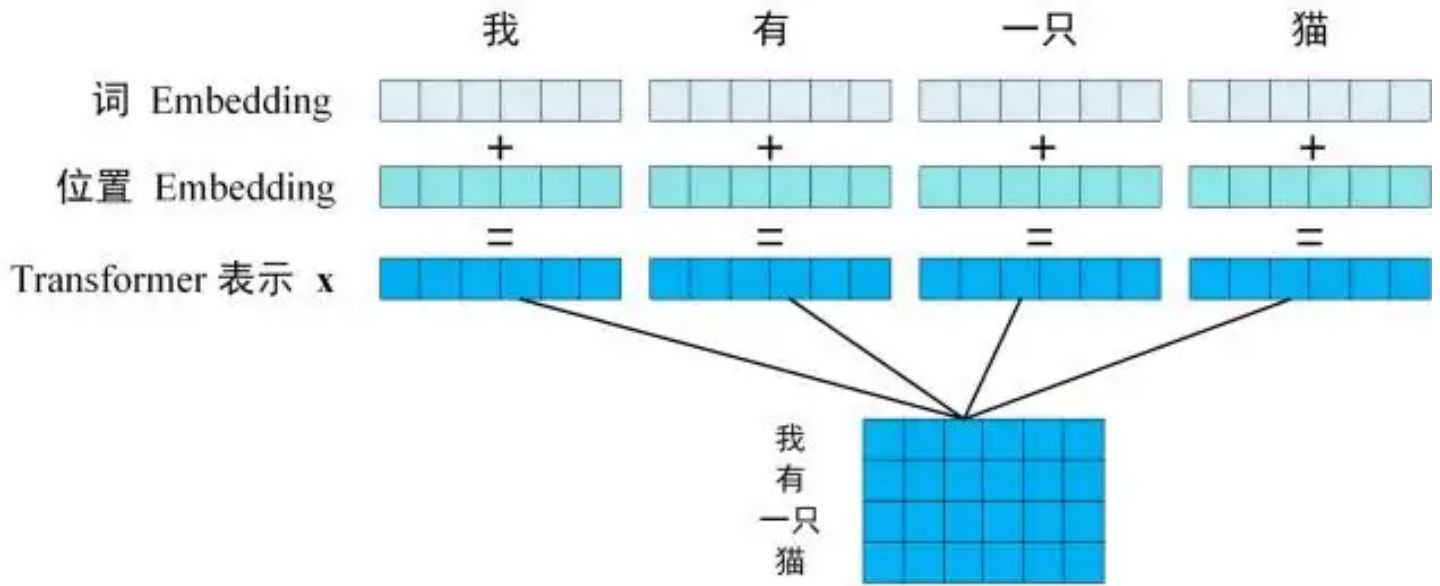
首先介绍 Transformer 的整体结构，下图是 Transformer 用于中英文翻译的整体结构：



Transformer 的整体结构，左图Encoder和右图Decoder

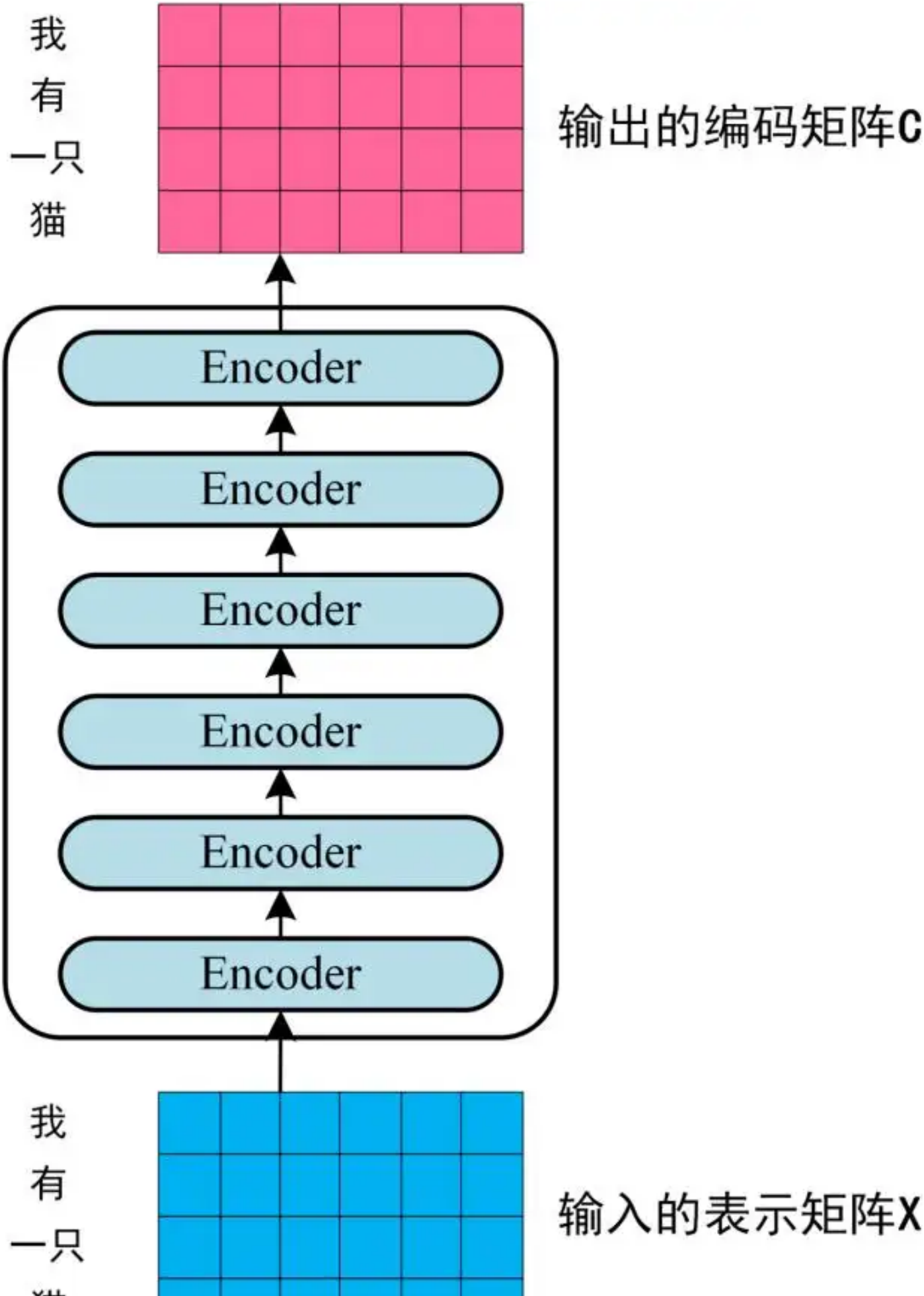
可以看到 **Transformer 由 Encoder 和 Decoder 两个部分组成**，Encoder 和 Decoder 都包含 6 个 block。Transformer 的工作流程大体如下：

第一步：获取输入句子的每一个单词的表示向量 $\mathbf{x}$ ， $\mathbf{x}$ 由单词的 Embedding（Embedding就是从原始数据提取出来的Feature）和单词位置的 Embedding 相加得到。



Transformer 的输入表示

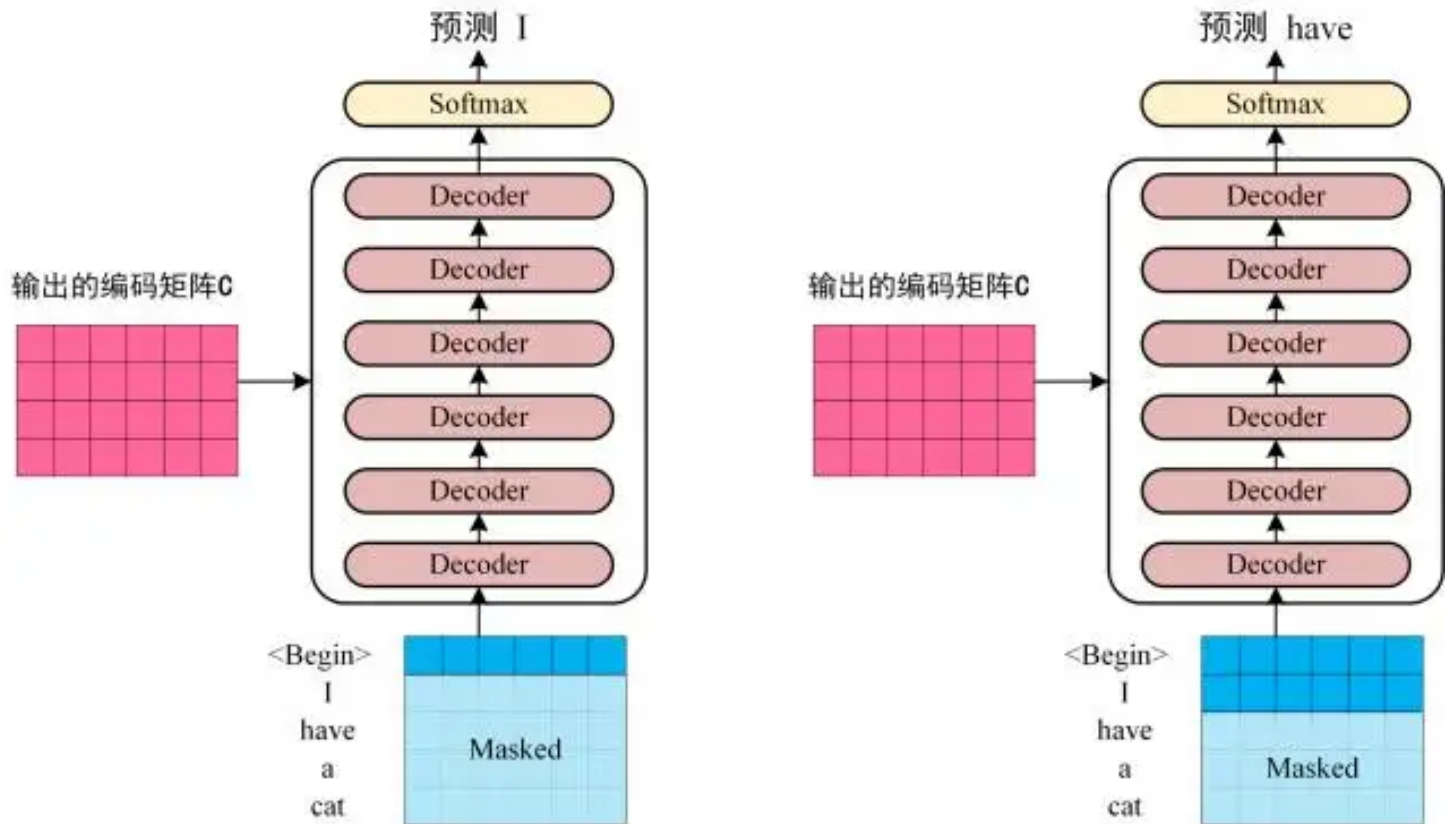
第二步：将得到的单词表示向量矩阵 (如上图所示，每一行是一个单词的表示 $\mathbf{x}$) 传入 Encoder 中，经过 6 个 Encoder block 后可以得到句子所有单词的编码信息矩阵 $\mathbf{C}$ ，如下图。单词向量矩阵用 $\mathbf{X}_{n \times d}$ 表示， n 是句子中单词个数， d 是表示向量的维度 (论文中 $d=512$)。每一个 Encoder block 输出的矩阵维度与输入完全一致。



猫



第三步：将 Encoder 输出的编码信息矩阵 **C** 传递到 Decoder 中，Decoder 依次会根据当前翻译过的单词 $1 \sim i$ 翻译下一个单词 $i+1$ ，如下图所示。在使用的过程中，翻译到单词 $i+1$ 的时候需要通过 **Mask (掩盖)** 操作遮盖住 $i+1$ 之后的单词。

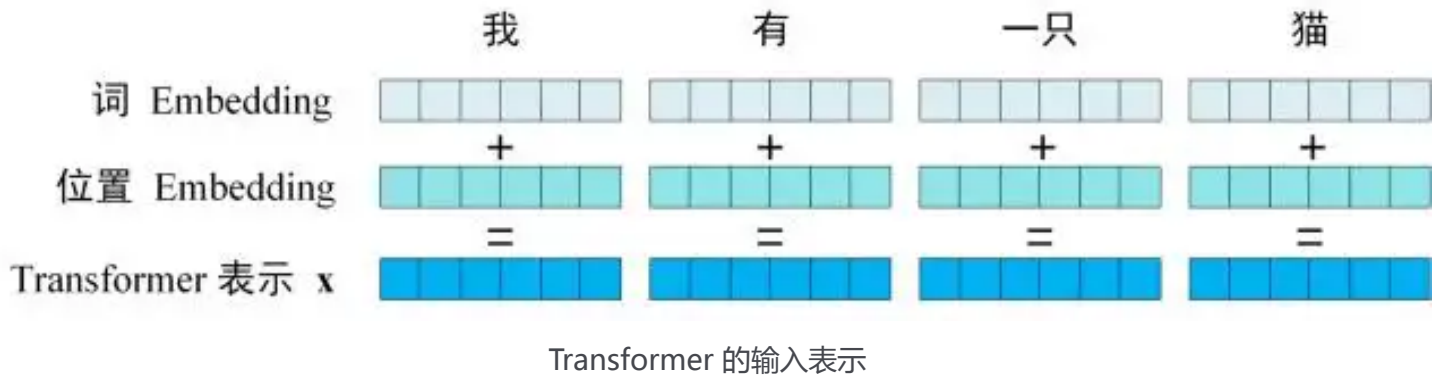


Transofrmer Decoder 预测

上图 Decoder 接收了 Encoder 的编码矩阵 **C**，然后首先输入一个翻译开始符 "<Begin>"，预测第一个单词 "I"；然后输入翻译开始符 "<Begin>" 和单词 "I"，预测单词 "have"，以此类推。这是 Transformer 使用时候的大致流程，接下来是里面各个部分的细节。

2. Transformer 的输入

Transformer 中单词的输入表示 **x** 由 **单词 Embedding** 和 **位置 Embedding** (Positional Encoding) 相加得到。



2.1 单词 Embedding

单词的 Embedding 有很多种方式可以获取，例如可以采用 Word2Vec、Glove 等算法预训练得到，也可以在 Transformer 中训练得到。

2.2 位置 Embedding

Transformer 中除了单词的 Embedding，还需要使用位置 Embedding 表示单词出现在句子中的位置。**因为 Transformer 不采用 RNN 的结构，而是使用全局信息，不能利用单词的顺序信息，而这部分信息对于 NLP 来说非常重要。**所以 Transformer 中使用位置 Embedding 保存单词在序列中的相对或绝对位置。

位置 Embedding 用 **PE** 表示，**PE** 的维度与单词 Embedding 是一样的。PE 可以通过训练得到，也可以使用某种公式计算得到。在 Transformer 中采用了后者，计算公式如下：

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d})$$

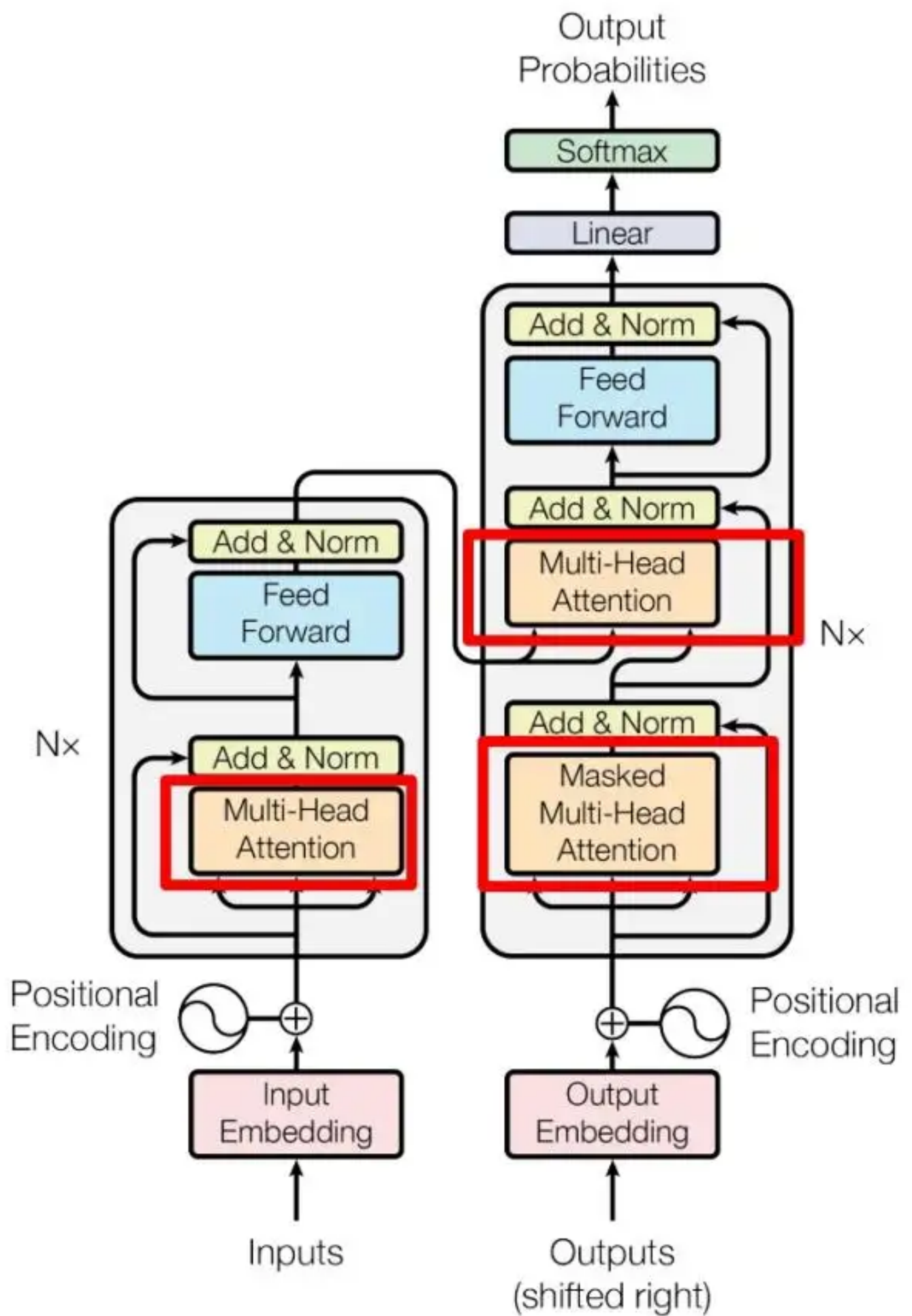
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d})$$

其中，pos 表示单词在句子中的位置，d 表示 PE 的维度 (与词 Embedding 一样)，2i 表示偶数的维度，2i+1 表示奇数维度 (即 $2i \leq d$, $2i+1 \leq d$)。使用这种公式计算 PE 有以下的好处：

- 使 PE 能够适应比训练集里面所有句子更长的句子，假设训练集里面最长的句子是有 20 个单词，突然来了一个长度为 21 的句子，则使用公式计算的方法可以计算出第 21 位的 Embedding。
- 可以让模型容易地计算出相对位置，对于固定长度的间距 k，**PE(pos+k)** 可以用 **PE(pos)** 计算得到。因为 $\sin(A+B) = \sin(A)\cos(B) + \cos(A)\sin(B)$, $\cos(A+B) = \cos(A)\cos(B) - \sin(A)\sin(B)$ 。

将单词的词 Embedding 和位置 Embedding 相加，就可以得到单词的表示向量 $\mathbf{x}$ ， $\mathbf{x}$ 就是 Transformer 的输入。

3. Self-Attention (自注意力机制)



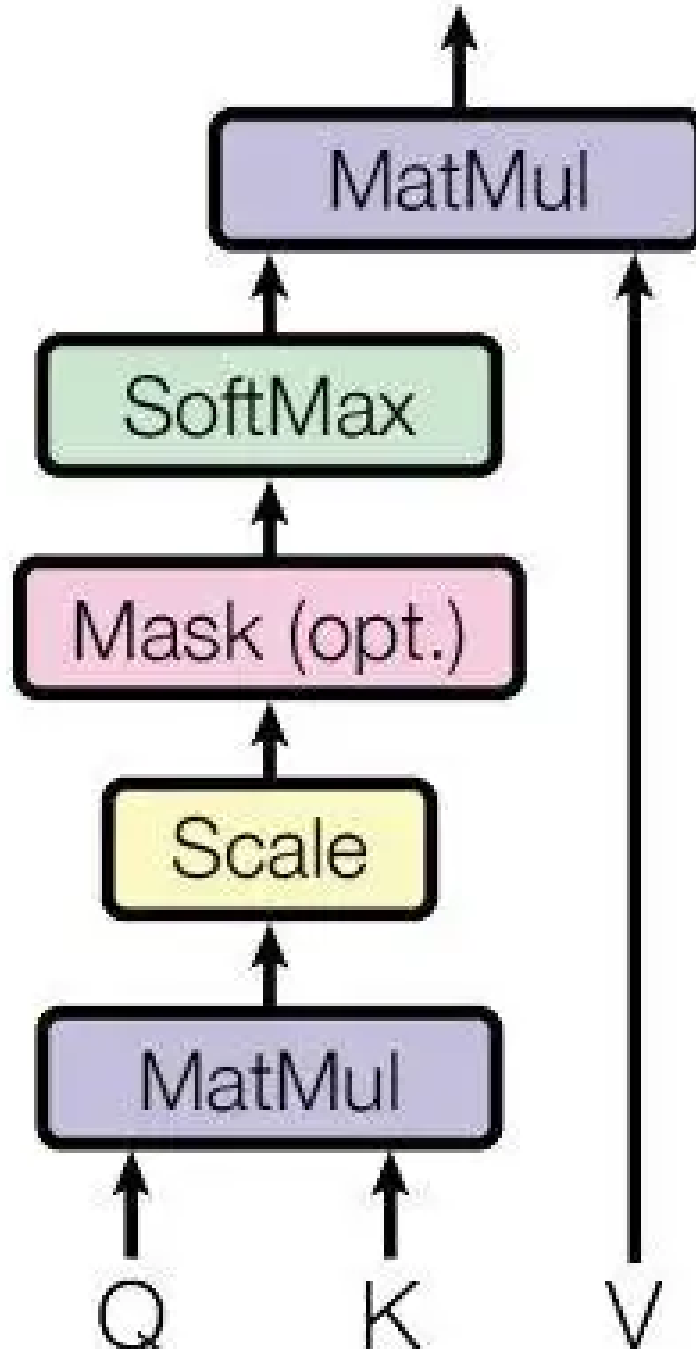
Transformer Encoder 和 Decoder

上图是论文中 Transformer 的内部结构图，左侧为 Encoder block，右侧为 Decoder block。红色圈中的部分为 **Multi-Head Attention**，是由多个 **Self-Attention**组成的，可以看到 Encoder block 包含一个 Multi-Head Attention，而 Decoder block 包含两个 Multi-Head Attention (其中有一个用到 Masked)。Multi-Head Attention 上方还包括一个 Add & Norm 层，Add 表示残差连接 ([Residual Connection](#)+) 用于防止网络退化，Norm 表示 [Layer Normalization](#)，用于对每一层的激活值进行归一化。

因为 **Self-Attention**是 Transformer 的重点，所以我们重点关注 Multi-Head Attention 以及 Self-Attention，首先详细了解一下 Self-Attention 的内部逻辑。

3.1 Self-Attention 结构

Scaled Dot-Product Attention



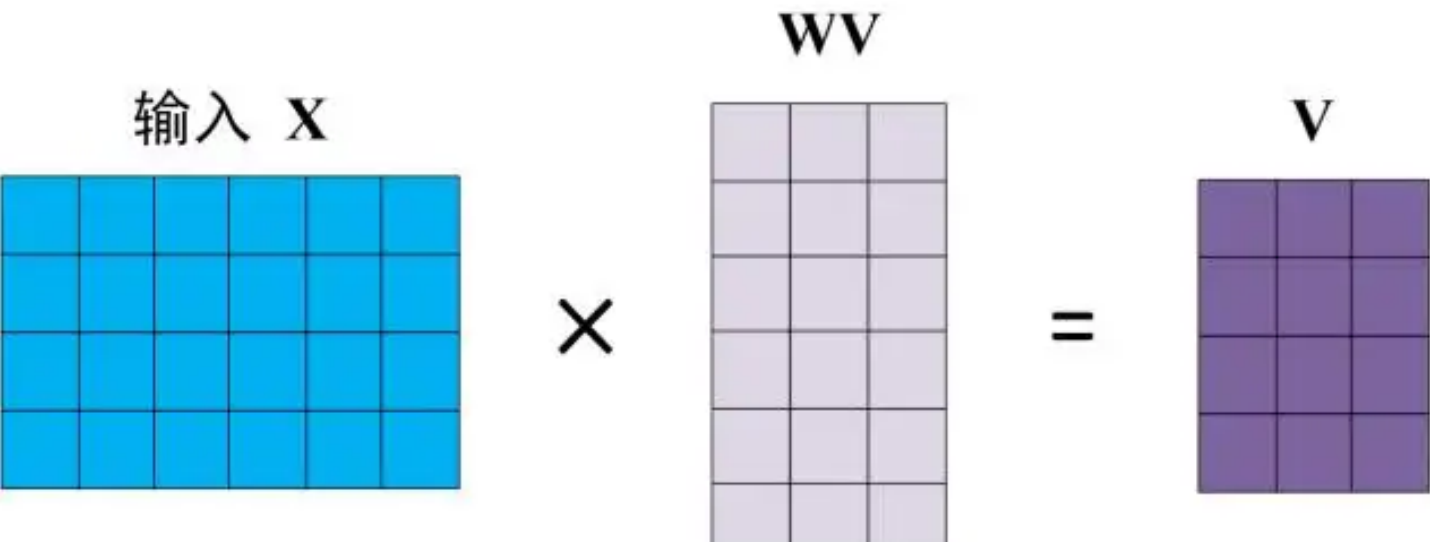
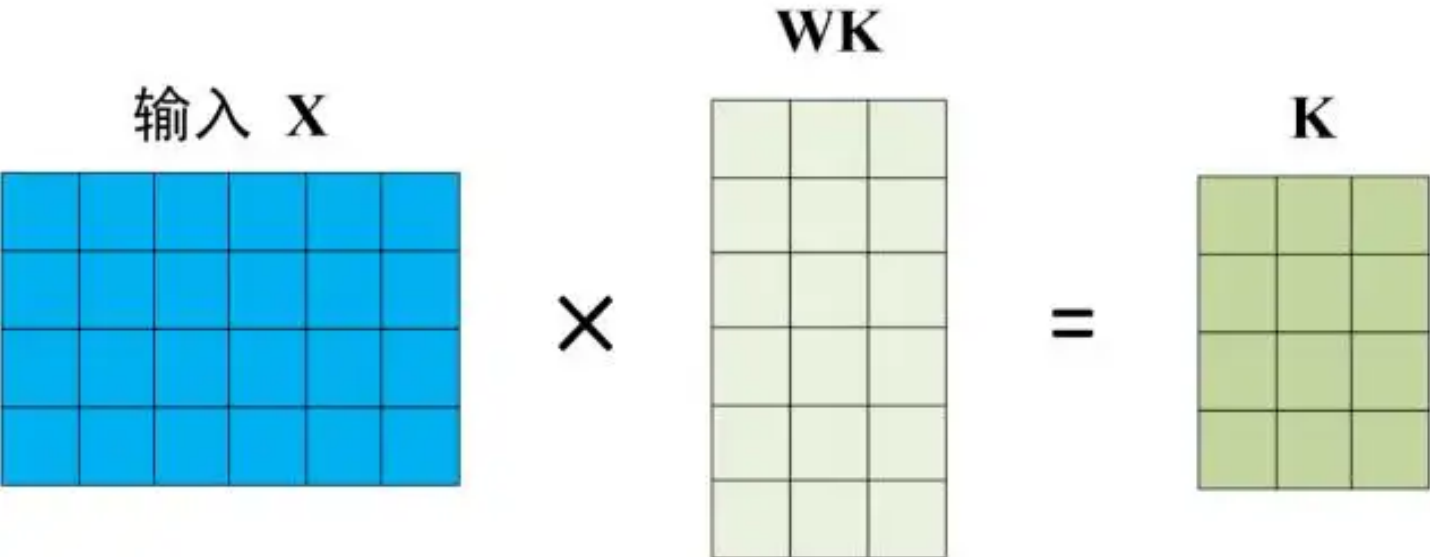
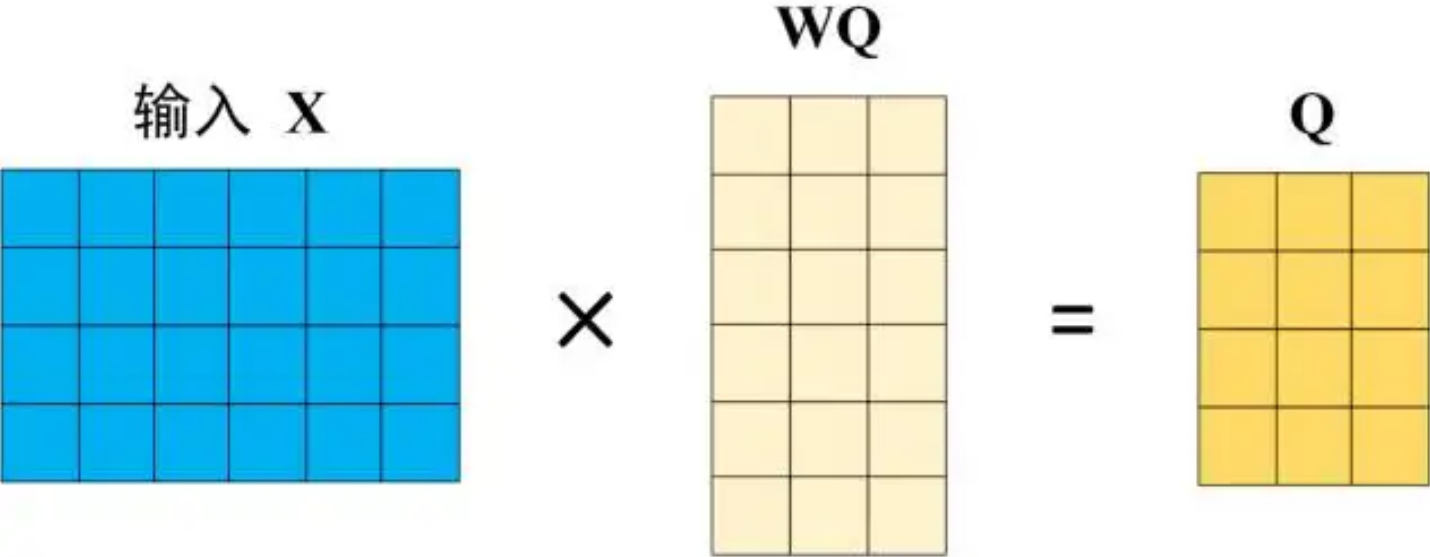
Self-Attention 结构

上图是 Self-Attention 的结构，在计算的时候需要用到矩阵 Q (查询), K (键值), V (值)。在实际中，Self-Attention 接收的是输入(单词的表示向量组成的矩阵 X) 或者上一个 Encoder block 的输出。而 Q,K,V

正是通过 Self-Attention 的输入进行线性变换得到的。

3.2 Q, K, V 的计算

Self-Attention 的输入用矩阵 X 进行表示，则可以使用线性变换矩阵 W_Q, W_K, W_V 计算得到 Q, K, V 。计算如下图所示，注意 X, Q, K, V 的每一行都表示一个单词。



3.3 Self-Attention 的输出

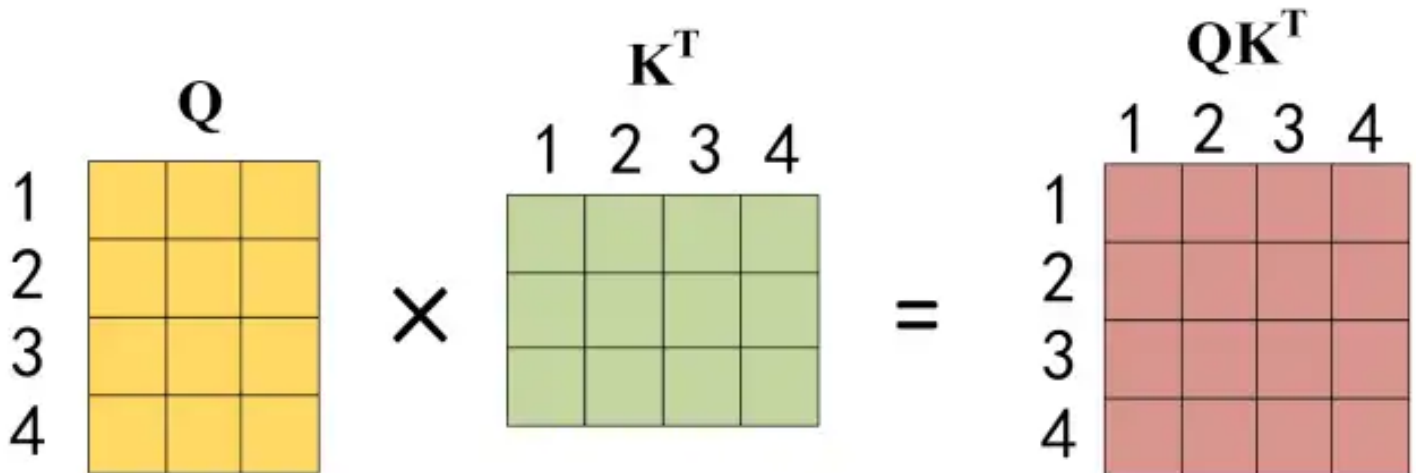
得到矩阵 Q, K, V 之后就可以计算出 Self-Attention 的输出，计算的公式如下：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

d_k 是 Q, K 矩阵的列数，即向量维度

Self-Attention 的输出

公式中计算矩阵 Q 和 K 每一行向量的内积，为了防止内积过大，因此除以 d_k 的平方根。 Q 乘以 K 的转置后，得到的矩阵行列数都为 n ， n 为句子单词数，这个矩阵可以表示单词之间的 attention 强度。下图为 Q 乘以 K^T ，1234 表示的是句子中的单词。



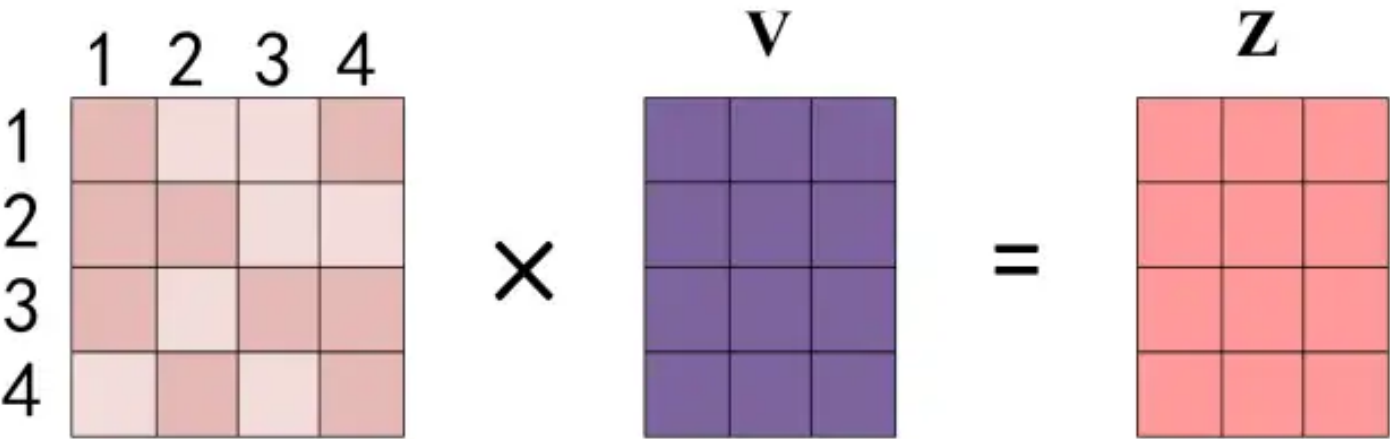
Q 乘以 K 的转置的计算

得到 QK^T 之后，使用 Softmax 计算每一个单词对于其他单词的 attention 系数，公式中的 Softmax 是对矩阵的每一行进行 Softmax，即每一行的和都变为 1。



对矩阵的每一行进行 Softmax

得到 Softmax 矩阵之后可以和 V 相乘，得到最终的输出 Z 。



Self-Attention 输出

上图中 Softmax 矩阵的第 1 行表示单词 1 与其他所有单词的 attention 系数，最终单词 1 的输出 Z_1 等于所有单词 i 的值 V_i 根据 attention 系数的比例加在一起得到，如下图所示：

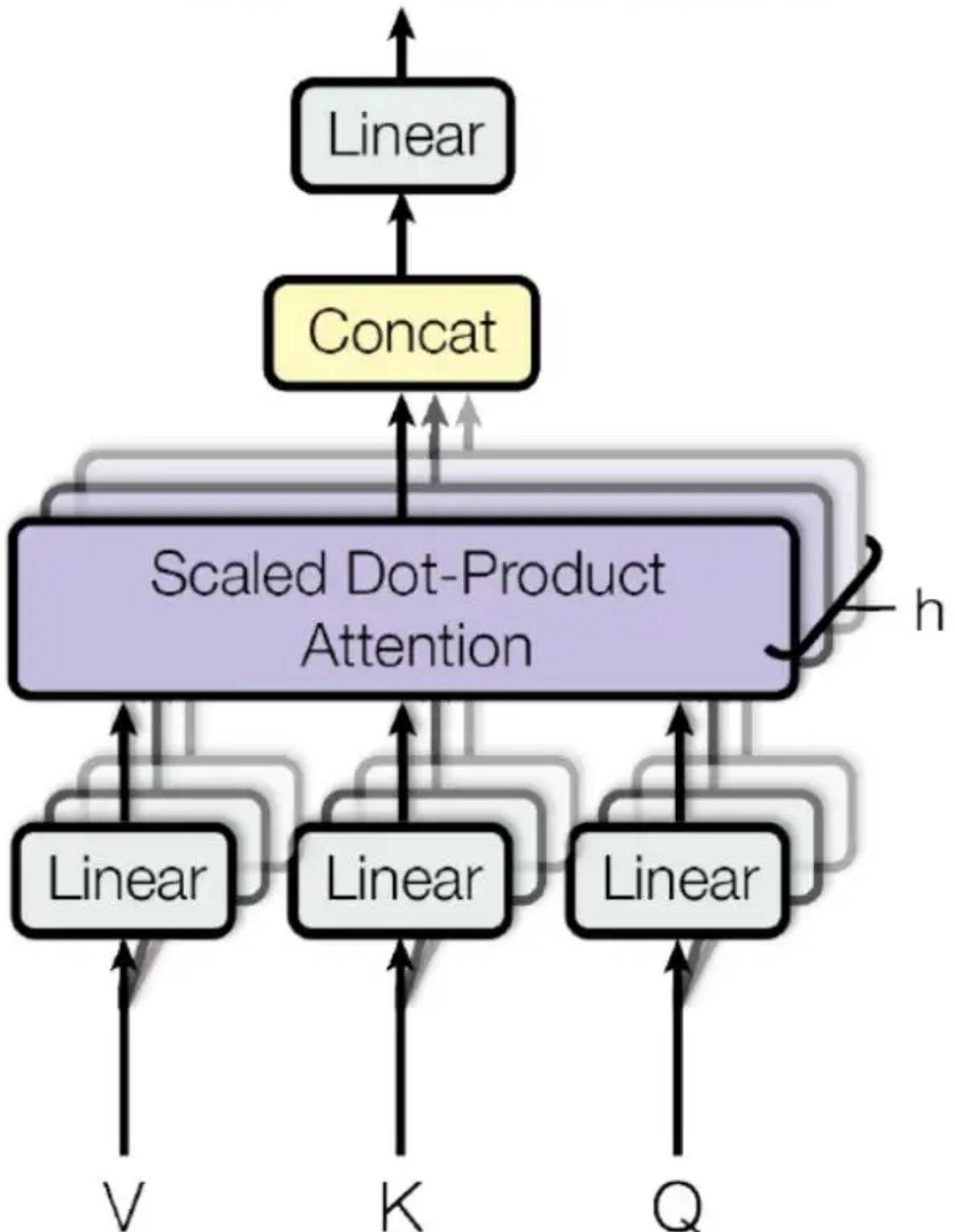
$$\mathbf{Z}_1 \begin{bmatrix} & & & \end{bmatrix} = 1 \begin{matrix} 1 & 2 & 3 & 4 \\ \begin{bmatrix} 0.3 & 0.2 & 0.2 & 0.3 \end{bmatrix} \end{matrix} \times \begin{matrix} \mathbf{V} \\ \begin{bmatrix} & & & \\ & & & \\ & & & \\ & & & \end{bmatrix} \end{matrix}$$
$$= 0.3 \times \begin{bmatrix} & & & \end{bmatrix} + 0.2 \times \begin{bmatrix} & & & \end{bmatrix} + 0.2 \times \begin{bmatrix} & & & \end{bmatrix} + 0.3 \times \begin{bmatrix} & & & \end{bmatrix}$$

Zi 的计算方法

3.4 Multi-Head Attention

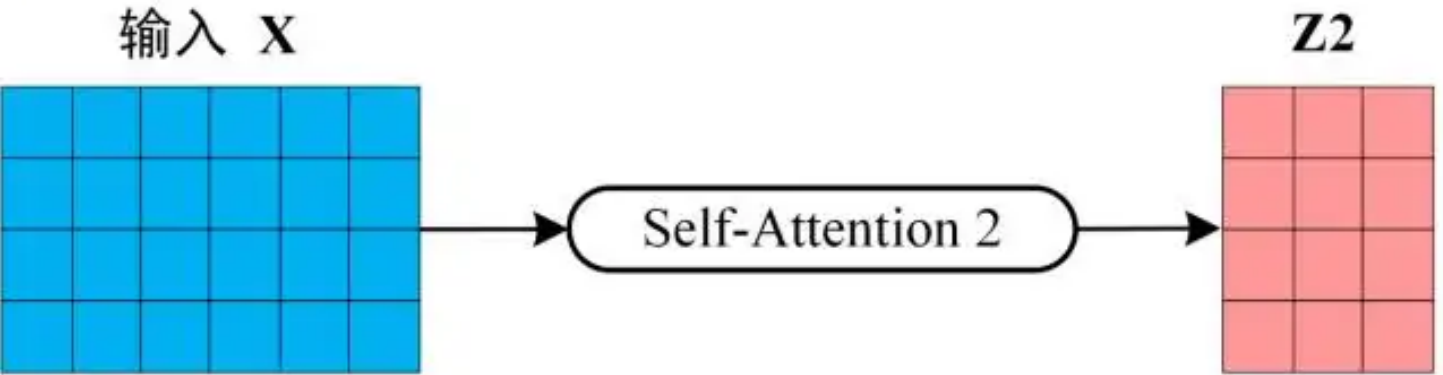
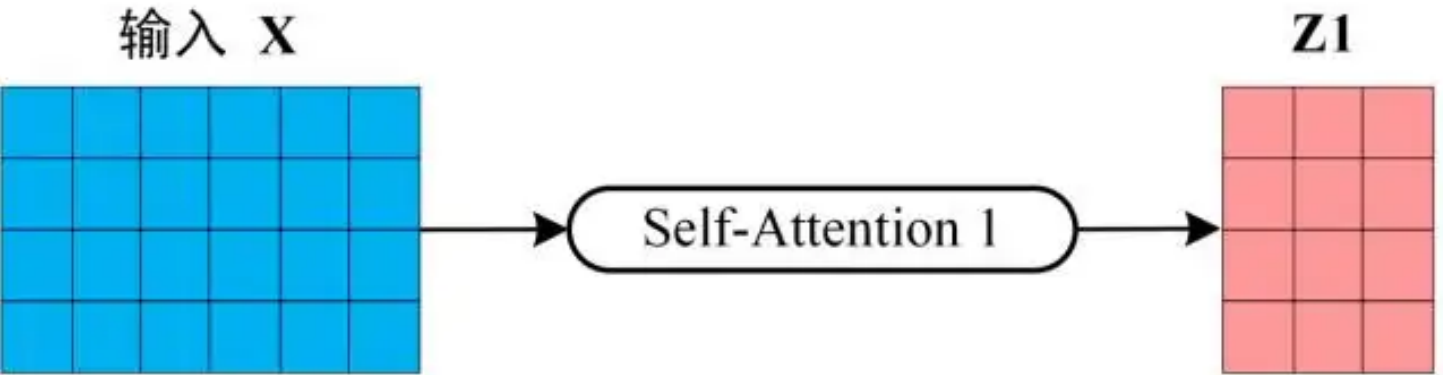
在上一步，我们已经知道怎么通过 Self-Attention 计算得到输出矩阵 Z，而 Multi-Head Attention 是由多个 Self-Attention 组合形成的，下图是论文中 Multi-Head Attention 的结构图。

Multi-Head Attention

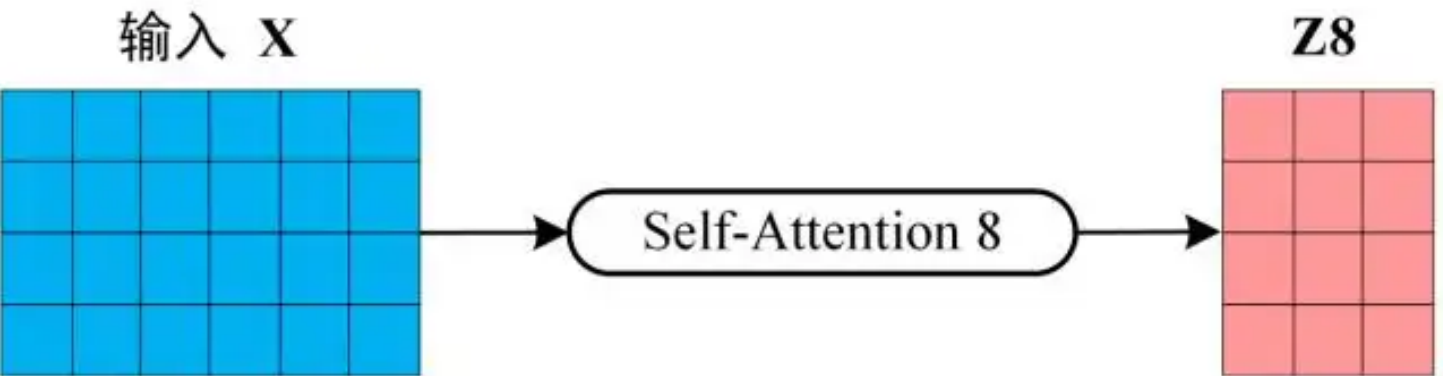


Multi-Head Attention

从上图可以看到 Multi-Head Attention 包含多个 Self-Attention 层，首先将输入 $\mathbf{X}$ 分别传递到 h 个不同的 Self-Attention 中，计算得到 h 个输出矩阵 $\mathbf{Z}$ 。下图是 $h=8$ 时候的情况，此时会得到 8 个输出矩阵 $\mathbf{Z}$ 。

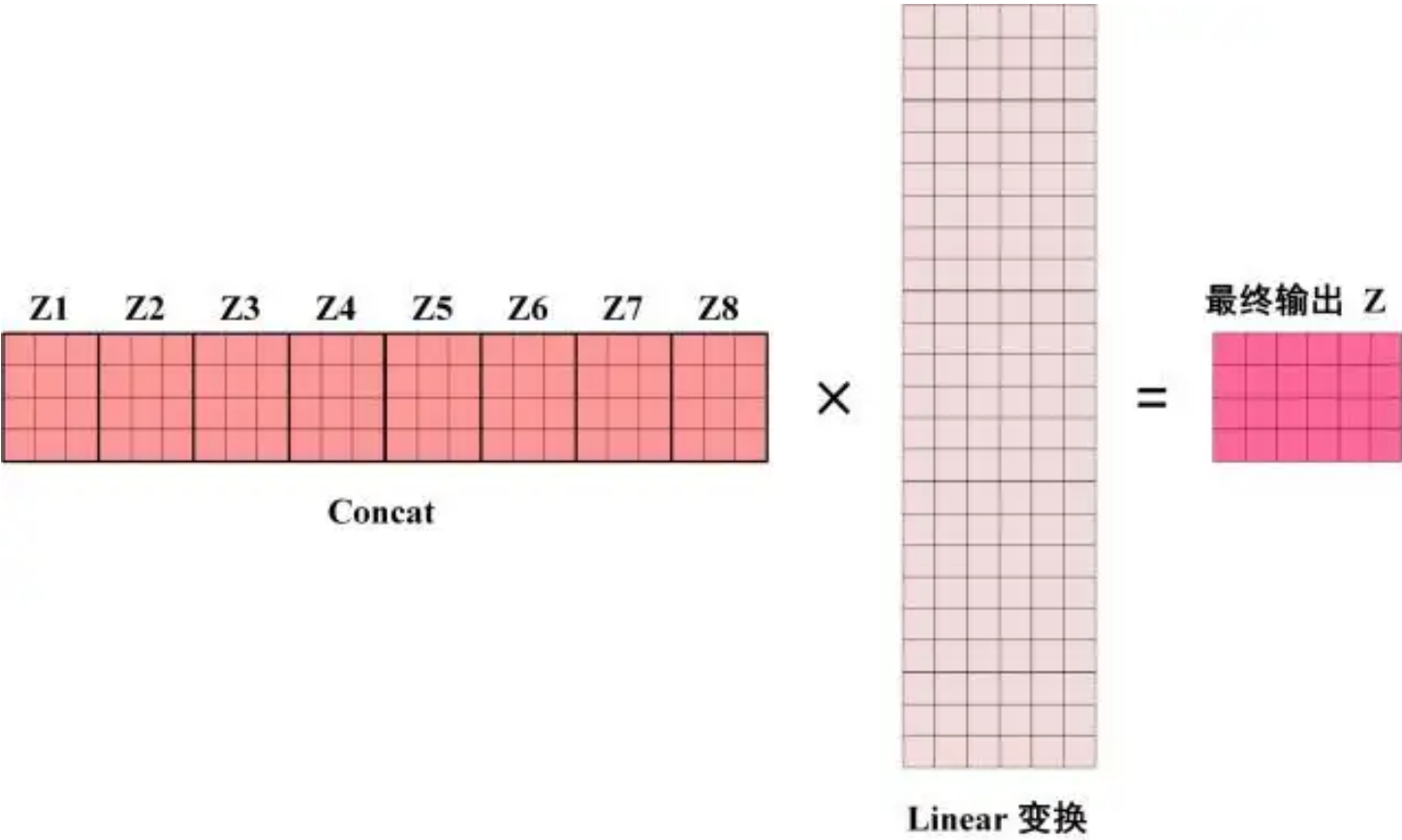


• • • • •



多个 Self-Attention

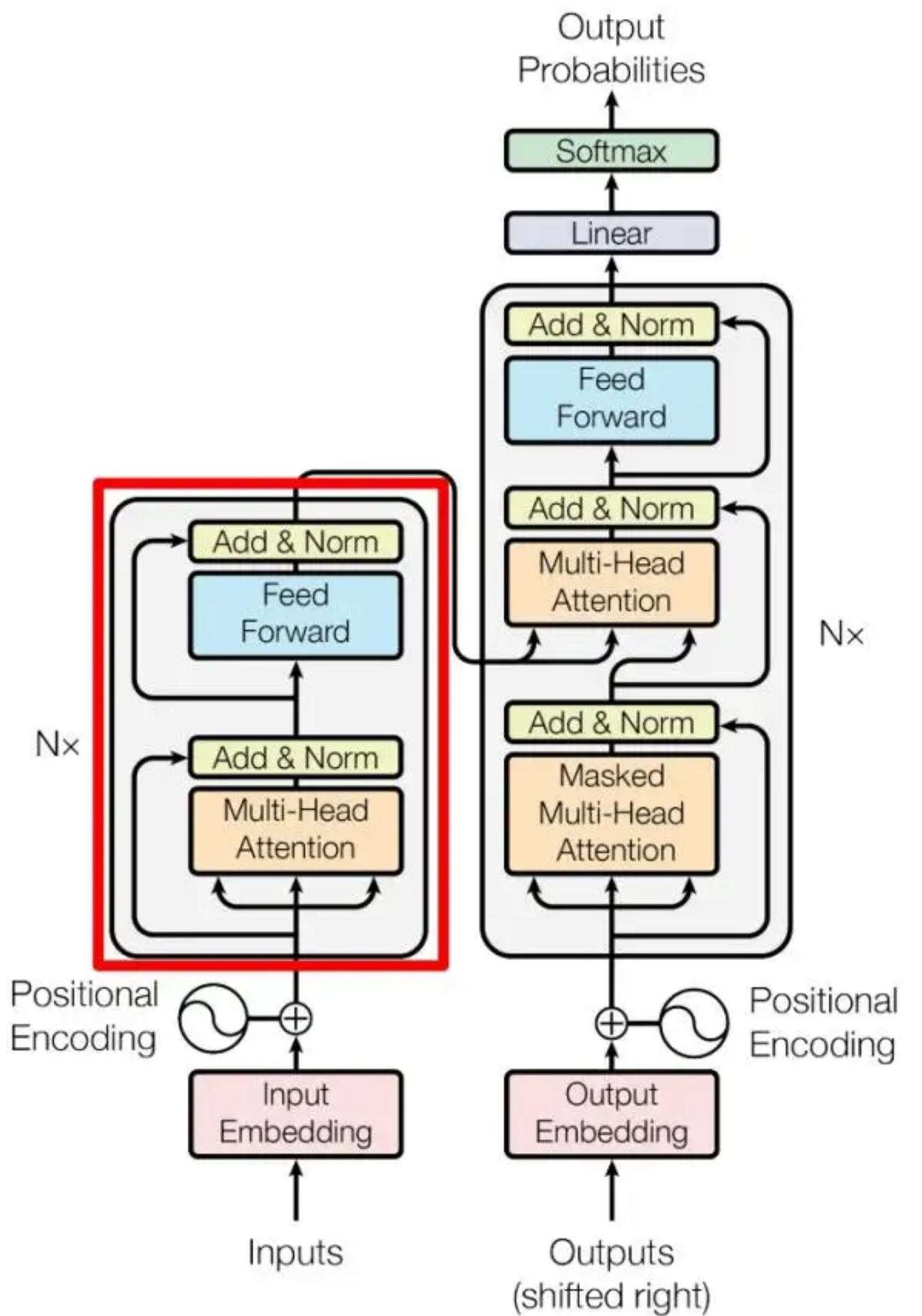
得到 8 个输出矩阵 Z_1 到 Z_8 之后，Multi-Head Attention 将它们拼接在一起 (**Concat**)，然后传入一个**Linear**层，得到 Multi-Head Attention 最终的输出**Z**。



Multi-Head Attention 的输出

可以看到 Multi-Head Attention 输出的矩阵**Z**与其输入的矩阵**X**的维度是一样的。

4. Encoder 结构



Transformer Encoder block

上图红色部分是 Transformer 的 Encoder block 结构，可以看到是由 Multi-Head Attention, **Add & Norm**, **Feed Forward**, **Add & Norm** 组成的。刚刚已经了解了 Multi-Head Attention 的计算过程，现在了解一下 Add & Norm 和 Feed Forward 部分。

4.1 Add & Norm

Add & Norm 层由 Add 和 Norm 两部分组成，其计算公式如下：

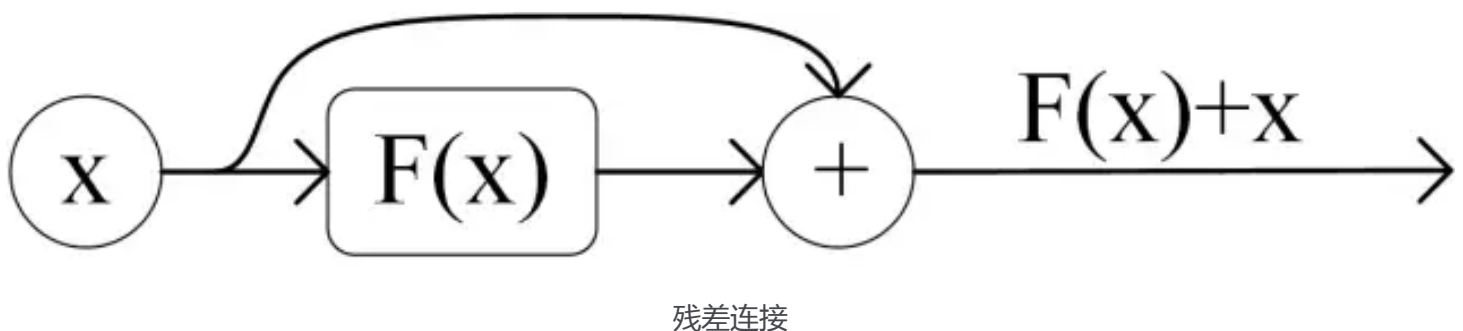
$$\text{LayerNorm}(X + \text{MultiHeadAttention}(X))$$

$$\text{LayerNorm}(X + \text{FeedForward}(X))$$

Add & Norm 公式

其中 **X** 表示 Multi-Head Attention 或者 Feed Forward 的输入，MultiHeadAttention(**X**) 和 FeedForward(**X**) 表示输出 (输出与输入 **X** 维度是一样的，所以可以相加)。

Add指 $X + \text{MultiHeadAttention}(X)$ ，是一种残差连接，通常用于解决多层网络训练的问题，可以让网络只关注当前差异的部分，在 ResNet 中经常用到：



Norm指 Layer Normalization，通常用于 RNN 结构，Layer Normalization 会将每一层神经元的输入都转成均值方差都一样的，这样可以加快收敛。

4.2 Feed Forward

Feed Forward 层比较简单，是一个两层的全连接层，第一层的激活函数为 Relu，第二层不使用激活函数，对应的公式如下。

$$\max(0, XW_1 + b_1)W_2 + b_2$$

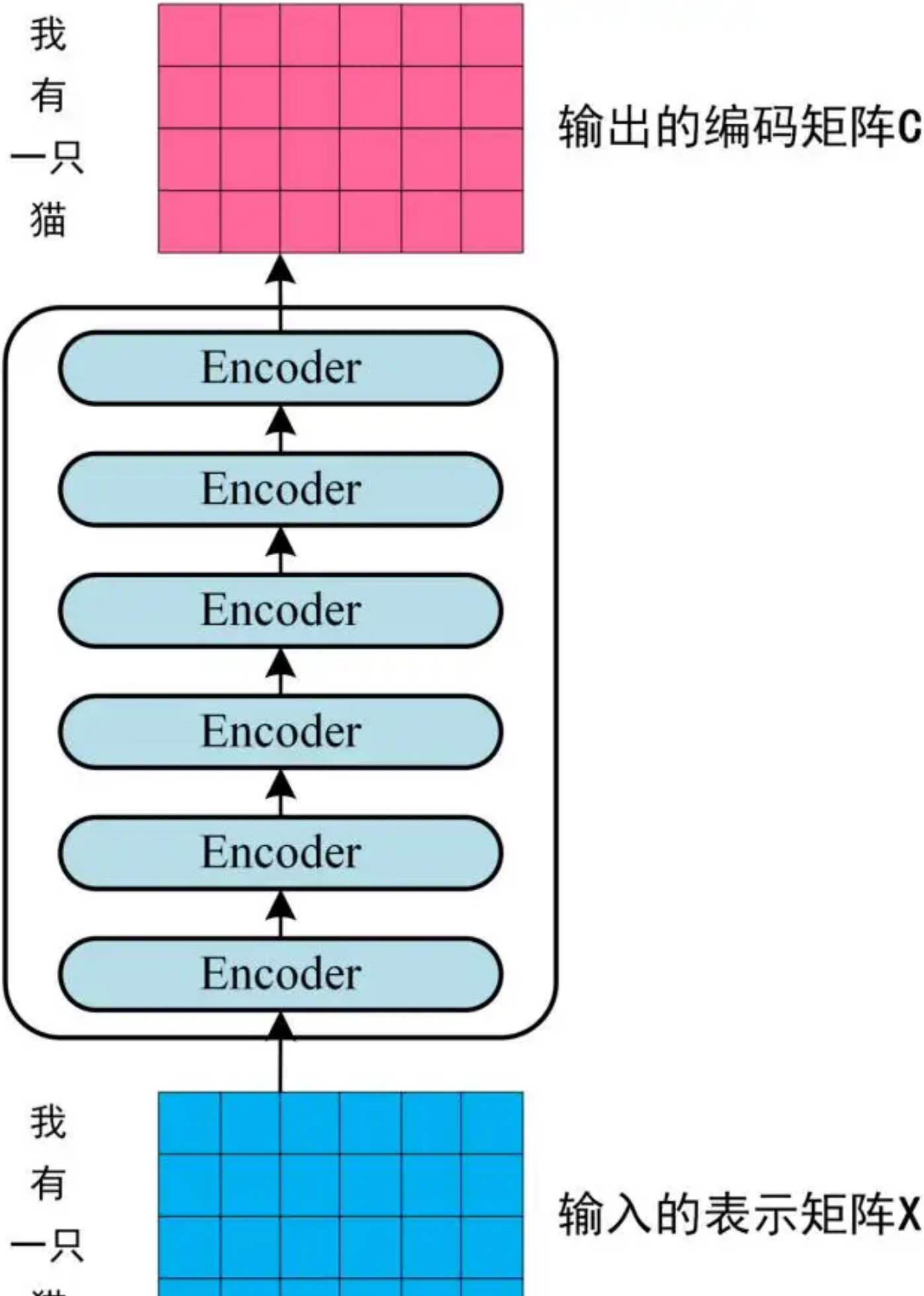
Feed Forward

X是输入，Feed Forward 最终得到的输出矩阵的维度与**X**一致。

4.3 组成 Encoder

通过上面描述的 Multi-Head Attention, Feed Forward, Add & Norm 就可以构造出一个 Encoder block，Encoder block 接收输入矩阵 $X_{(n \times d)}$ ，并输出一个矩阵 $O_{(n \times d)}$ 。通过多个 Encoder block 叠加就可以组成 Encoder。

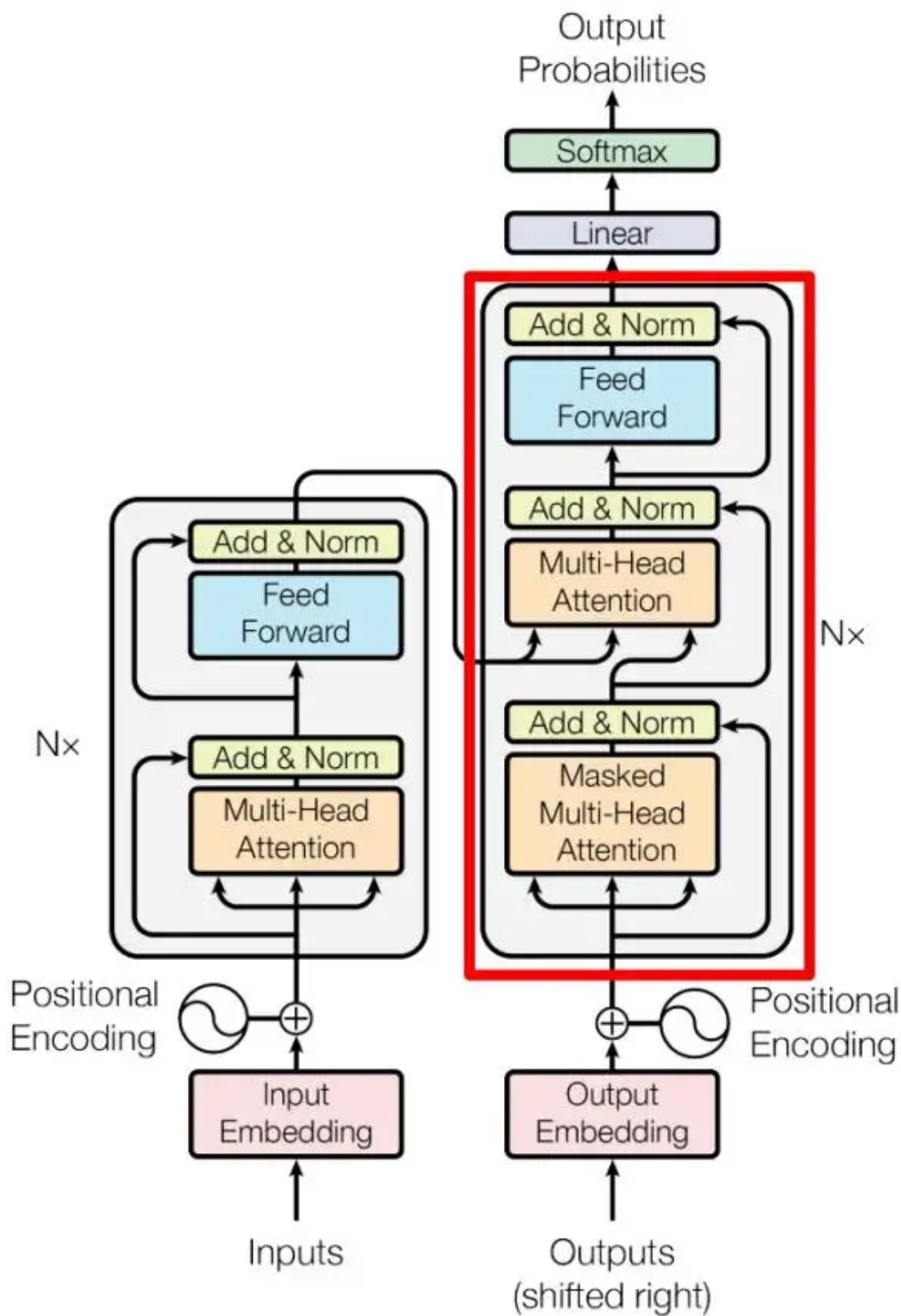
第一个 Encoder block 的输入为句子单词的表示向量矩阵，后续 Encoder block 的输入是前一个 Encoder block 的输出，最后一个 Encoder block 输出的矩阵就是**编码信息矩阵 C**，这一矩阵后续会用到 Decoder 中。



猫



5. Decoder 结构



Transformer Decoder block

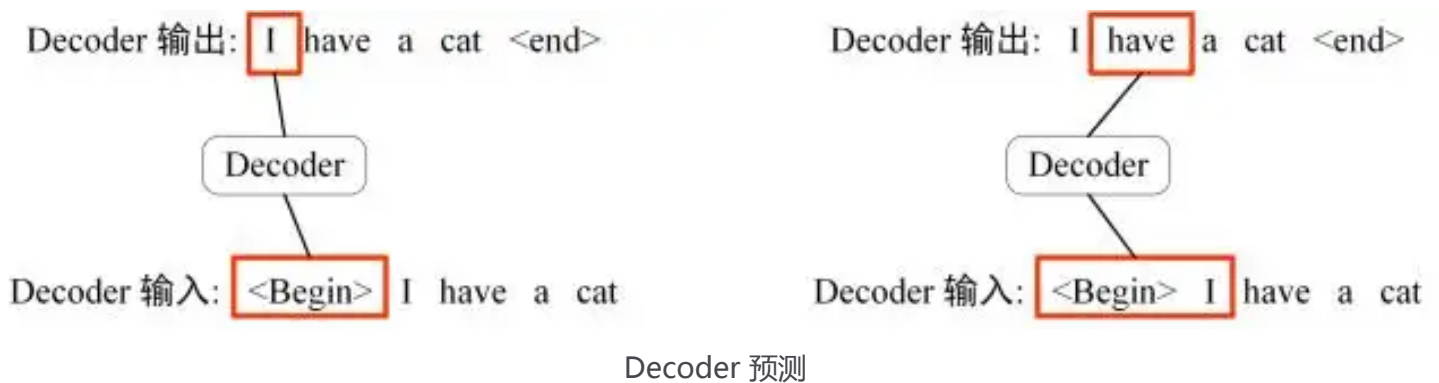
上图红色部分为 Transformer 的 Decoder block 结构，与 Encoder block 相似，但是存在一些区别：

- 包含两个 Multi-Head Attention 层。
- 第一个 Multi-Head Attention 层采用了 Masked 操作。
- 第二个 Multi-Head Attention 层的 K, V 矩阵使用 Encoder 的编码信息矩阵 C 进行计算，而 Q 使用上一个 Decoder block 的输出计算。
- 最后有一个 Softmax 层计算下一个翻译单词的概率。

5.1 第一个 Multi-Head Attention

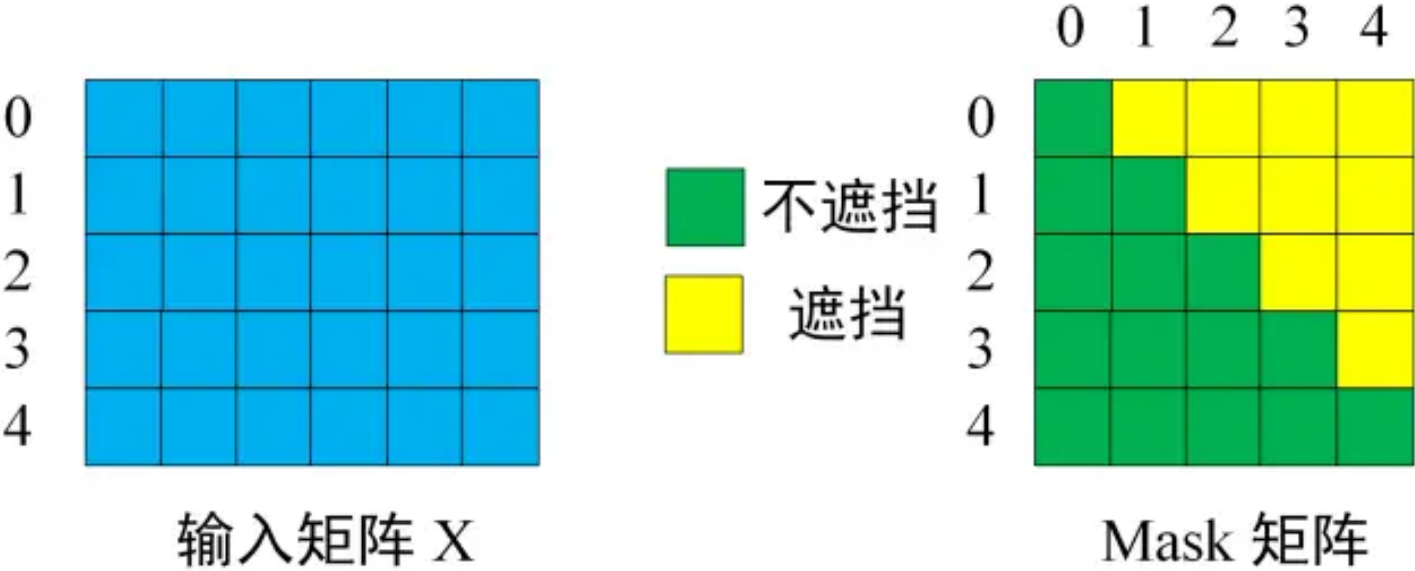
Decoder block 的第一个 Multi-Head Attention 采用了 Masked 操作，因为在翻译的过程中是顺序翻译的，即翻译完第 i 个单词，才可以翻译第 $i+1$ 个单词。通过 Masked 操作可以防止第 i 个单词知道 $i+1$ 个单词之后的信息。下面以 "我有一只猫" 翻译成 "I have a cat" 为例，了解一下 Masked 操作。

下面的描述中使用了类似 [Teacher Forcing](#) 的概念，不熟悉 Teacher Forcing 的童鞋可以参考以下上一篇文章 Seq2Seq 模型详解。在 Decoder 的时候，是需要根据之前的翻译，求解当前最有可能的翻译，如下图所示。首先根据输入 "<Begin>" 预测出第一个单词为 "I"，然后根据输入 "<Begin> I" 预测下一个单词 "have"。



Decoder 可以在训练的过程中使用 Teacher Forcing 并且并行化训练，即将正确的单词序列 (<Begin> I have a cat) 和对应输出 (I have a cat <end>) 传递到 Decoder。那么在预测第 i 个输出时，就要将第 $i+1$ 之后的单词掩盖住，**注意 Mask 操作是在 Self-Attention 的 Softmax 之前使用的**，下面用 0 1 2 3 4 5 分别表示 "<Begin> I have a cat <end>"。

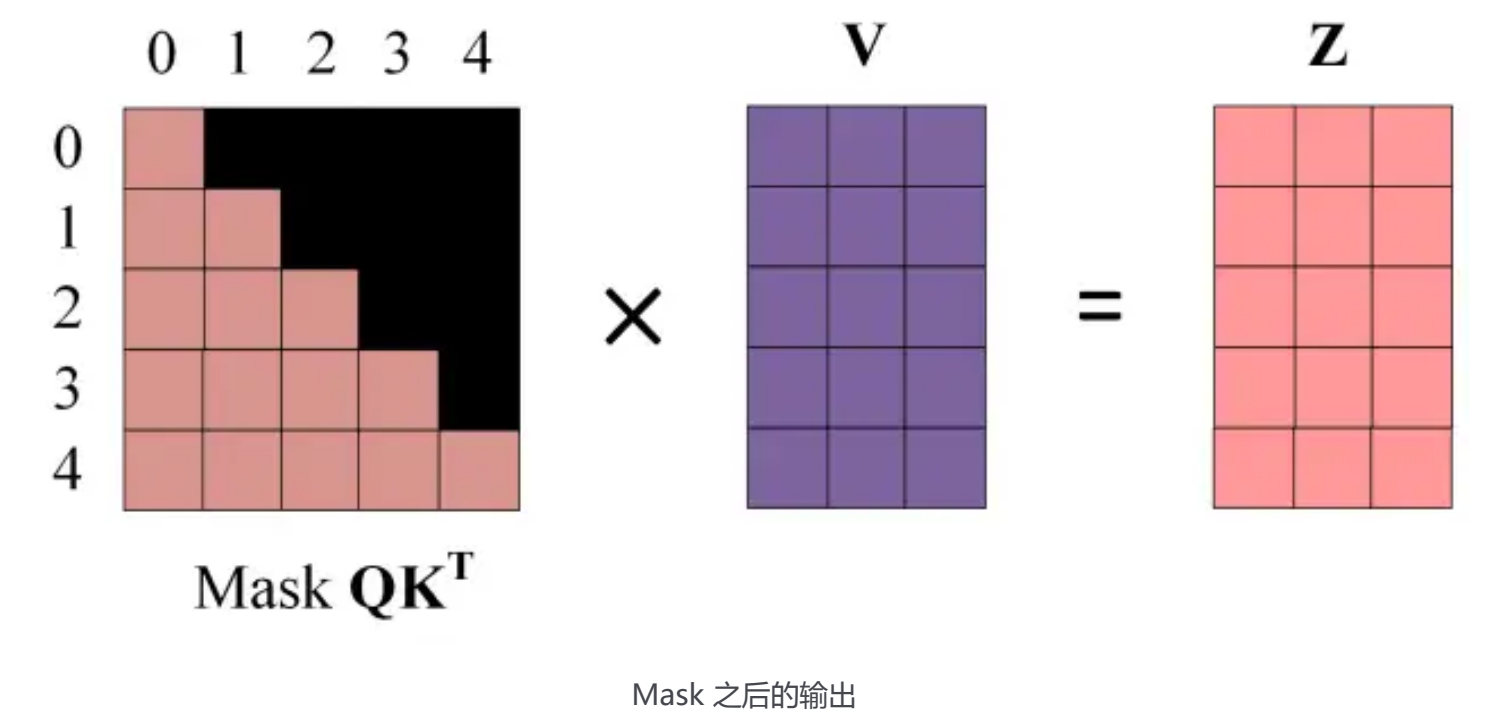
第一步：是 Decoder 的输入矩阵和 **Mask** 矩阵，输入矩阵包含 "<Begin> I have a cat" (0, 1, 2, 3, 4) 五个单词的表示向量，**Mask** 是一个 5×5 的矩阵。在 **Mask** 可以发现单词 0 只能使用单词 0 的信息，而单词 1 可以使用单词 0, 1 的信息，即只能使用之前的信息。





得到 $\text{Mask } QK^T$ 之后在 $\text{Mask } QK^T$ 上进行 Softmax，每一行的和都为 1。但是单词 0 在单词 1, 2, 3, 4 上的 attention score 都为 0。

第四步：使用 $\text{Mask } QK^T$ 与矩阵 V 相乘，得到输出 Z ，则单词 1 的输出向量 Z_1 是只包含单词 1 信息的。



第五步：通过上述步骤就可以得到一个 Mask Self-Attention 的输出矩阵 Z_i ，然后和 Encoder 类似，通过 Multi-Head Attention 拼接多个输出 Z_i 然后计算得到第一个 Multi-Head Attention 的输出 Z ， Z 与输入 X 维度一样。

5.2 第二个 Multi-Head Attention

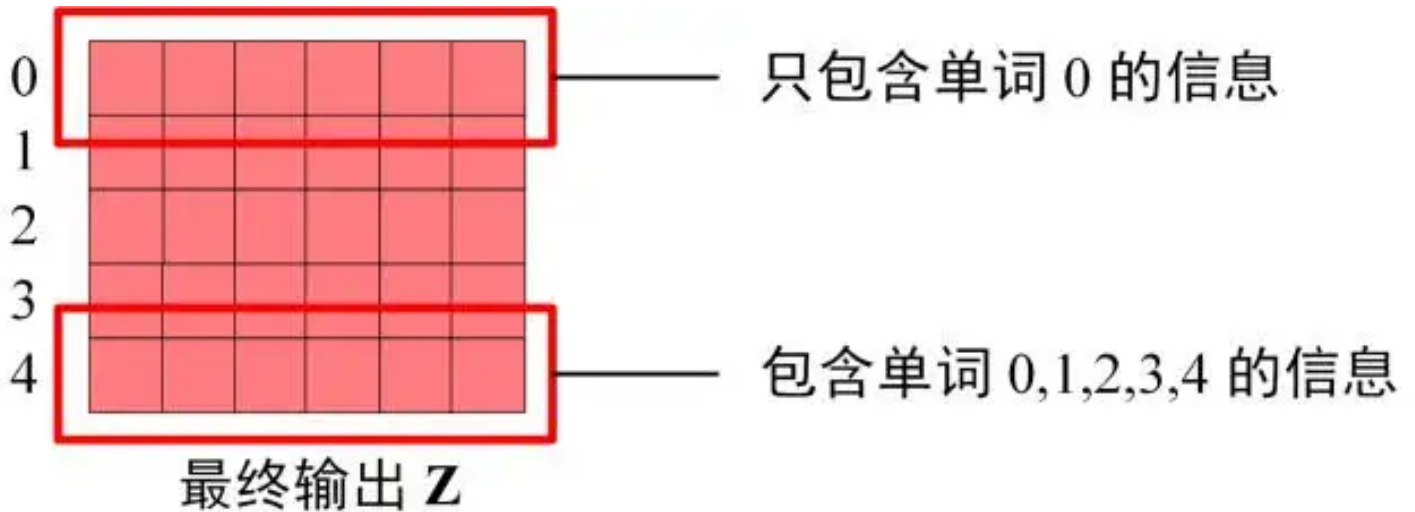
Decoder block 第二个 Multi-Head Attention 变化不大，主要的区别在于其中 Self-Attention 的 $\mathbf{K}$, $\mathbf{V}$ 矩阵不是使用 上一个 Decoder block 的输出计算的，而是使用 **Encoder 的编码信息矩阵 $\mathbf{C}$** 计算的。

根据 Encoder 的输出 $\mathbf{C}$ 计算得到 $\mathbf{K}$, $\mathbf{V}$ ，根据上一个 Decoder block 的输出 $\mathbf{Z}$ 计算 $\mathbf{Q}$ (如果是第一个 Decoder block 则使用输入矩阵 $\mathbf{X}$ 进行计算)，后续的计算方法与之前描述的一致。

这样做的好处是在 Decoder 的时候，每一位单词都可以利用到 Encoder 所有单词的信息 (这些信息无需 **Mask**)。

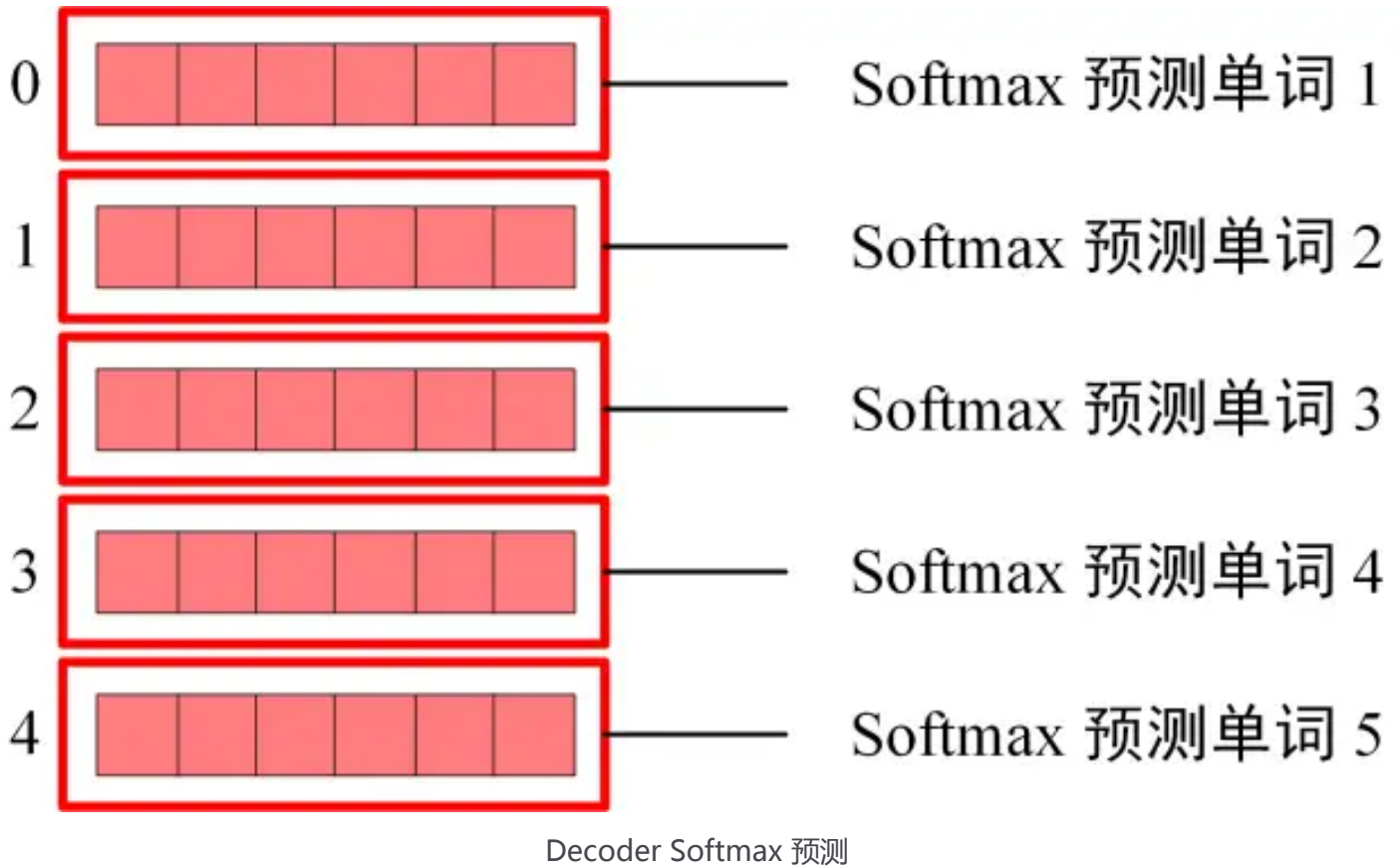
5.3 Softmax 预测输出单词

Decoder block 最后的部分是利用 Softmax 预测下一个单词，在之前的网络层我们可以得到一个最终的输出 $\mathbf{Z}$ ，因为 Mask 的存在，使得单词 0 的输出 $\mathbf{Z}_0$ 只包含单词 0 的信息，如下：



Decoder Softmax 之前的 $\mathbf{Z}$

Softmax 根据输出矩阵的每一行预测下一个单词：



这就是 Decoder block 的定义，与 Encoder 一样，Decoder 是由多个 Decoder block 组合而成。

6. Transformer 总结

- Transformer 与 RNN 不同，可以比较好地并行训练。
- Transformer 本身是不能利用单词的顺序信息的，因此需要在输入中添加位置 Embedding，否则 Transformer 就是一个词袋模型了。
- Transformer 的重点是 Self-Attention 结构，其中用到的 Q, K, V 矩阵通过输入进行线性变换得到。
- Transformer 中 Multi-Head Attention 中有多个 Self-Attention，可以捕获单词之间多种维度上的相关系数 attention score。

[\[1\]](#) [\[2\]](#)



初识CV

👍 109 次咨询

★★★★★5.0



西安电子科技大学 电子科学与技术硕士



43633 次赞同

去咨询 ›

参考

1. [论文:Attention Is All You Need https://arxiv.org/abs/1706.03762](https://arxiv.org/abs/1706.03762)

2. [Transformer 模型详解 https://baijiahao.baidu.com/s?](https://baijiahao.baidu.com/s?id=1651219987457222196&wfr=spider&for=pc)

[id=1651219987457222196&wfr=spider&for=pc](https://baijiahao.baidu.com/s?id=1651219987457222196&wfr=spider&for=pc)

发布于 2024-05-08 03:18

▲赞同 1329 ▼

● 18 条评论

🔗分享

★收藏

♥喜欢

...

收起 ▼



[bigpunch](#)

马甲是暂时的囚笼，思想是永久的野火

+ 关注

2922 人赞同了该回答

›

你脑子没问题，是那些教程有问题。

Transformer+的训练方式跟你理解的神经网络+一模一样，就是反向传播+，就是调整权重参数，没有任何新东西。

你之所以困惑，是因为99%的教程犯了一个致命错误：它们花大量篇幅讲**注意力机制+**的前向传播过程，把Q、K、V的矩阵运算讲得天花乱坠，然后到了训练部分就一笔带过甚至直接跳过。这给人一种错觉，好像Transformer有什么神秘的训练方法。

没有。真的没有。

你的困惑根源是把前向传播和训练机制搞混了。我先帮你理清一个概念上的混乱。

当人们讲Transformer的时候，讲的是它的网络结构和前向传播的计算流程。Q、K、V那套公式，Softmax加权求和，多头注意力，这些东西描述的是数据怎么从输入流到输出。这是前向传播。

而你问的反向传播怎么调参数，这是训练机制。

关键来了：训练机制跟网络结构是两回事。不管你的网络结构长什么样，只要它是由可微分的操作组成的，反向传播就能用。Transformer里面的每一个操作，矩阵乘法、Softmax、[LayerNorm](#)、[残差连接](#)，全都是可微分的。所以反向传播直接就能用，跟训练一个普通的三层全连接网络没有本质区别。

你之前理解的那套东西，损失函数对参数求偏导，链式法则一层层往回传，梯度下降更新权重，这套东西在Transformer里面一字不改，直接照搬。

所以你的问题其实不应该是Transformer怎么反向传播，而应该是：Transformer里面到底有哪些参数是需要训练的？

这才是你真正没搞清楚的地方。

Transformer里到底有哪些可训练参数？

第一块：[词嵌入层](#) + Embedding

输入是一串token ID，比如 [101, 2769, 3221, 4638, 102]。词嵌入层是一个查找表，维度是 词表大小 $\times$ 嵌入维度，比如 50000×768 。这整个嵌入矩阵就是可训练参数，跟你训练word2vec没区别。

第二块：[位置编码](#) + Positional Encoding

原始论文用的是正弦余弦函数算出来的固定值，不需要训练。但现在很多模型用的是可学习的位置嵌入，比如[BERT](#)和[GPT系列](#)，这又是一个 最大序列长度 $\times$ 嵌入维度的矩阵，也是可训练参数。

第三块：[自注意力层](#)，这是重点

这里是大家讲Q、K、V的地方，但大多数教程只讲了公式没讲参数。我给你说清楚。

Q、K、V是怎么来的？是输入X分别乘以三个权重矩阵得到的：

$$Q = X \times W_q$$

$$K = X \times W_k$$

$$V = X \times W_v$$

W_q 、 W_k 、 W_v 这三个矩阵就是可训练参数。假设嵌入维度是768，那每个矩阵就是 768×768 的，三个加起来就是将近180万个参数。

然后多头注意力算完之后，还要再乘一个输出投影矩阵 W_o ，又是 768×768 的参数。

所以一个自注意力层里面，光权重矩阵就有四个： W_q 、 W_k 、 W_v 、 W_o 。这就是需要训练的东西。

第四块：前馈神经网络+ FFN

每个Transformer层里面，注意力算完之后还要过一个两层的前馈网络。标准配置是先升维到 $768 \times 4 = 3072$ ，过一个激活函数，再降回 768。

这两层的权重矩阵分别是 768×3072 和 3072×768 ，外加两个偏置向量。参数量比注意力层还大，差不多470万个参数。

第五块：LayerNorm层

每个子层后面都有一个Layer Normalization，它有两个可训练参数向量，一个是缩放系数gamma，一个是偏移系数beta，每个都是768维。参数量不大，但也是要训练的。

好，我帮你算一下。一个标准的Transformer层，包含一个多头自注意力和一个FFN，参数量大概是：

- 注意力部分： $768 \times 768 \times 4 =$ 约240万
- FFN部分： $768 \times 3072 \times 2 =$ 约470万
- LayerNorm： $768 \times 4 =$ 约3000

一层就是700多万参数。BERT-base有12层，光这部分就是8000多万。再加上词嵌入层的3800万，总共1.1亿参数左右，跟官方公布的110M对得上。

我把这些参数展开了讲，就是想让你明白：**Transformer不是什么玄学，它就是一堆矩阵乘法和非线性变换堆起来的，每个矩阵都是可训练参数。**

反向传播的时候，损失函数对这些参数求梯度，梯度下降更新参数，跟你训练一个最简单的MLP没有任何本质区别。PyTorch的autograd或者TensorFlow的GradientTape帮你自动算好了所有梯度，你甚至不需要手推公式。

为什么教程都不讲这些？因为讲这些没啥可讲的。

你想想，一个教程作者写Transformer介绍，如果训练部分写的是这就是普通的反向传播没什么特别的，读者会觉得你在水字数。所以大家都把篇幅花在注意力机制的原理上，那玩意儿看起来高大上，有公式有图，写起来也有东西可写。

但这就造成了一个信息缺口：初学者看完之后知道Transformer的前向传播是怎么算的，但对于它为什么能训练、怎么训练完全没概念。

还有一个原因是，写教程的人默认你已经懂反向传播了。他们觉得既然你知道神经网络怎么训练，那Transformer作为神经网络的一种，你自然知道它也是一样训练的。但他们没想到，正是因为Transformer的前向传播过程太特别了，注意力机制太新颖了，反而让初学者产生了它的训练方式也应该很特别的错觉。

我早年带新人的时候就发现这个问题。有个刚毕业的小伙子，NLP背景很好，LSTM什么的门清，但看了一周Transformer愣是没看懂。后来我问他具体哪里不懂，他说的跟你一模一样：反向传播怎么做的，参数怎么更新的，那些教程都没讲。

我当时给他的回答也是这个：没什么特别的，就是普通的反向传播。他说不可能吧，那个注意力权重是动态算出来的，不是固定的参数，这怎么反向传播？

需要注意的是：注意力权重不是参数，但它是参数的函数。这是很多人混淆的地方。

$\text{Softmax}(QK^T / \sqrt{d})$ 这个东西算出来的注意力权重矩阵，确实不是直接的可训练参数。它是输入X经过Wq和Wk变换之后算出来的中间结果。每次输入不同的句子，这个注意力矩阵都是不一样的。

但这不影响反向传播。

反向传播的核心是链式法则。损失函数L对某个参数 θ 的梯度是：

$$\partial L / \partial \theta = \partial L / \partial \text{输出} \times \partial \text{输出} / \partial \text{中间变量} \times \dots \times \partial \text{某个中间变量} / \partial \theta$$

注意力权重虽然是动态计算的，但它是Wq、Wk这些参数的函数。只要这个函数是可微分的，梯度就能沿着计算图一路传回去。

Softmax可微，矩阵乘法可微，除法可微，所以整条链路都是通的。

具体来说，反向传播的时候，梯度会这样流动：

损失 → 模型输出 → 最后一层FFN → 最后一层注意力输出 → W_o → $V \times$ 注意力权重 → Softmax → QK^T → Q和K → Wq和Wk → 输入X → ... → 一直到词嵌入层

每一步都是标准的矩阵求导。这里面唯一稍微复杂一点的是Softmax的导数，但那也是有解析形式的。如果你对这些矩阵变换和链式法则的几何直觉还不够清晰，去看 3Blue1Brown 的《线性代数的本质》和《微积分的本质》。那个动画演示能让你亲眼看到矩阵是如何扭曲空间的，梯度又是如何沿着函数曲面下滑的。Transformer 里那些看似吓人的公式，拆解到底层就是这些最基础的几何变换。

[3Blue1Brown线性代数笔记：可能是全网最好的中英文整理](#)

如果你想亲手验证这件事，最好的办法是自己用numpy从零实现一遍。网上有一份代码我觉得写得挺清楚的，是哈佛NLP组做的 The Annotated Transformer，用PyTorch从零实现了整个Transformer，每一行代码都有注释。你跑一遍，打印一下每个参数的梯度，就彻底明白了。

还有一个被圈内人称为保姆级的神作——李沐老师的《动手学深度学习》(Dive into Deep Learning) [《动手学深度学习》2.0版本中英文+在线电子书](#)。这本书最大的好处是所见即所得，理论旁边直接放着可运行的 PyTorch 代码。你看着公式如果发懵，直接点一下运行，看数据在代码里是怎么流转的，立马就懂了。

我给你还原一个真实的训练过程。

光讲原理可能还是有点抽象，我给你描述一下实际训练一个Transformer模型是什么体验。

假设你要训练一个做文本分类的BERT模型，数据集是中文情感分析，二分类任务。

第一步，准备数据

你有一堆句子和对应的标签，正面是1负面是0。分词之后转成token ID，加上特殊标记 [CLS] 和 [SEP]，padding到统一长度，打包成batch。

第二步，前向传播

一个batch的数据喂进模型。词嵌入层把token ID转成向量，加上位置编码。然后进入12层Transformer encoder，每一层做自注意力和FFN，中间有残差连接和LayerNorm。最后取 [CLS] 位置的输出向量，过一个分类头得到二分类的logits。

这一步就是所有教程在讲的东西。

第三步，算损失

logits和真实标签算交叉熵损失，得到一个标量loss。

第四步，反向传播

调用 loss.backward()，PyTorch自动帮你算所有参数的梯度。这一步在代码里就一行，背后的计算图遍历、链式法则应用、梯度累加，全是框架帮你做的。

第五步，更新参数

调用 `optimizer.step()`，根据梯度更新所有参数。用的优化器一般是AdamW，学习率调度用warmup加线性衰减。

第六步，清空梯度，进入下一个batch

调用 `optimizer.zero_grad()`，然后重复第二步到第五步。

你看，整个流程跟训练一个简单的CNN分类器一模一样。区别只在于模型结构不同，参数量不同，其他流程完全相同。

我2019年第一次从零训练一个Transformer模型的时候，最大的感受就是这个东西跟我训练LSTM没啥区别，除了显存占用大了一个数量级、训练时间长了好几倍之外，代码模式完全一样。

我再多说一点，帮你建立一个更完整的认知。

很多人以为Transformer的核心创新是注意力机制。其实不完全对。

注意力机制这个概念在Transformer之前就有了，Bahdanau在2014年的seq2seq论文里就提出了。那时候的注意力是用在encoder-decoder架构里，让decoder在生成每个词的时候能动态关注encoder输出的不同位置。

Transformer的真正创新是什么呢？是用注意力完全替代了循环结构。

在Transformer之前，处理序列数据的标准范式是RNN/LSTM/GRU这类循环神经网络。这些模型有个致命问题：序列处理是串行的，第t步的计算依赖于第t-1步的隐状态，没法并行。这导致训练速度很慢，也没法充分利用GPU的并行计算能力。

Transformer把循环结构干掉了，整个序列的所有位置可以同时计算。一个1024长度的序列，在RNN里要算1024步，在Transformer里自注意力一次矩阵乘法就搞定了。这才是它能scale up到现在这个规模的根本原因。

所以Transformer的革命性不在于注意力机制本身有多神奇，而在于它证明了：不用循环结构也能处理序列，而且效果更好。这打开了一个新世界的大门。

如果你想深入理解这一点，可以去看原始论文 [Attention Is All You Need](#)。这篇文章写得非常清楚，没有故弄玄虚。特别是第三节 Model Architecture 和第四节 Why Self-Attention，把设计动机讲得很透彻。

既然聊到这了，我顺便讲讲Q、K、V这个设计的直觉是什么，帮你从另一个角度理解。

自注意力的核心操作是：对于序列中的每个位置，计算它应该关注其他哪些位置，然后把信息聚合起来。

最朴素的做法是直接原始向量算相似度，两个位置的向量点积大，就说明它们相关性高，应该多关注。但这样太简单了，表达能力不够。

Q、K、V的设计引入了三个独立的线性变换，相当于让模型学习三个不同的投影空间：Query空间、Key空间、Value空间。

Query代表我要查询什么信息，Key代表我能提供什么信息用于匹配，Value代表我实际能提供的内容。Query和Key在各自空间里算相似度，然后用这个相似度去加权Value。

这三个空间是独立学习的，给了模型更大的灵活性。它可以学到：某两个词在语义上相关（Query-Key匹配分高），但提取的特征不同（各自的Value不同）。

多头注意力更进一步，有8个或者16个独立的头，每个头学习不同的关注模式。有的头可能专注于语法关系，有的头专注于语义关系，有的头专注于位置距离。最后把所有头的结果拼起来，再投影回原始维度。

这个设计非常优雅，而且确实work。后来的研究发现，不同的头真的学到了不同类型的attention pattern。你可以用一些可视化工具看看，比如 BertViz 这个项目，把预训练BERT的注意力权重可视化出来，能看到很有意思的规律。

但说到底，这些只是模型结构的设计选择。训练的时候， W_q 、 W_k 、 W_v 这些矩阵就是普通的参数，用反向传播更新，跟其他参数没有区别。

我一直觉得，对于Transformer这种稍微复杂一点的模型，光看教程是不够的，必须自己动手写一遍。

不是说你要从零写一个工业级的实现，那没必要也没意义。但是用numpy或者纯PyTorch手写一个最简版本，把forward和backward都过一遍，对于理解模型结构有巨大的帮助。

我推荐两个资源：

一个是前面提到的 The Annotated Transformer，哈佛NLP组出的，用PyTorch实现了原始论文的所有细节，代码和注释对照着看非常清晰。这个适合你想完整理解Transformer架构的情况。

另一个是Andrej Karpathy的 minGPT 和 nanoGPT 项目，代码极其简洁，实现了一个最小化的GPT模型。如果你想快速上手玩起来，这个是最佳选择。Karpathy还配了一个两小时的视频教程叫 Let's

build GPT: from scratch, in code, spelled out，讲得非常好，我看完之后对于GPT系列的理解提升了一大截。

你可以按这个顺序来：先跟着Karpathy的视频把nanoGPT跑通，理解decoder-only的架构。然后看The Annotated Transformer，理解encoder-decoder的完整架构。最后如果有兴趣，去看看HuggingFace的transformers库源码，看看工业级实现是怎么处理各种细节的。

这三步走下来，Transformer对你来说就不再神秘了。

到这一步，如果你已经理解了基础的训练机制，接下来可以关注一些进阶问题：

大模型的训练稳定性

模型大了之后训练会变得不稳定，loss容易spike甚至nan。这时候需要一些trick：Pre-LayerNorm而不是Post-LayerNorm、参数初始化的scale要根据层数调整、学习率warmup、梯度裁剪等等。这些不是什么神秘的黑魔法，都是工程上踩坑踩出来的经验。

显存优化

Transformer的显存占用跟序列长度的平方成正比，因为要存注意力矩阵。长序列训练时显存很容易爆。解决方案包括梯度检查点（用计算换显存）、Flash Attention（通过kernel fusion和tiling减少显存访问）、混合精度训练等。这些是做大模型训练必须要了解的技术。

各种高效注意力变体

标准自注意力的计算复杂度是 $O(n^2)$ ，对于长序列不友好。所以有很多改进版本：Sparse Attention只计算部分位置对、Linear Attention把复杂度降到 $O(n)$ 、Sliding Window Attention只看局部窗口等。这些变体在特定场景下有用，但也有各自的trade-off。

位置编码的演进

原始Transformer用的是固定的正弦位置编码，后来BERT改成了可学习的位置编码。再后来ALiBi用位置偏移代替位置编码，RoPE用旋转矩阵编码相对位置。这些改进主要是为了解决外推问题，让模型能处理比训练时更长的序列。

这些进阶话题每一个都可以展开很多，但都不改变一个基本事实：Transformer就是一个神经网络，用反向传播训练，跟你之前学的东西是一脉相承的。

回到你的问题，你说看了很多资料还是不理解Transformer。我觉得你其实已经理解了最核心的东西，就是神经网络通过反向传播调整参数来拟合数据。Transformer也是这样的。

你之所以觉得不理解，是因为教程们给你制造了一个错误的期待：你以为Transformer有什么特殊的训练方式，但其实没有。

Q、K、V的公式只是描述了注意力计算的前向过程，告诉你数据怎么从输入变成输出。训练的时候， W_q 、 W_k 、 W_v 这些矩阵跟普通的全连接层权重一样，用反向传播算梯度，用优化器更新参数，nothing special。

如果你还想要一个更形象的理解：可以把Transformer想象成一个超大号的矩阵乘法堆叠，中间夹着一些非线性函数。数据从左边进去，经过一堆矩阵变换，从右边出来。训练就是调整这些矩阵里的数字，让输出尽可能接近你想要的结果。

所以总结一下：

Transformer的训练机制跟普通神经网络没有区别，就是反向传播加梯度下降。你觉得不理解，是因为大多数教程只讲前向传播不讲训练，造成了信息缺口。

Transformer里的可训练参数包括：词嵌入矩阵、位置编码（如果是可学习的话）、每一层的 $W_q/W_k/W_v/W_o$ 、FFN的权重偏置、LayerNorm的参数。这些参数的梯度通过链式法则计算，跟任何其他可微网络一样。

想真正理解Transformer，最好的办法是自己动手实现一遍。推荐Karpathy的nanoGPT和哈佛的The Annotated Transformer，代码都很清晰。

不要被注意力机制的新颖性吓到，它只是一种计算方式，不改变神经网络训练的基本范式。你之前积累的所有关于神经网络的知识，在这里全都用得上。

就这样。去写代码吧。

 送礼物

还没有人送礼物，鼓励一下作者吧

[编辑于 2025-12-04 20:27](#)

▲赞同 2922



●77 条评论

🔗分享

★收藏

♥喜欢

...